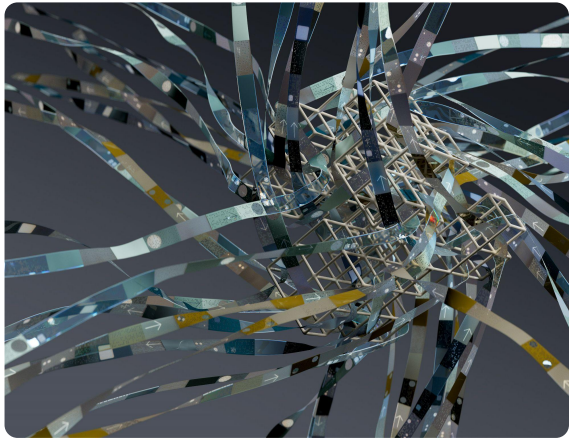
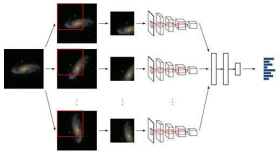
ACDL 2025 – Sander Dieleman

Iterative refinement & diffusion (part 1)

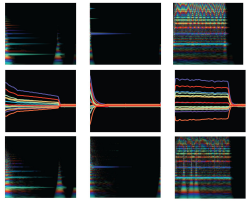
Iterative refinement & diffusion (part 2)



Training neural networks

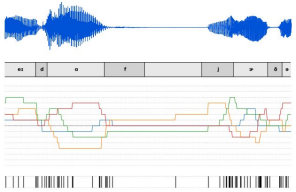


Galaxy morphology prediction



NSynth: WaveNet autoencoders

Piano Genie

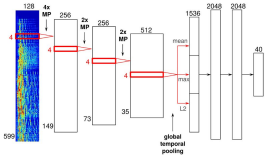


Variable-rate discrete representation learning

Imagen 4 & Veo 3: diffusion at scale

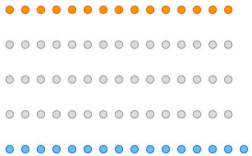
2014 2015 2016 2017 2018 2019 2020 2021 2022 2025

Music recommendation

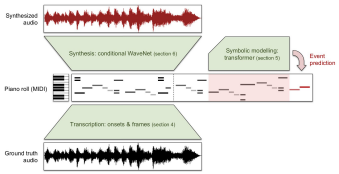


Spotify

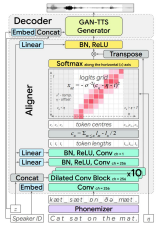
WaveNet: a generative model for raw audio



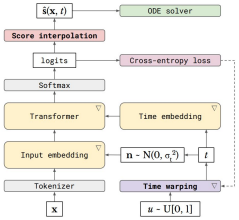
MAESTRO & hierarchical WaveNets for music



End-to-end adversarial text-to-speech



Continuous diffusion for categorical data



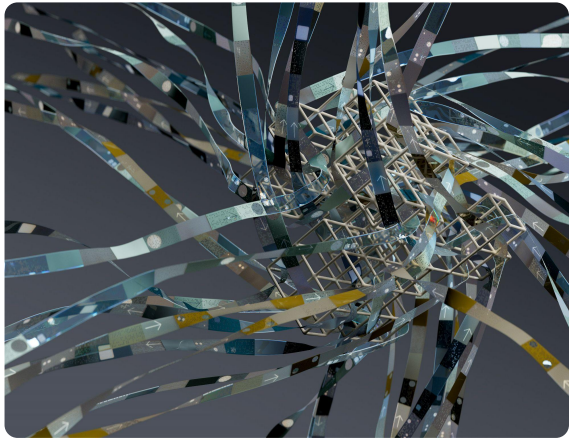
Musings on typicality	https://sander.ai/2020/09/01/typicality.html
Diffusion models are autoencoders	https://sander.ai/2022/01/31/diffusion.html
Guidance: a cheat code for diffusion models	https://sander.ai/2022/05/26/guidance.html
Diffusion language models	https://sander.ai/2023/01/09/diffusion-language.html
Perspectives on diffusion	https://sander.ai/2023/07/20/perspectives.html
The geometry of diffusion guidance	https://sander.ai/2023/08/28/geometry.html
The paradox of diffusion distillation	https://sander.ai/2024/02/28/paradox.html
Noise schedules considered harmful	https://sander.ai/2024/06/14/noise-schedules.html
Diffusion is spectral autoregression	https://sander.ai/2024/09/02/spectral-autoregression.html
Generative modelling in latent space	https://sander.ai/2025/04/15/latents.html

Three lectures

AM PM

Iterative refinement & diffusion (part 1)

Iterative refinement & diffusion (part 2)



Training neural networks

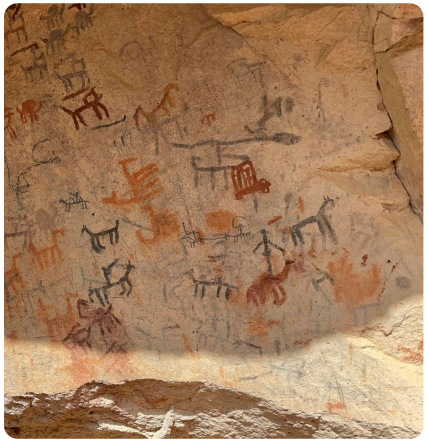
Google DeepMind

Iterative refinement & diffusion

(part 1: intuition & perspectives)

Sander Dieleman
Research Scientist, Google DeepMind

Overview



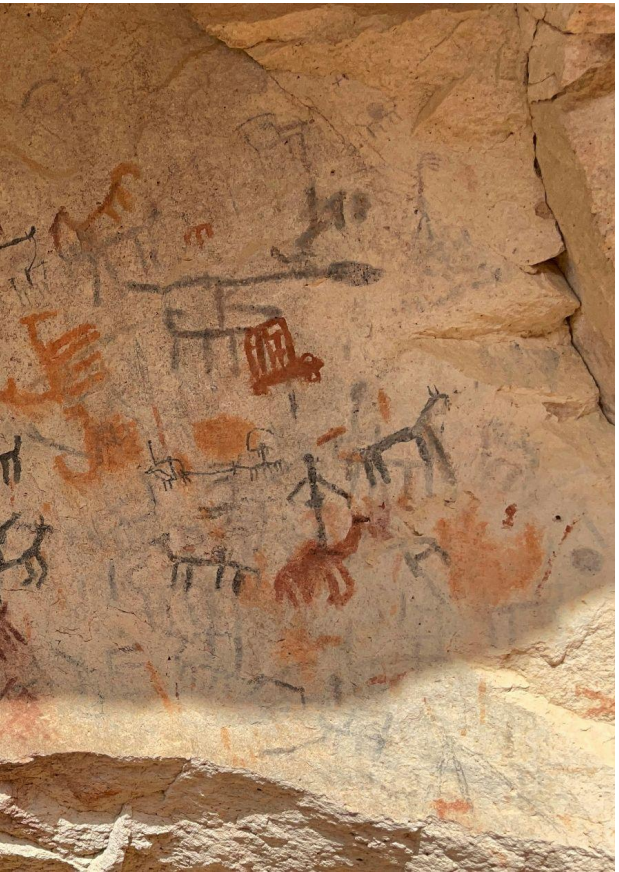
I. Generative modelling

II. Iterative refinement

III. Diffusion models

IV. Examples

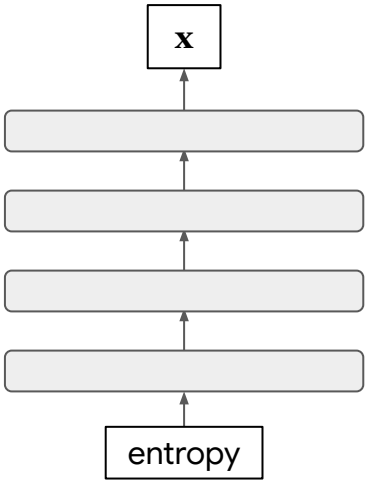
Google DeepMind



I. Generative modelling

Generative modelling: the probabilistic perspective

$$x \sim p(x)$$



Explicit: autoregression, flows, VAEs, ...

Implicit: GANs, ...

Conditional generative models

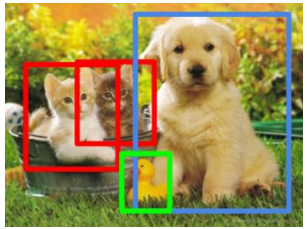
Control model output using a **control signal**: $p(x|c)$ vs. $p(x)$

sparsely conditioned → densely conditioned

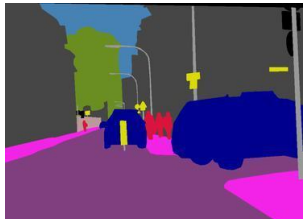
class labels

y = “cat”

bounding boxes

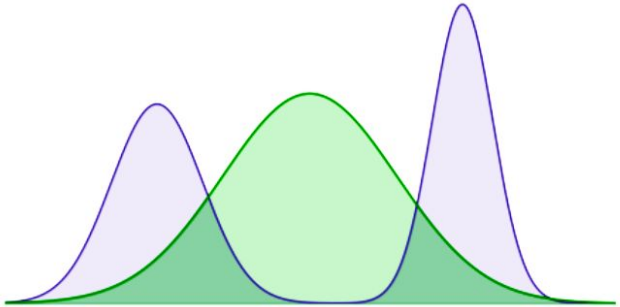


segmentation



grayscale image (colorisation)

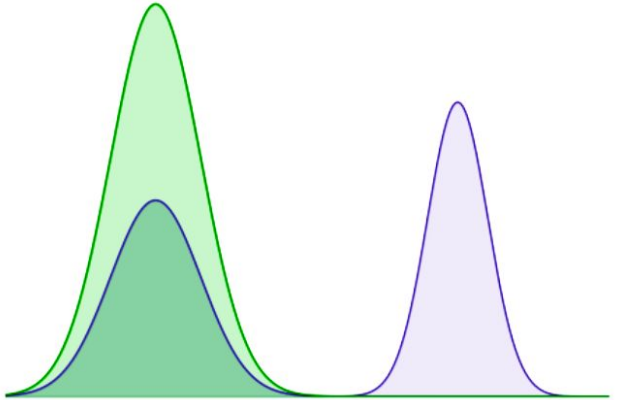
Mode-covering vs. mode-seeking behaviour



mode-covering

focus on **diversity**

e.g. likelihood-based models



mode-seeking

focus on **realism**

e.g. adversarial models

Google DeepMind

II. Iterative refinement

Divide and conquer

- **Iterative refinement**: break up the generative procedure into simpler steps
- All steps are similar
- Learn to perform a single step at a time
- Apply model repeatedly to generate
- Almost **all modern generative models operate in this way**
- Challenges:
  - Train-test discrepancies
  - High cost of generation

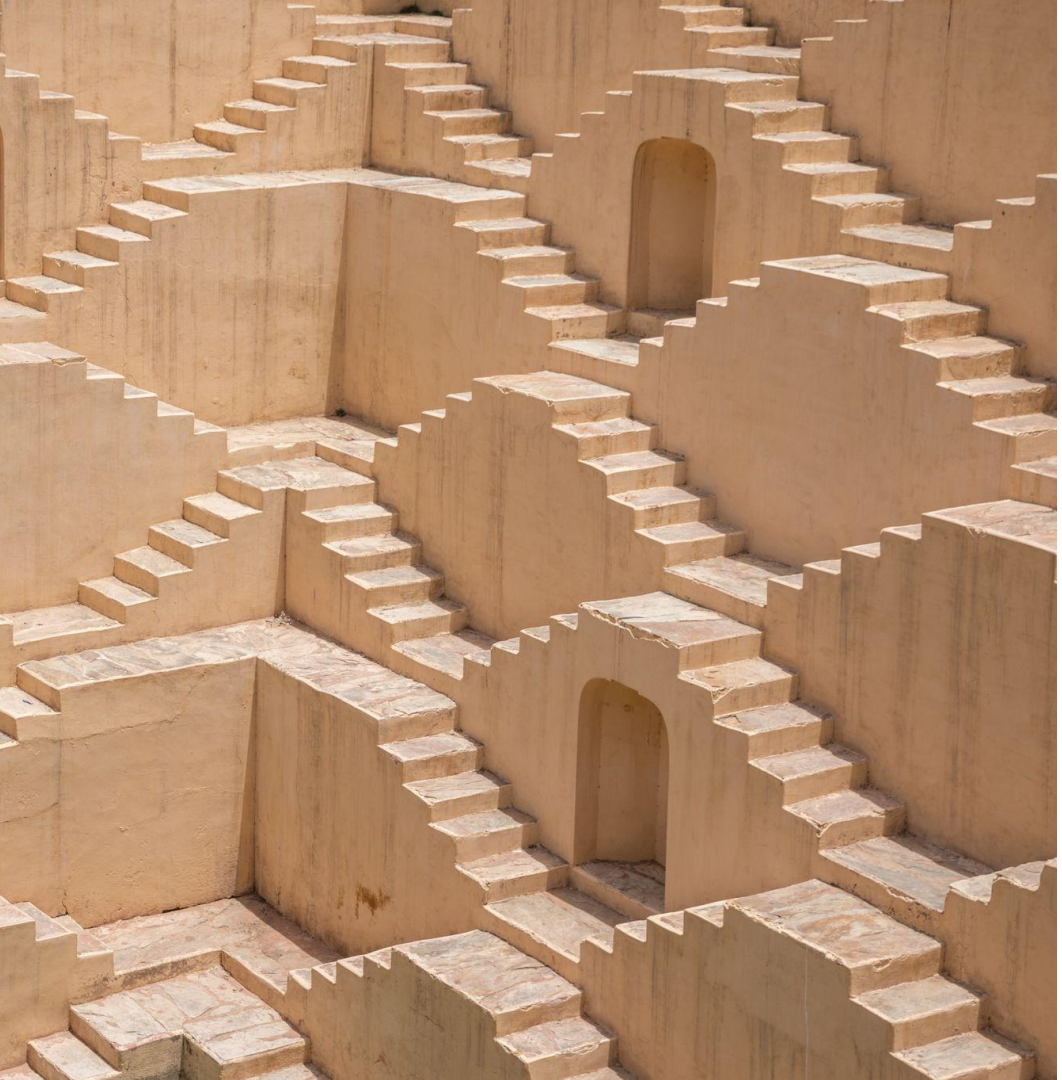
Two approaches to iterative refinement

Autoregression: step-by-step

Turn everything into a 1D sequence,
generate it one step at a time

Diffusion: iterative denoising

Gradually add noise until all information is destroyed,
then learn to invert this procedure step by step



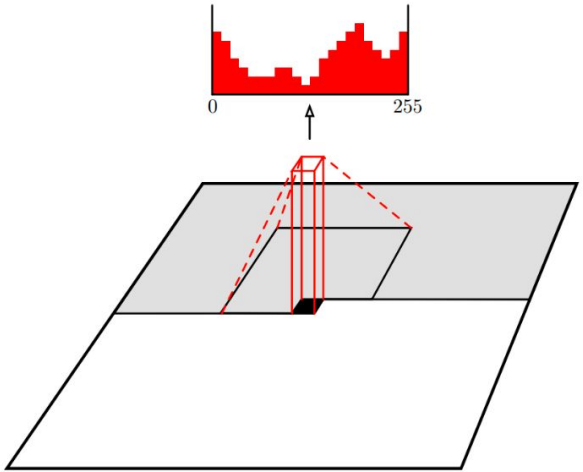
Autoregression: step-by-step

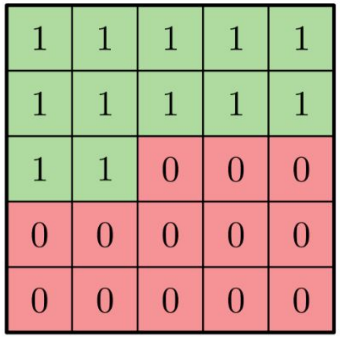
chain rule of probability

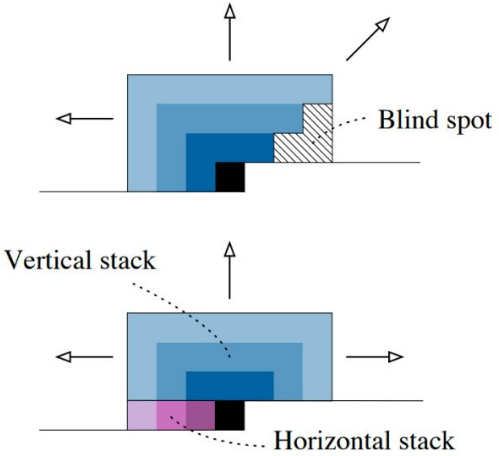
$$p(x) = \prod_i p(x_i | x_{<i})$$

- factorise $p(x)$ into **sequential conditionals**
- $p(x_i | x_{<i})$ are simple scalar distributions
- use the **same model** for all of them

Autoregression in pixel space: PixelRNN & PixelCNN

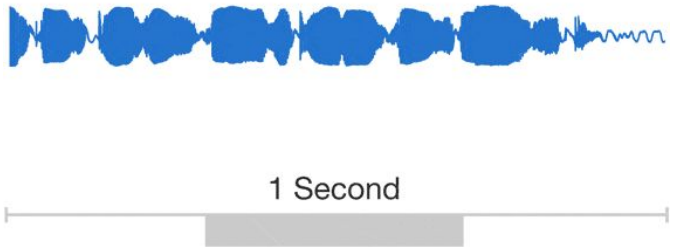


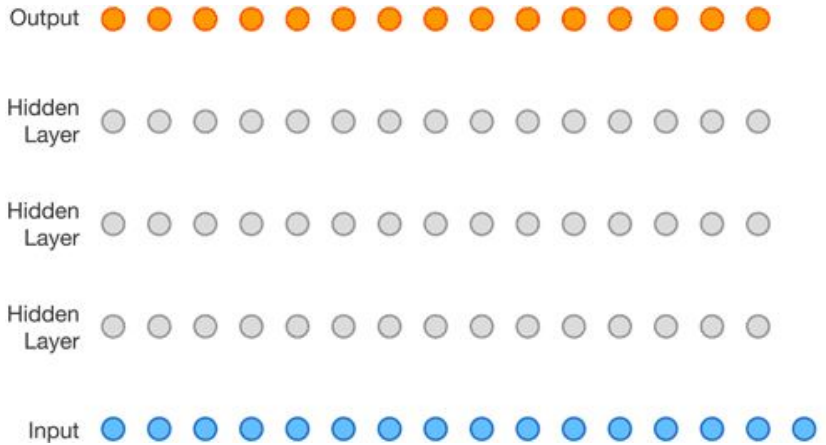




van den Oord et al. 'Pixel Recurrent Neural Networks' (2016)
van den Oord et al. 'Conditional Image Generation with PixelCNN Decoders' (2016)

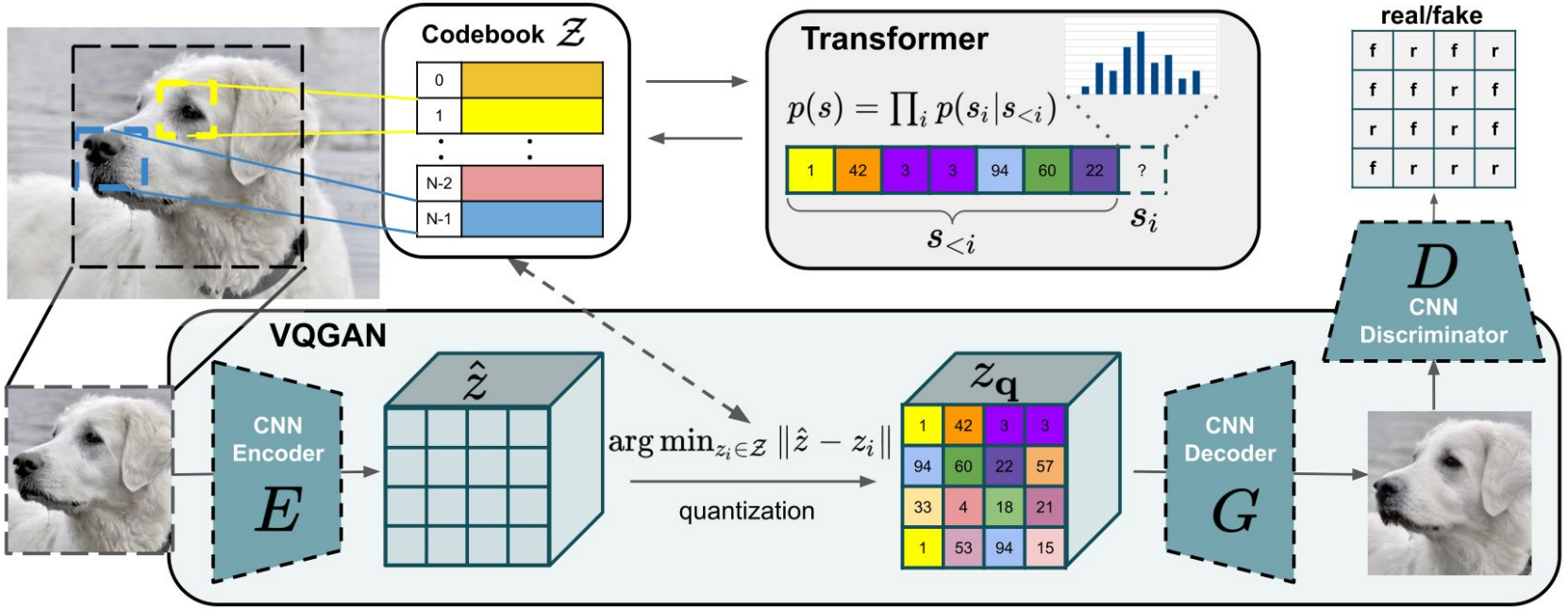
Autoregression in amplitude space: WaveNet & SampleRNN





van den Oord et al. 'WaveNet: a Generative Model for Raw Audio' (2016)
Mehri et al. 'SampleRNN: An Unconditional End-to-End Neural Audio Generation Model' (2016)

Autoregression in latent space: VQ-VAE & VQGAN



van den Oord et al. 'Neural Discrete Representation Learning' (2017)
Esser et al. 'Taming Transformers for High-Resolution Image Synthesis' (2020)

Autoregression in latent space: representation learning in service of generative modelling

*van den Oord et al. 'Neural Discrete Representation Learning' (2017) [**VQ-VAE**]*

Dieleman et al. 'The challenge of realistic music generation: modelling raw audio at scale' (2018)

De Fauw et al. 'Hierarchical Autoregressive Image Models with Auxiliary Decoders' (2019)

Razavi et al. 'Generating Diverse High-Fidelity Images with VQ-VAE-2' (2019)

*Dhariwal et al. 'Jukebox: A Generative Model for Music' (2020) [**Jukebox**]*

*Esser et al. 'Taming Transformers for High-Resolution Image Synthesis' (2020) [**VQGAN**]*

Lakhotia et al. 'Generative Spoken Language Modeling from Raw Audio' (2021)

*Ramesh et al. 'Zero-Shot Text-to-Image Generation' (2021) [**DALL-E**]*

Dieleman et al. 'Variable-rate discrete representation learning' (2021)

*Borsos et al. 'AudioLM: a Language Modeling Approach to Audio Generation' (2022) [**AudioLM**]*

*Kreuk et al. 'AudioGen: Textually Guided Audio Generation' (2022) [**AudioGen**]*

*Yu et al. 'Scaling Autoregressive Models for Content-Rich Text-to-Image Generation' (2022) [**Parti**]*

...

Google DeepMind

III. Diffusion models

Diffusion: iterative denoising

Diffusion: forward process

x_0

training data

x_t

noisy data

x_∞

Gaussian noise

$$\delta \sim \mathcal{N}(0, \sigma^2)$$

Diffusion: forward process

x_0

training data

x_t

noisy data

x_∞

Gaussian noise

$$x_t = x_o + \sigma(t)\cdot\varepsilon$$

$$\varepsilon \sim \mathcal{N}(0,1)$$

Diffusion: forward process

x_o

training data

x_t

noisy data

x_T

Gaussian noise

$$x_t = \alpha(t) \cdot x_o + \sigma(t) \cdot \varepsilon$$

$$\varepsilon \sim \mathcal{N}(0, 1)$$

Diffusion: forward process

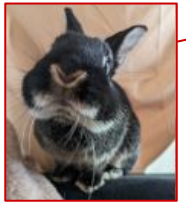
- $\sigma(t)$ is the **noise schedule**
- controls the **rate of corruption** over the course of the process
- Several choices for $\alpha(t)$:
  - variance-preserving (VP)
    $\alpha = \sqrt{(1 - \sigma^2)}$
  - variance-exploding (VE)
    $\alpha = 1$
  - rectified flow, flow matching (RF)
    $\alpha = 1 - \sigma$

Diffusion: backward process

$$\delta = ???$$

Diffusion: backward process

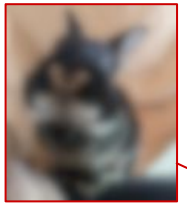
x_t



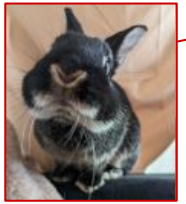
x_0

Diffusion: backward process

x_t



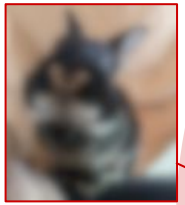
$\hat{x}_0$

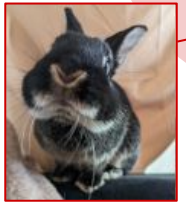


x_0

predict x_0

Diffusion: backward process





$\hat{x}_0$

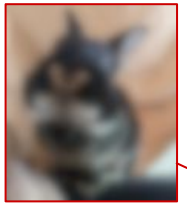
x_0

x_t

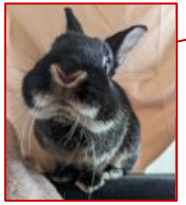
predict x_0

Diffusion: backward process

x_t



$\hat{x}_0$



x_0

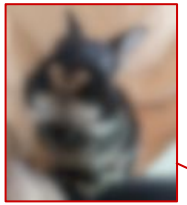
take a small step

Diffusion: backward process

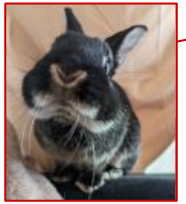
x_{t-1}

ξ

x_t



$\hat{x}_0$



x_0

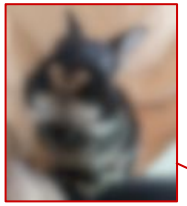
add some noise

Diffusion: backward process

x_{t-1}

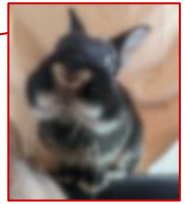
ξ

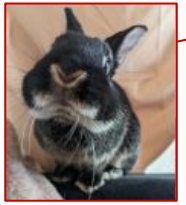
x_t



$\hat{x}_0$

$\hat{x}_0$

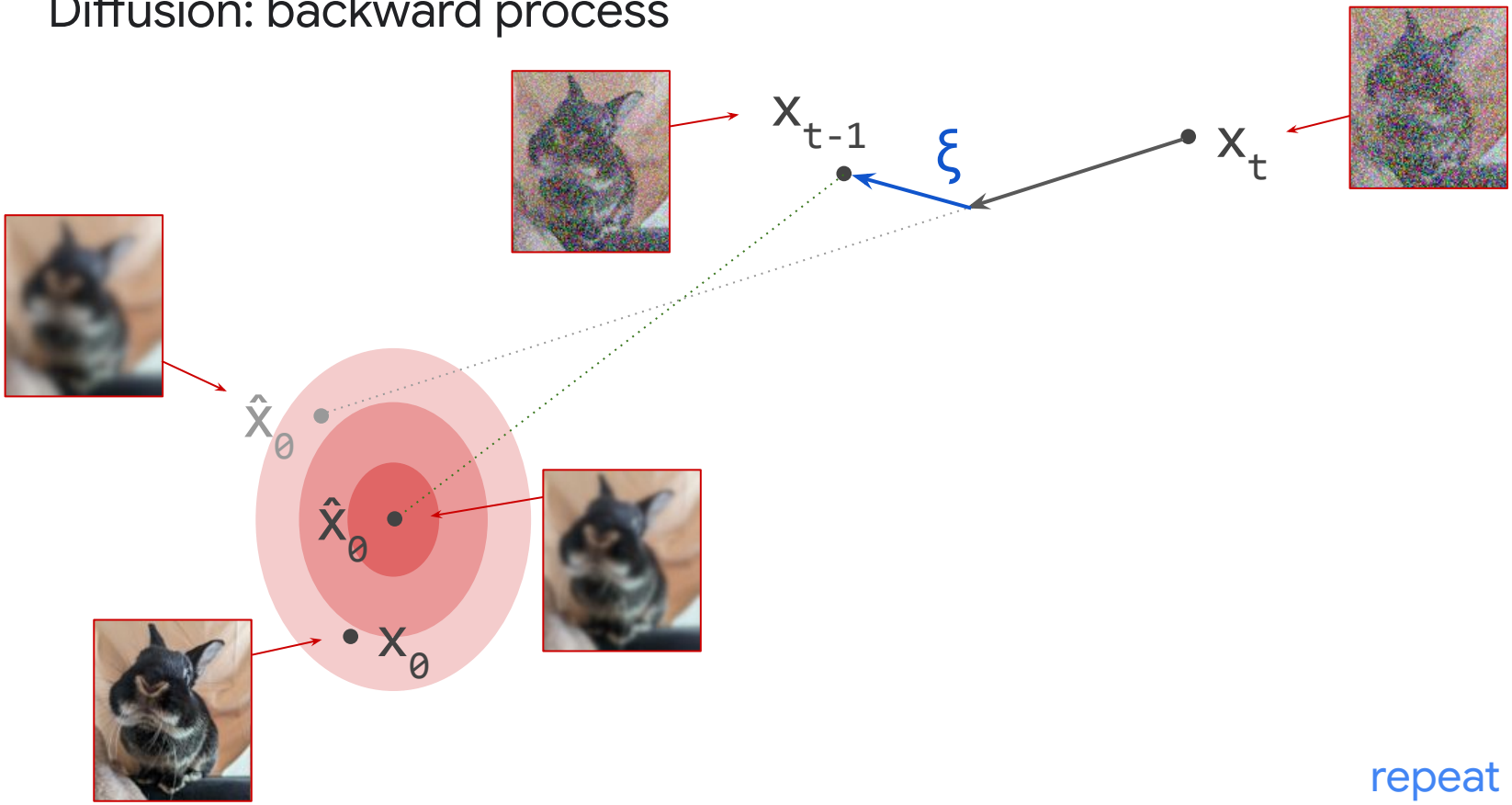




x_0

repeat

Diffusion: backward process



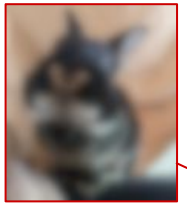
repeat

Diffusion: backward process

x_{t-1}

ξ

x_t

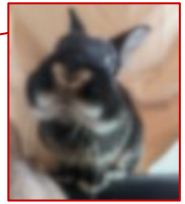


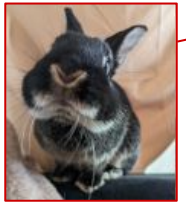
ξ

x_{t-2}

$\hat{x}_0$

$\hat{x}_0$





x_0

repeat

Diffusion: predict ε instead of x_0?

- x_t is a linear combination of x_0 and ε
- Diffusion models predict x_0 from x_t
- ... but predicting ε is also an option
- Given x_t, we can convert a prediction for ε into one for x_0, and vice versa
- Other linear combinations of ε and x_0 are also possible:
  - v-prediction: $v = \alpha(t)\cdot\varepsilon - \sigma(t)\cdot x_0$
  - Flow matching: $\varepsilon - x_0$

Salimans & Ho 'Progressive Distillation [..]' (2022)
Lipman et al. 'Flow matching [..]' (2022)

$$x_t = \alpha(t)\cdot x_0 + \sigma(t)\cdot\varepsilon$$

Diffusion: predict x_{t-1} instead of x_0?

- x_t is a linear combination of x_0 and ε
- x_{t-1} is **also** a linear combination of x_0 and ε
- Use substitution to eliminate ε
- Given x_t, we can convert a prediction for x_0 into one for x_{t-1}, and vice versa
- All of x_0, x_{t-1}, ε are **linearly related** when x_t is known
- All prediction targets are **equivalent** (except for relative weighting of noise levels, see later)

$$\mathbf{x}_t = \alpha(t) \cdot \mathbf{x}_0 + \sigma(t) \cdot \varepsilon$$

$$\mathbf{x}_{t-1} = \alpha(t-1) \cdot \mathbf{x}_0 + \sigma(t-1) \cdot \varepsilon$$

$$\Rightarrow \varepsilon = \frac{1}{\sigma(t)} \left(\mathbf{x}_t - \alpha(t) \cdot \mathbf{x}_0 \right)$$

$$\Rightarrow \mathbf{x}_{t-1} = \alpha(t-1) \cdot \mathbf{x}_0 + \frac{\sigma(t-1)}{\sigma(t)} \cdot \left(\mathbf{x}_t - \alpha(t) \cdot \mathbf{x}_0 \right)$$

$$\Rightarrow \mathbf{x}_{t-1} = \left(\alpha(t-1) - \frac{\sigma(t-1)}{\sigma(t)} \alpha(t) \right) \cdot \mathbf{x}_0 + \frac{\sigma(t-1)}{\sigma(t)} \cdot \mathbf{x}_t$$

Diffusion training: summary

For each training example x_0:

- Sample random time step t
- Corrupt x_0 to get x_t

$$x_t = \alpha(t) \cdot x_0 + \sigma(t) \cdot \varepsilon$$

- Predict x_0 (or ε) from x_t

$$\hat{x}_0 = f(x_t, t) \text{ or } \hat{\varepsilon} = f(x_t, t)$$

- Minimise squared prediction error

$$\min (\hat{x}_0 - x_0)^2 \text{ or } \min (\hat{\varepsilon} - \varepsilon)^2$$

- Repeat

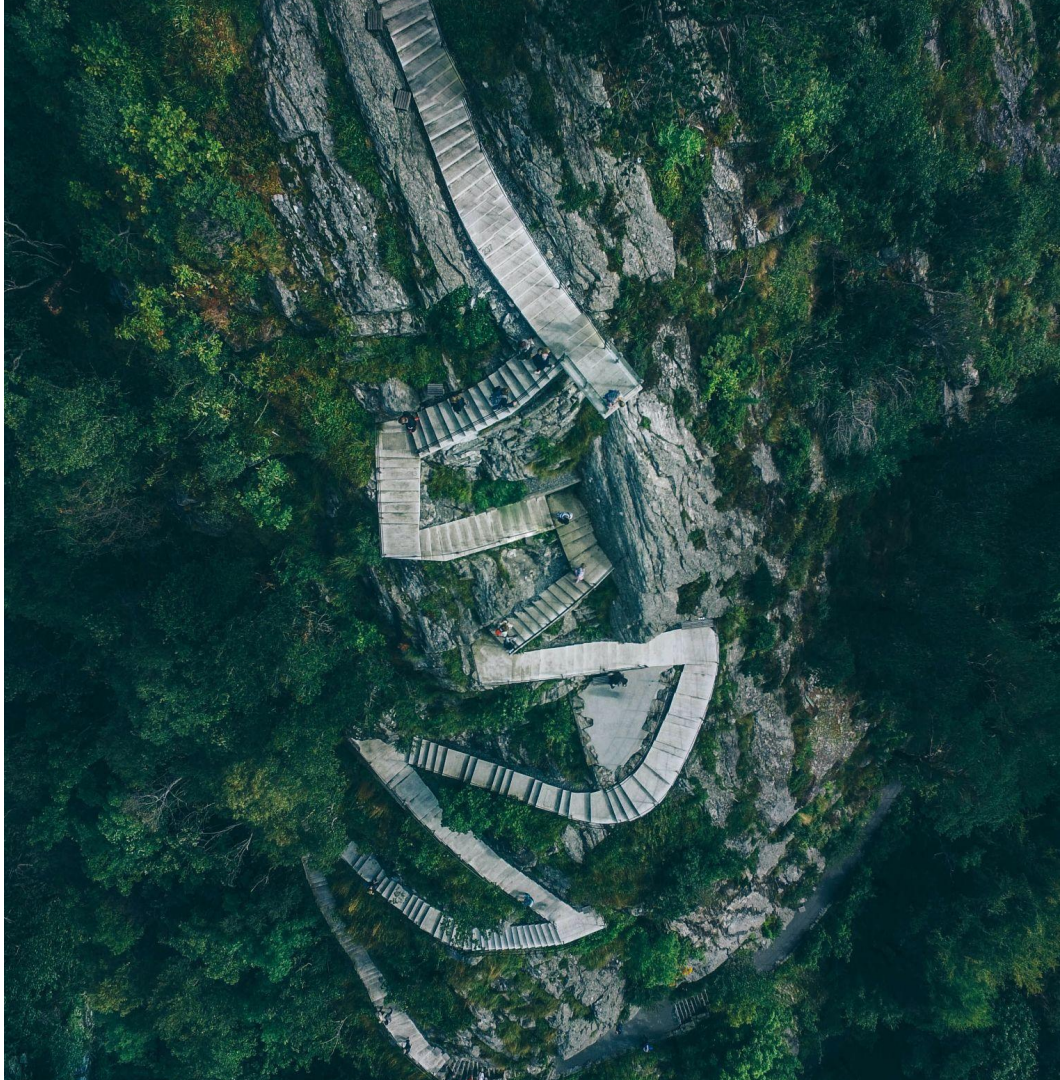
Diffusion sampling: summary

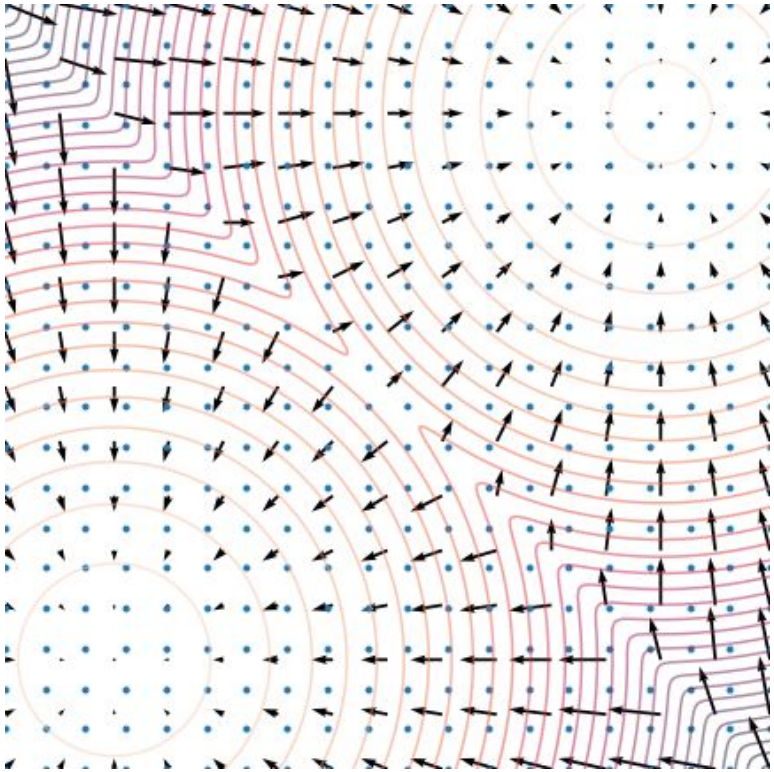
At each sampling time step t:

- Predict x_0 (or ε) from x_t

$$\hat{x}_0 = f(x_t, t) \text{ or } \hat{\varepsilon} = f(x_t, t)$$

- Take a small step in the predicted direction to partially denoise $x_t \rightarrow x_{t-1}$
- Optionally add back some noise (only in some algorithms)
- Repeat





Diffusion: score matching

score function

$$\nabla_x \log p(x)$$

- Gradient points to regions of high density
- If we know the gradient, we know $p(x)$
- Predict gradients, minimise the **MSE**

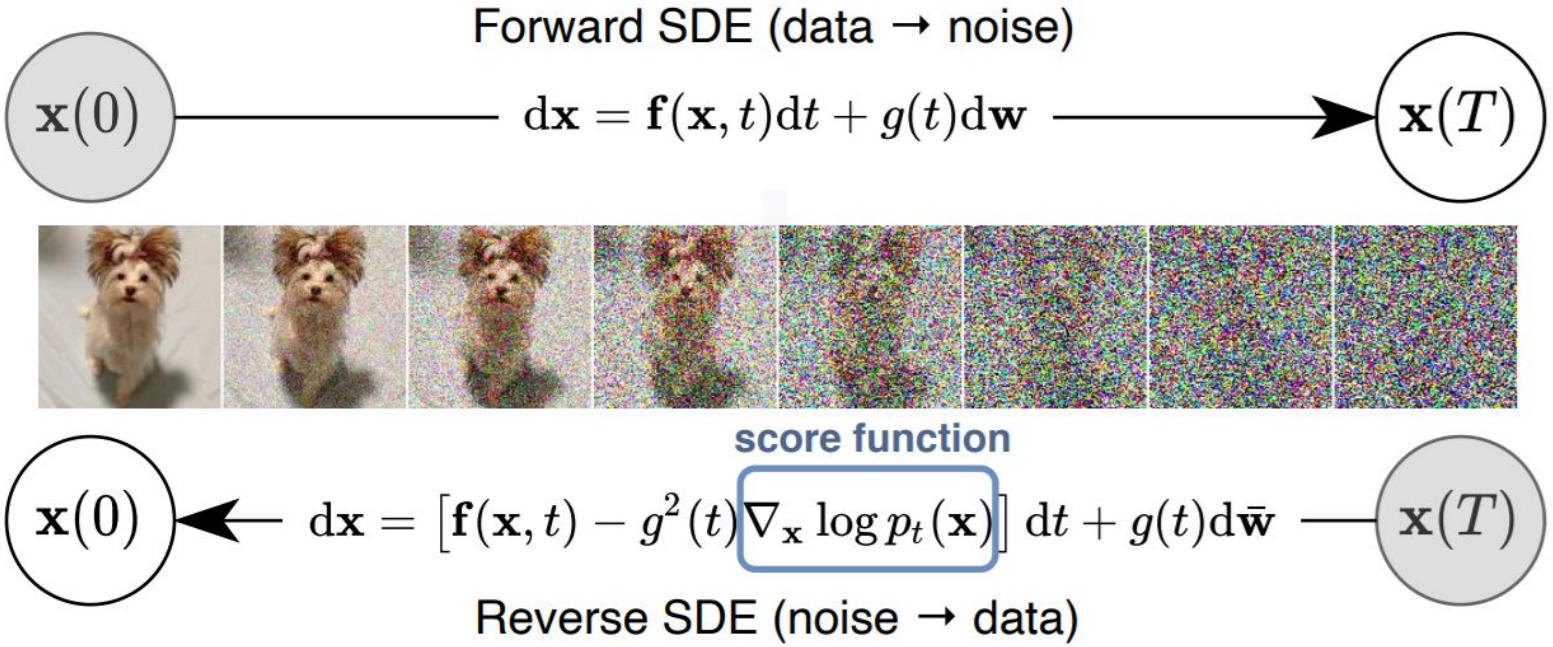
Hyvärinen 'Estimation of Non-Normalized Statistical Models by Score Matching' (2005)
Vincent 'A Connection Between Score Matching and Denoising Autoencoders' (2010)

Denoising score matching

$$\mathbb{E}_{x_t}\left[(s_t(x_t) - \nabla_{x_t} \log p(x_t))^2\right] \sim \mathbb{E}_{x_0}\left[\mathbb{E}_{x_t|x_0}\left[(s_t(x_t) - \nabla_{x_t} \log p(x_t|x_0))^2\right]\right]$$

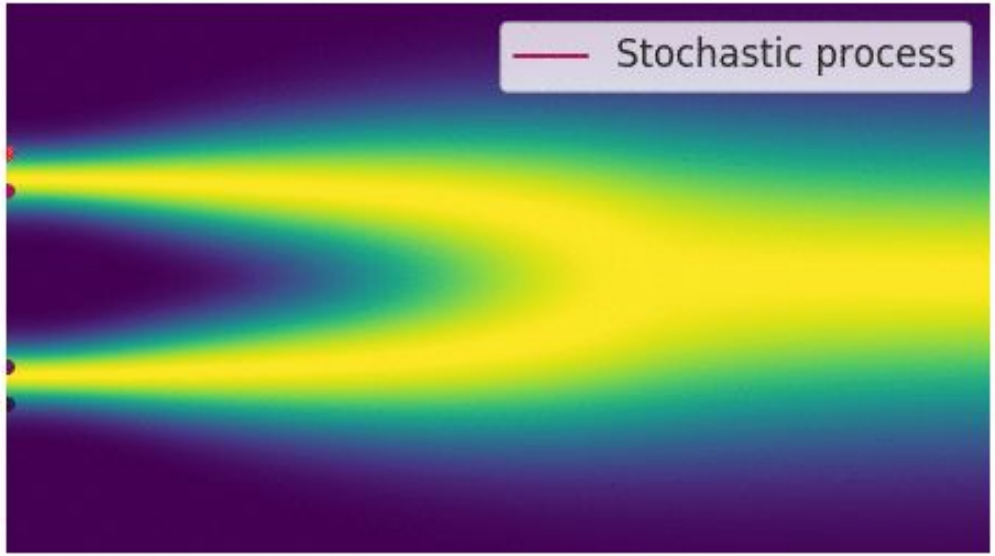
- We don't have "ground truth" for $\nabla_x \log p(x)$
- ... but $\nabla_x \log p(x|x_o)$ is easy: it's $\varepsilon / \sigma(t)$!
- Loss functions **have the same minimum**

Diffusion: stochastic differential equations



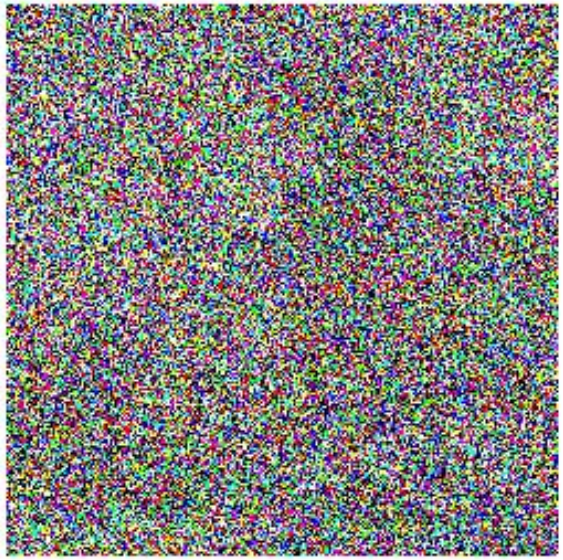
Song et al. 'Score-Based Generative Modeling through Stochastic Differential Equations' (2020)

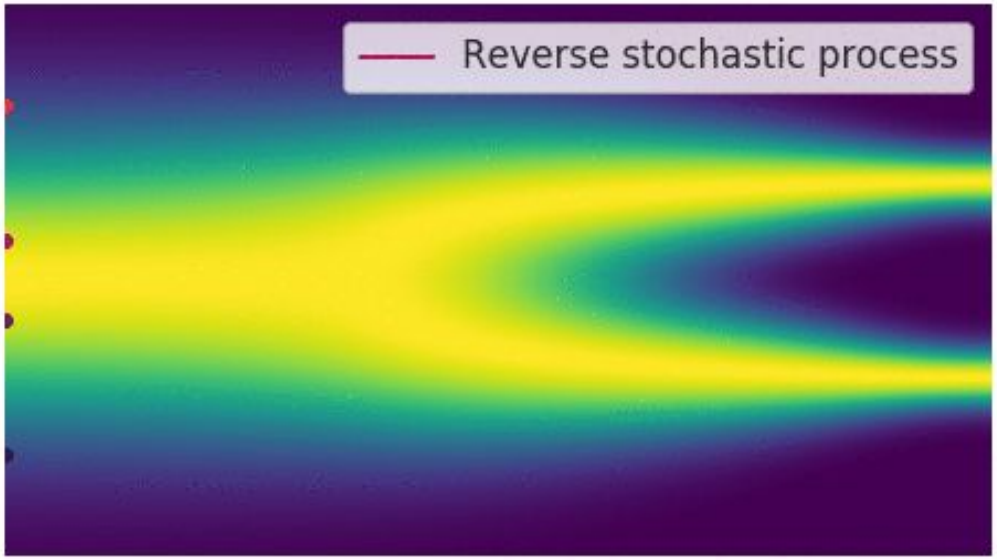
Diffusion: stochastic differential equations



Song et al. 'Score-Based Generative Modeling through Stochastic Differential Equations' (2020)

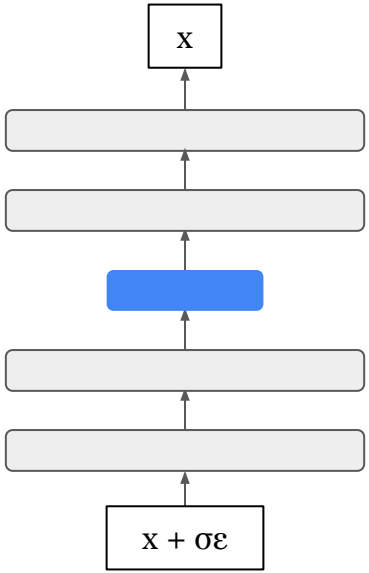
Diffusion: stochastic differential equations



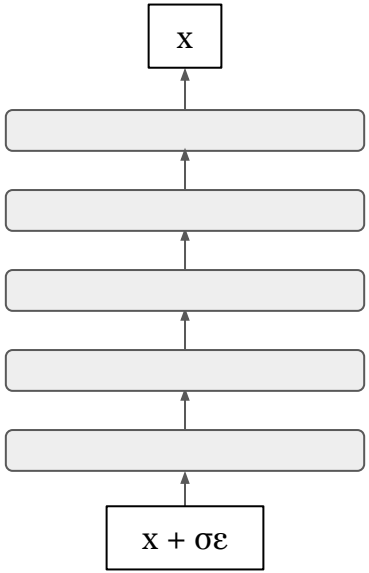


Song et al. 'Score-Based Generative Modeling through Stochastic Differential Equations' (2020)

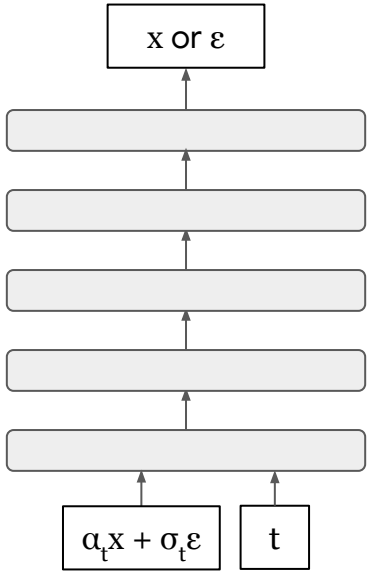
Diffusion models are autoencoders



denoising autoencoder



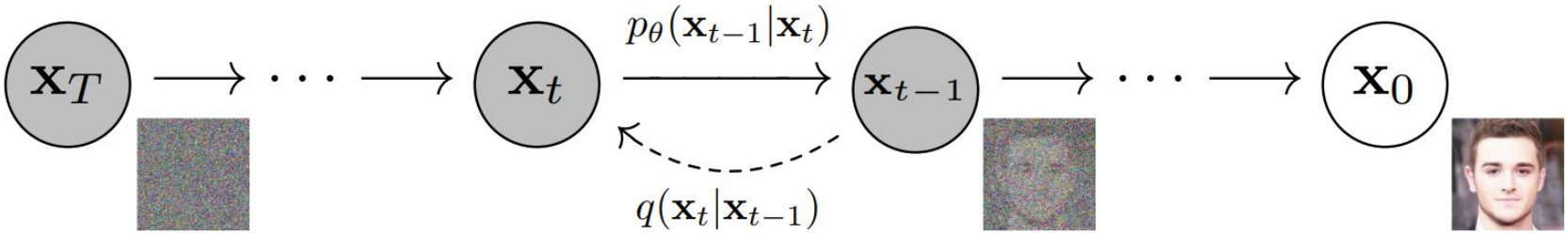
denoiser



diffusion model

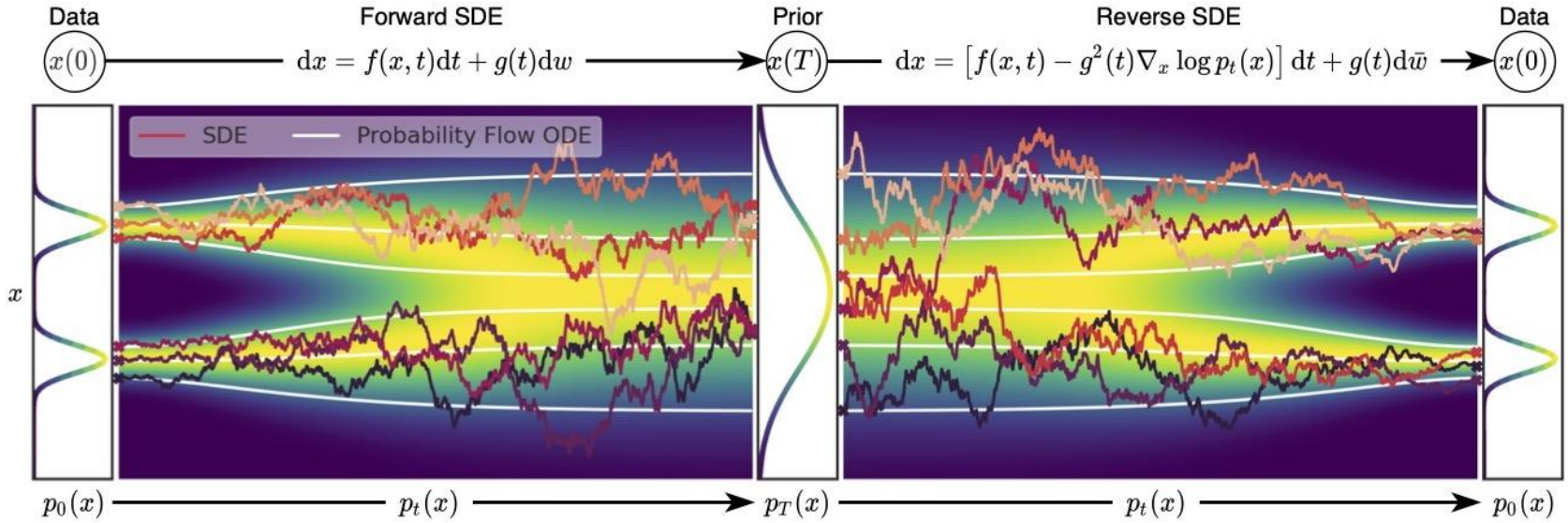
https://sander.ai/2022/01/31/diffusion.html

Diffusion models are deep latent variable models



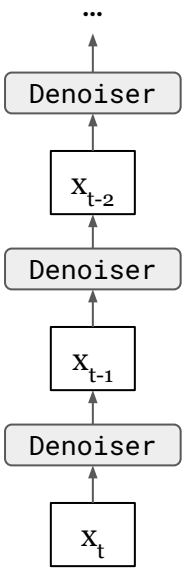
Sohl-Dickstein et al. 'Deep Unsupervised Learning using Nonequilibrium Thermodynamics' (2015)
Ho et al. 'Denoising Diffusion Probabilistic Models' (2020)

Diffusion models are flow-based models



Song et al. 'Score-Based Generative Modeling through Stochastic Differential Equations' (2020)

Diffusion models are recurrent neural networks (RNNs)



unrolled sampling loop
(ODE solver)

- Train with backprop through time (BPTT)
  → **continuous normalising flow (CNF)**
- Train with score matching
  → **diffusion model**
- **Deep RNN trained without BPTT!** 😮

Grathwohl et al. 'FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models' (2018)

Google DeepMind

IV. Examples

Image generation at scale: Imagen 4
https://deepmind.google/models/imagen/

Image generation at scale: Imagen 4
https://deepmind.google/models/imagen/

Image generation at scale: Imagen 4

https://deepmind.google/models/imagen/

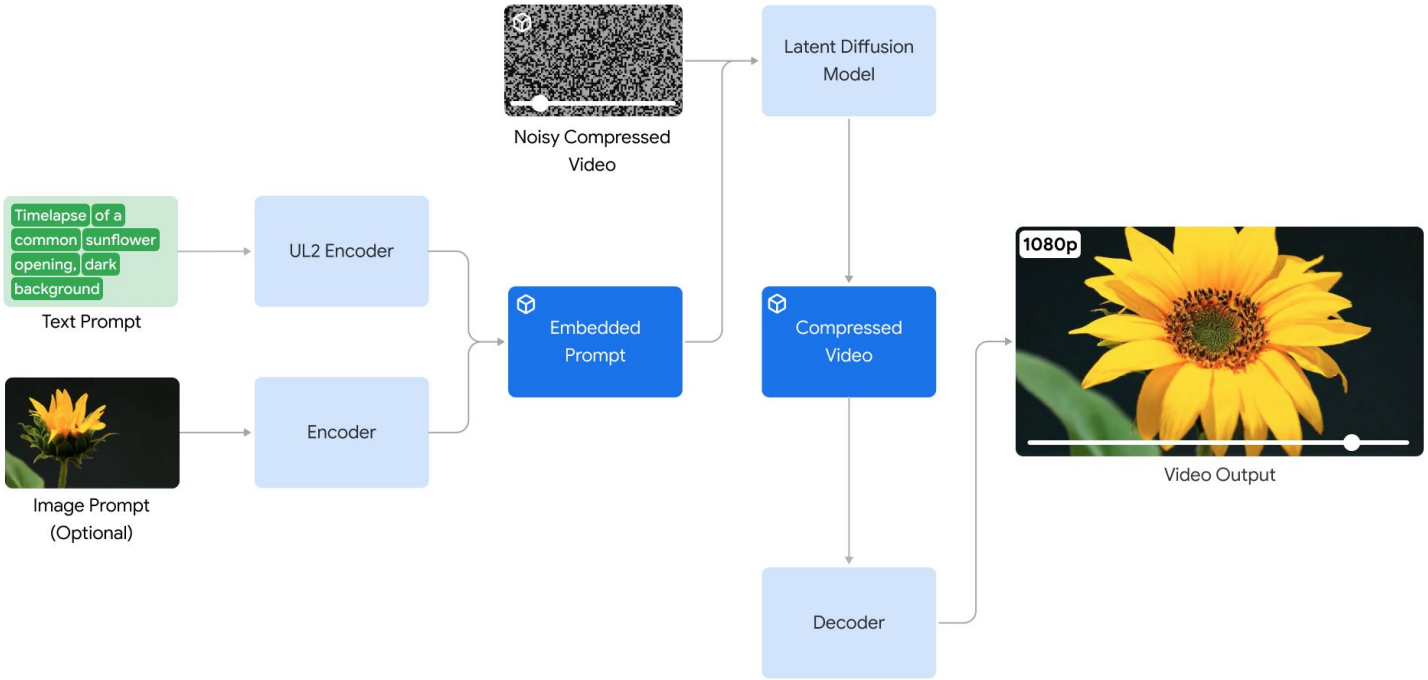
Image generation at scale: Imagen 4
https://deepmind.google/models/imagen/

Image generation at scale: Imagen 4

https://deepmind.google/models/imagen/

Video generation at scale: Veo 3

https://deepmind.google/technologies/veo/



Thank you! Grazie!

Blog: https://sander.ai/

Twitter/X: @sedielem

BlueSky: @sedielem.bsky.social

Email: sanderdieleman@gmail.com

Google DeepMind

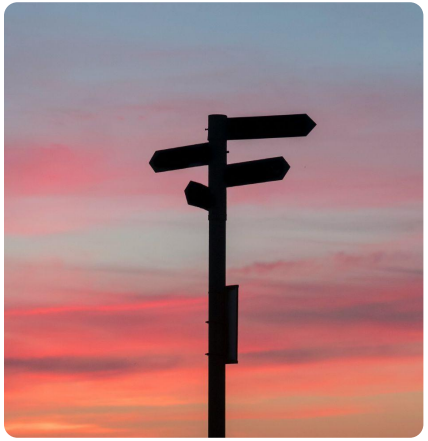
Google DeepMind

Iterative refinement & diffusion

(part 2: guidance, distillation, advanced topics)

Sander Dieleman
Research Scientist, Google DeepMind

Overview



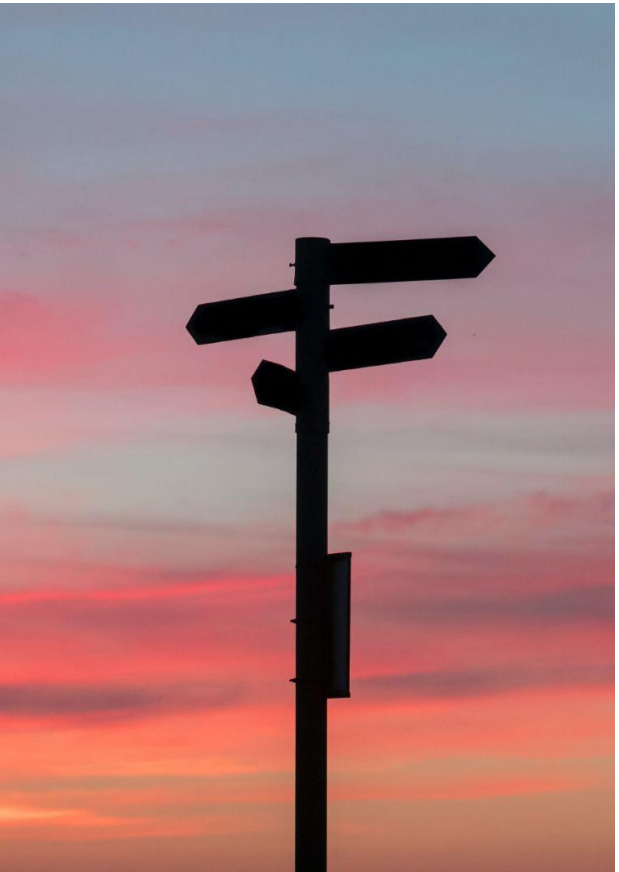
V. Guidance

VI. Distillation

VII. Spectral analysis

VIII. Other topics

Google DeepMind



V. Guidance

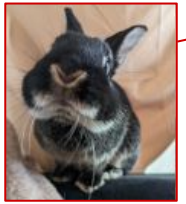
Guidance: a cheat code for diffusion models

- Guidance enables trading off sample quality for diversity
- It allows diffusion models to **punch well above their weight**

https://sander.ai/2022/05/26/guidance.html
https://sander.ai/2023/08/28/geometry.html

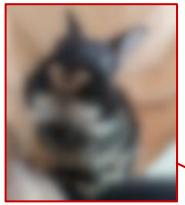
Diffusion: classifier guidance

x_t



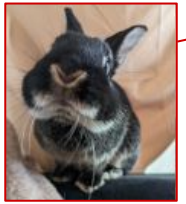
x_0

Diffusion: classifier guidance



$\hat{x}_0$

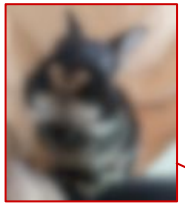
x_t



x_0

predict x_0

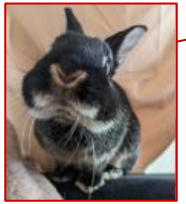
Diffusion: classifier guidance



$\hat{x}_0$

x_t

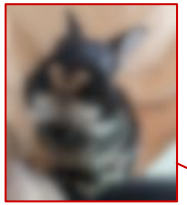
$\nabla_x \log p(c=\text{‘bunny’}|x_t)$



x_0

calculate $\nabla_x \log p(c|x_t)$

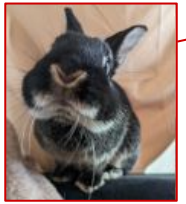
Diffusion: classifier guidance



$\hat{x}_0$

x_t

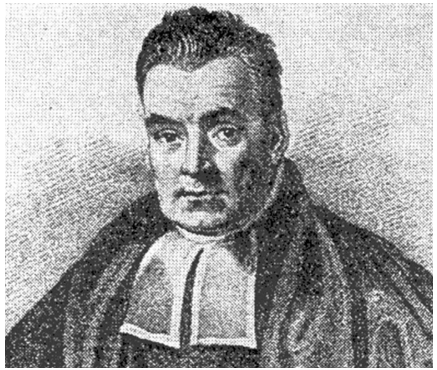
$\nabla_x \log p(c=\text{‘bunny’}|x_t)$



x_0

combine directions

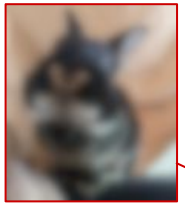
Classifier guidance: the Bayesian perspective



$$p(x|c) \propto p(c|x)p(x)$$
$$\log p(x|c) = \log p(c|x) + \log p(x) + C$$
$$\nabla_x \log p(x|c) = \nabla_x \log p(c|x) + \nabla_x \log p(x)$$

- Conditional score = unconditional score + classifier gradient w.r.t. input
- Turn an unconditional model conditional
- Without retraining!

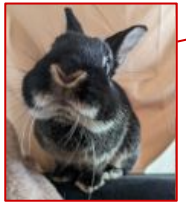
Diffusion: classifier guidance



$\hat{x}_0$

x_t

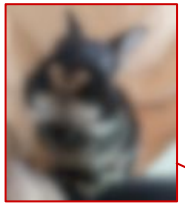
$\nabla_x \log p(c=\text{‘bunny’}|x_t)$



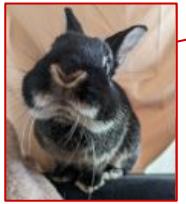
x_0

calculate $\nabla_x \log p(c|x_t)$

Diffusion: classifier guidance



$\hat{x}_0$



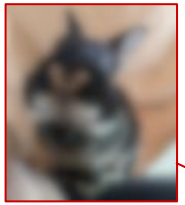
x_0

x_t

$y \cdot \nabla_x \log p(c=\text{‘bunny’}|x_t)$

scale by y

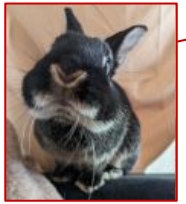
Diffusion: classifier guidance



$\hat{x}_0$

x_t

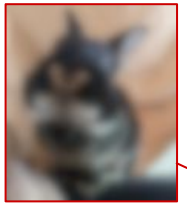
$y \cdot \nabla_x \log p(c=\text{‘bunny’}|x_t)$



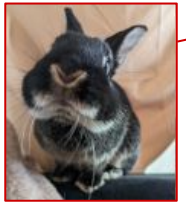
x_0

combine directions

Diffusion: classifier guidance



$\hat{x}_0$



x_0

x_t

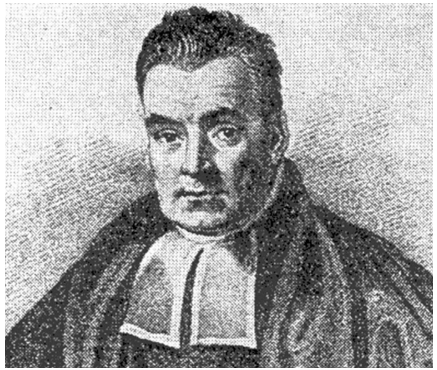
$y \cdot \nabla_x \log p(c=\text{‘bunny’}|x_t)$

x_{t-1}

ξ

add some noise

Classifier guidance: the Bayesian perspective

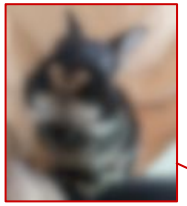


$$p_\gamma(x|c) \propto p(c|x)^\gamma p(x)$$
$$\log p_\gamma(x|c) = \gamma \cdot \log p(c|x) + \log p(x) + C$$
$$\nabla_x \log p_\gamma(x|c) = \gamma \cdot \nabla_x \log p(c|x) + \nabla_x \log p(x)$$

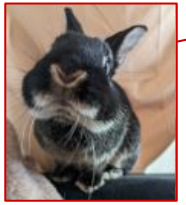
- Scale factor γ acts as an (inverse) **temperature**
- "Sharpening" of $p(c|x)$
- Different from sharpening $p(x)$ or $p(x|c)$!

Diffusion: classifier-free guidance

x_t



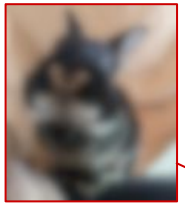
$\hat{x}_0$

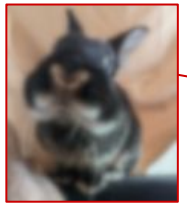


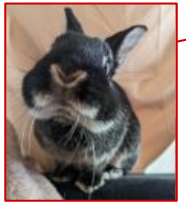
x_0

predict x_0

Diffusion: classifier-free guidance







$\hat{x}_0$

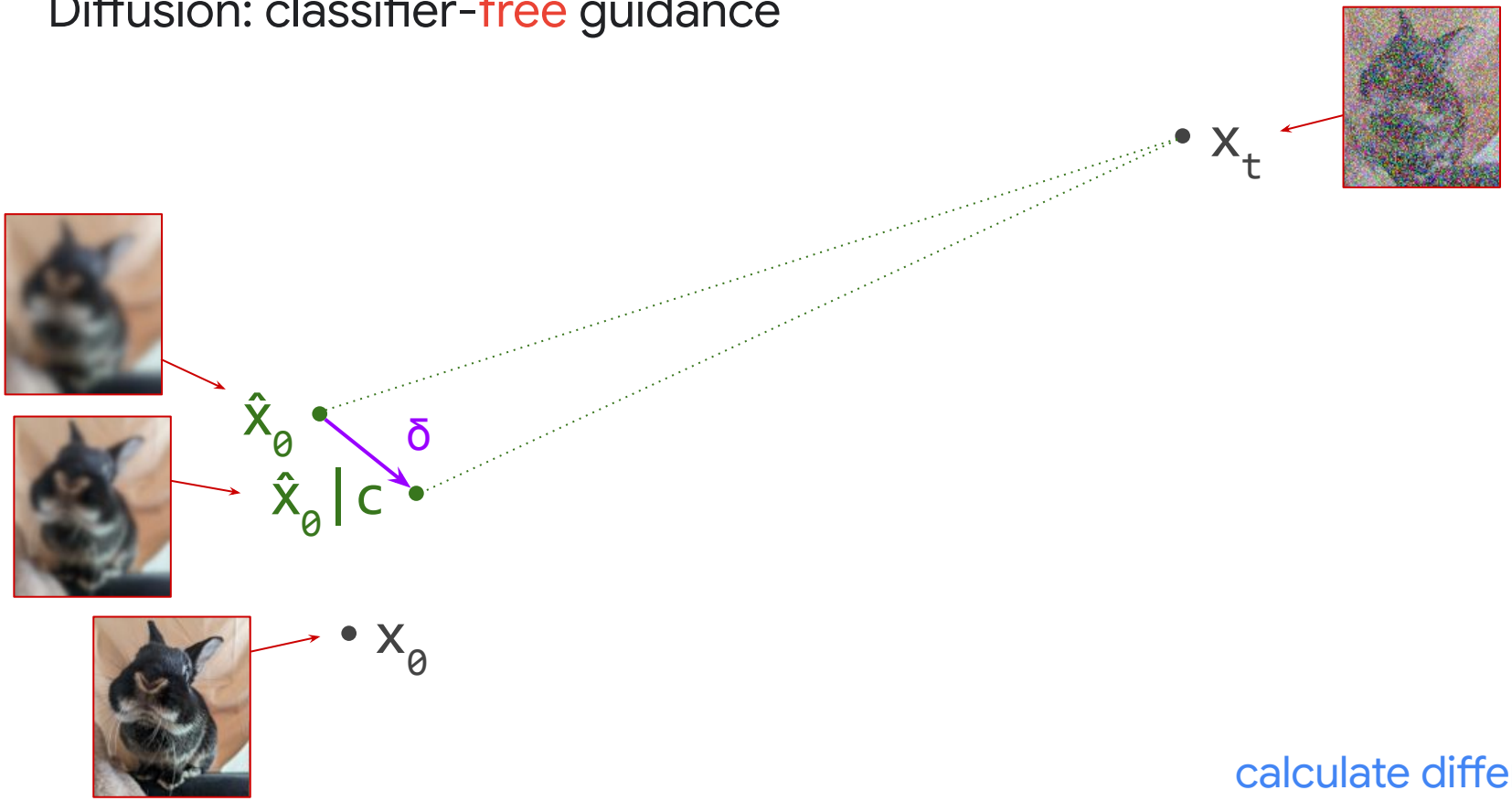
$\hat{x}_0 | c$

x_0

x_t

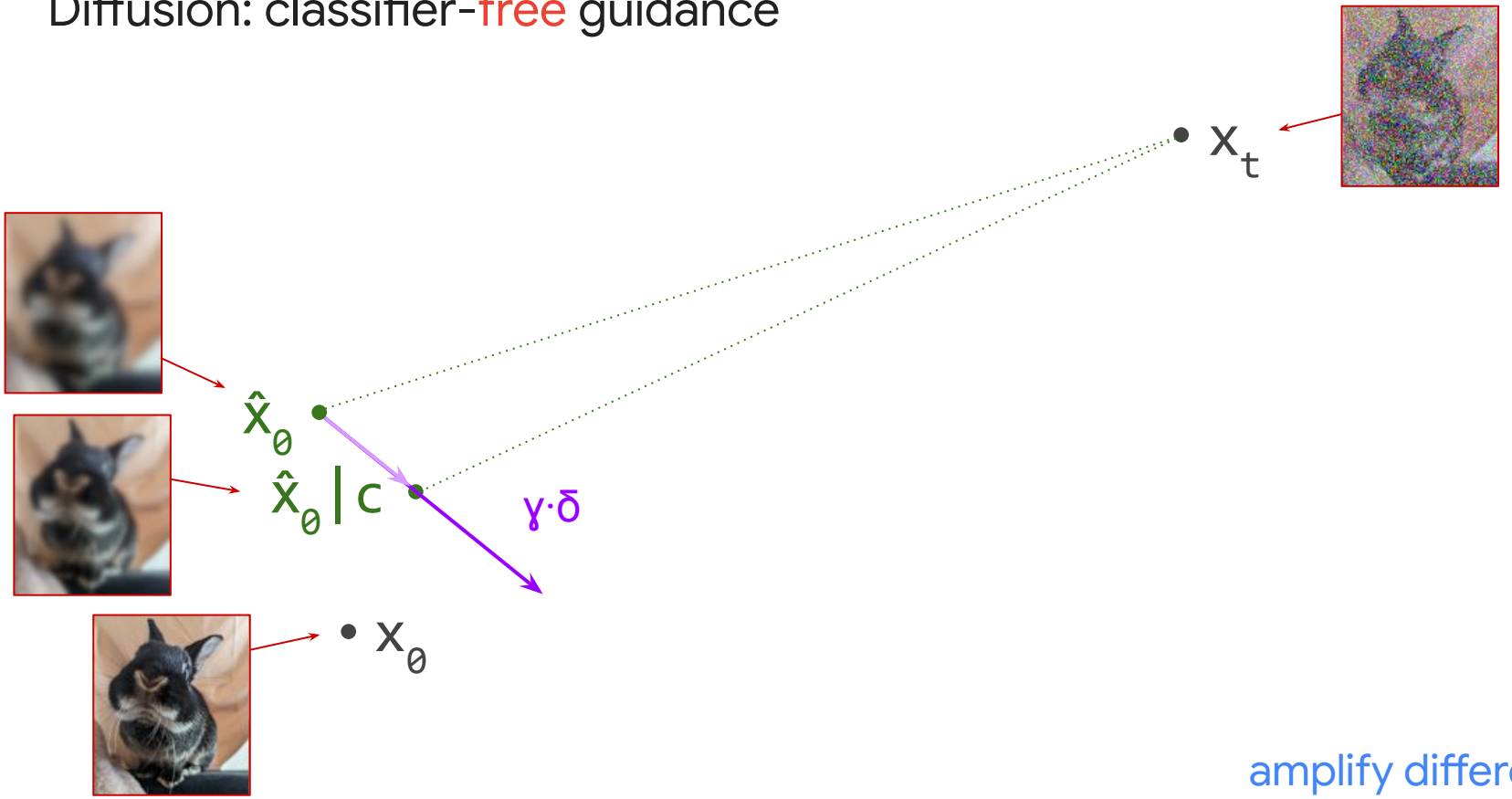
predict $x_0 | c$

Diffusion: classifier-free guidance



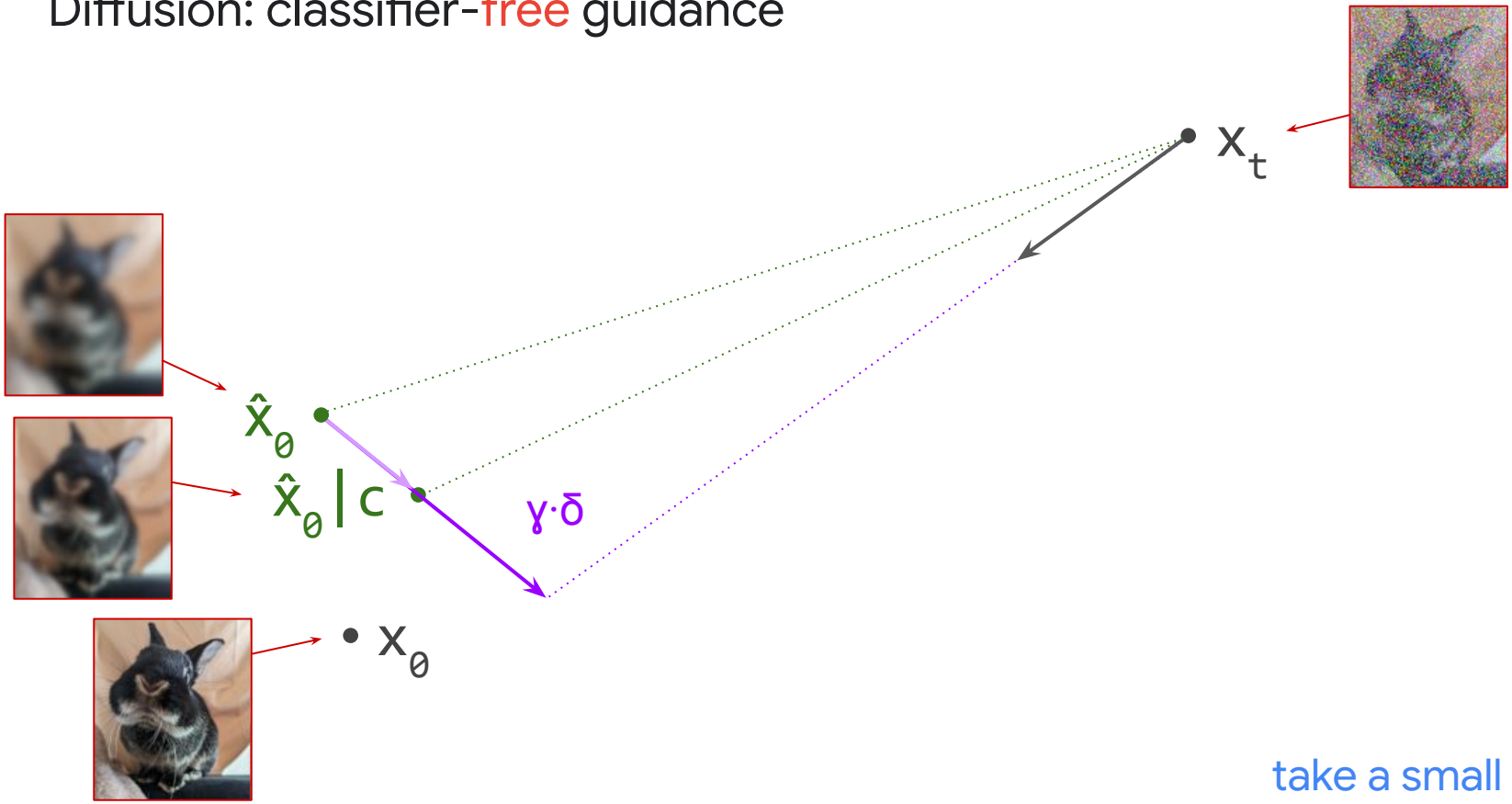
calculate difference

Diffusion: classifier-free guidance



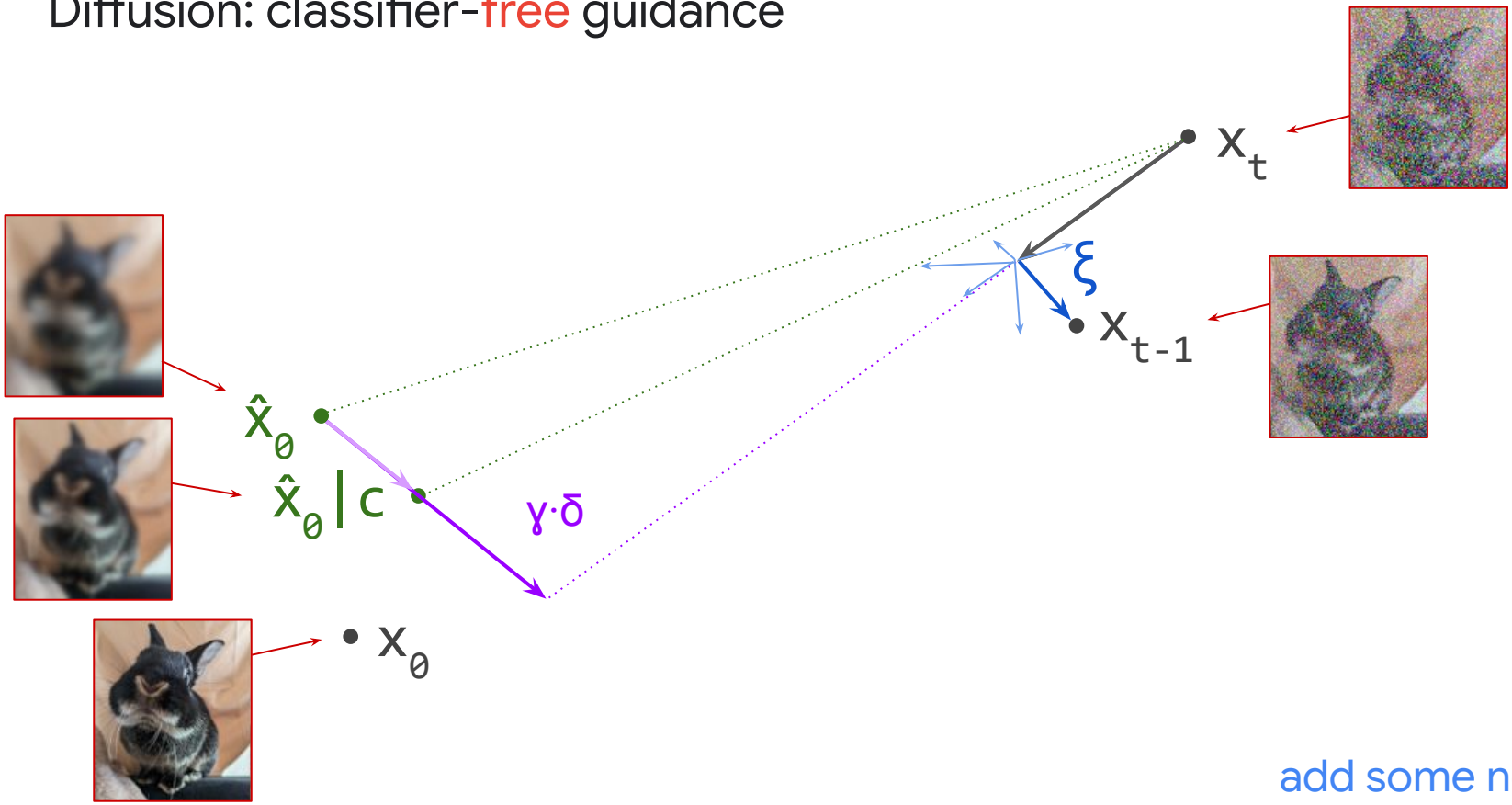
amplify difference

Diffusion: classifier-free guidance

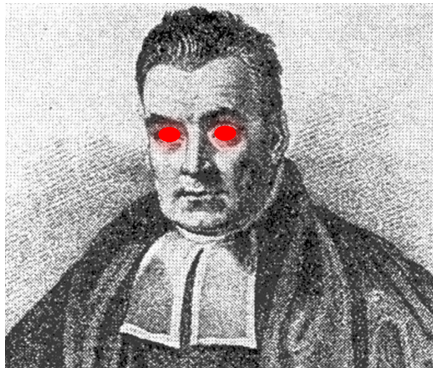


take a small step

Diffusion: classifier-free guidance



Classifier-free guidance: the Bayesian perspective



$$p_\gamma(x|c) \propto \left[\frac{p(x|c)}{p(x)}\right]^\gamma p(x)$$

$$\log p_\gamma(x|c) = \gamma \cdot [\log p(x|c) - \log p(x)] + \log p(x) + C$$

$$\nabla_x \log p_\gamma(x|c) = \gamma \cdot [\nabla_x \log p(x|c) - \nabla_x \log p(x)] + \nabla_x \log p(x)$$

- The difference vector δ is effectively a **Bayesian classifier**
- **We've simply applied Bayes' rule – twice!**
- We can get both $p(x)$ and $p(x|c)$ from the same model, by training with **conditioning dropout**
- Less prone to "adversarial examples" than classifier guidance

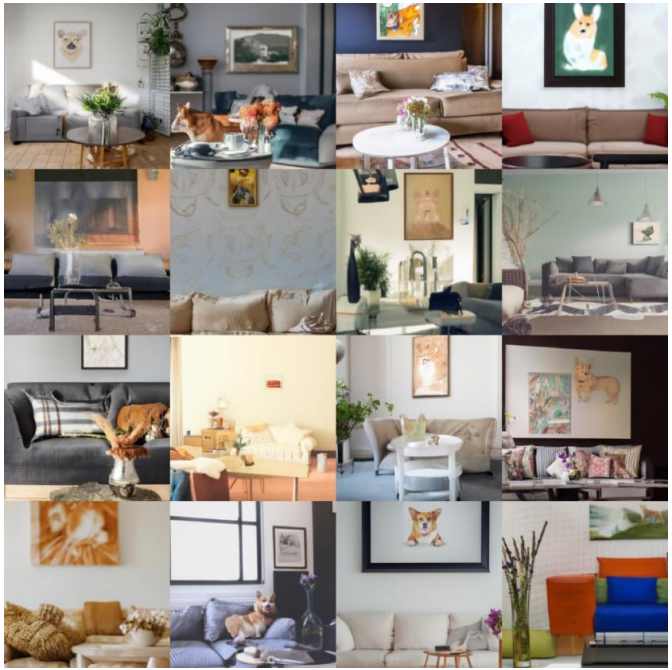
The power of classifier-free guidance

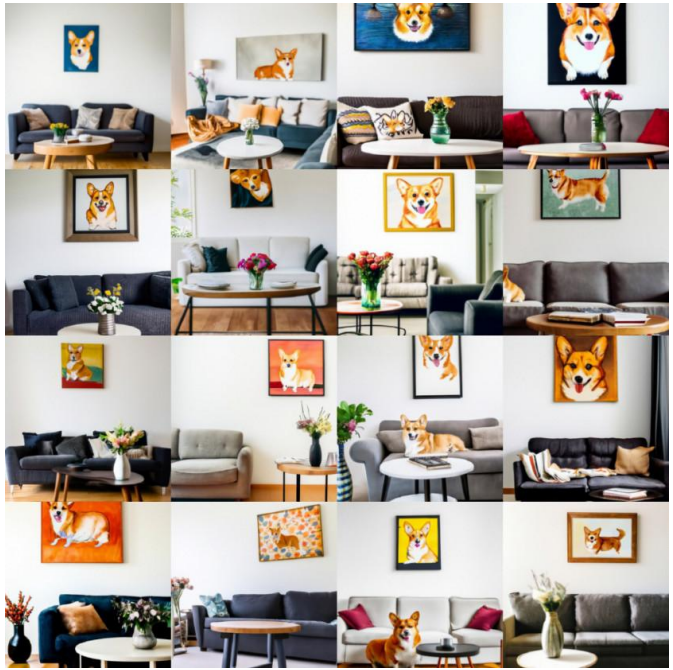
A stained glass window of a panda eating bamboo

Nichol et al. 'GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models' (2021)
https://sander.ai/2022/05/26/guidance.html

The power of classifier-free guidance

A cozy living room with a painting of a corgi on the wall [...]





Nichol et al. 'GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models' (2021)
https://sander.ai/2022/05/26/guidance.html

Always has been
$(1-\gamma)\nabla_x \log p(x)$
$+\gamma\nabla_x \log p(x \mid y)$
Wait, it's just
Bayes' rule?

Guiding a Diffusion Model with a Bad Version of Itself

Tero Karras
NVIDIA

Miika Aittala
NVIDIA

Tuomas Kynkäänniemi
Aalto University

Jaakko Lehtinen
NVIDIA, Aalto University

Timo Aila
NVIDIA

Samuli Laine
NVIDIA

Abstract

The primary axes of interest in image-generating diffusion models are image quality, the amount of variation in the results, and how well the results align with a given condition, e.g., a class label or a text prompt. The popular classifier-free guidance approach uses an unconditional model to guide a conditional model, leading to simultaneously better prompt alignment and higher-quality images at the cost of reduced variation. These effects seem inherently entangled, and thus hard to control. We make the surprising observation that it is possible to obtain disentangled control over image quality without compromising the amount of variation by guiding generation using a smaller, less-trained version of the model itself rather than an unconditional model. This leads to significant improvements in ImageNet generation, setting record FIDs of 1.01 for 64×64 and 1.25 for 512×512, using publicly available networks. Furthermore, the method is also applicable to unconditional diffusion models, drastically improving their quality.

Google DeepMind

VI. Distillation

Distillation: cutting corners

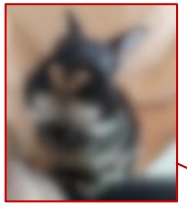
- Distill many-step sampling into fewer steps
- This requires **cutting corners**
  - Reduced entropy/diversity
  - Reduced visual fidelity
  - ...
- Many different algorithms exist
  - Most rely on deterministic teachers
  - Student initialised from teacher
  - Perceptual / adversarial losses
  - Bootstrapping

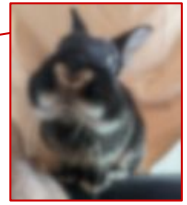
https://sander.ai/2024/02/28/paradox.html

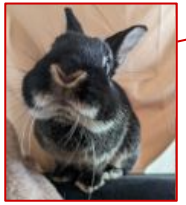
Distillation: cutting corners

- Distill many-step sampling into fewer steps
- This requires **cutting corners**
  - Reduced entropy/diversity
  - Reduced visual fidelity
  - ...

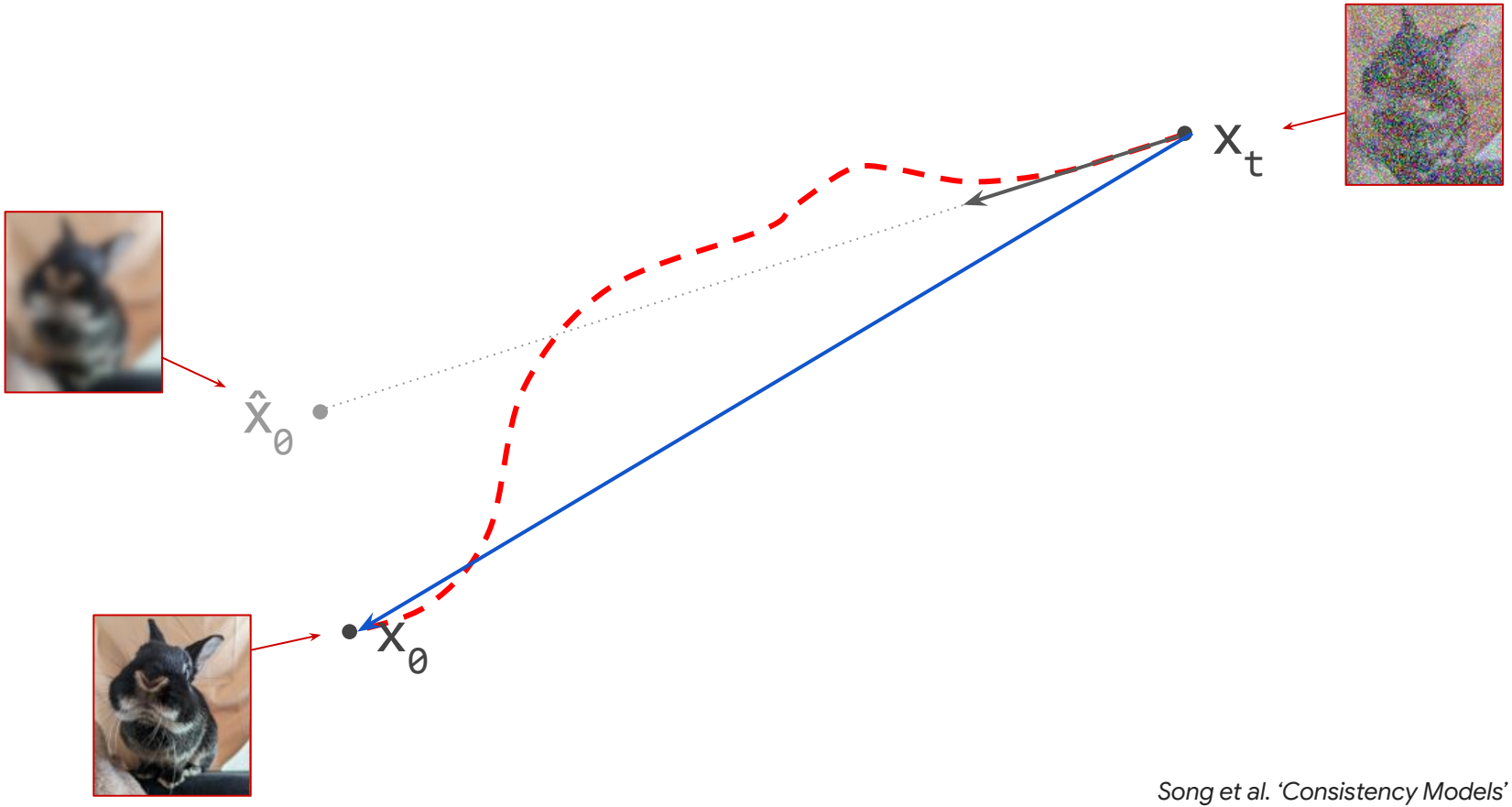
https://sander.ai/2024/02/28/paradox.html



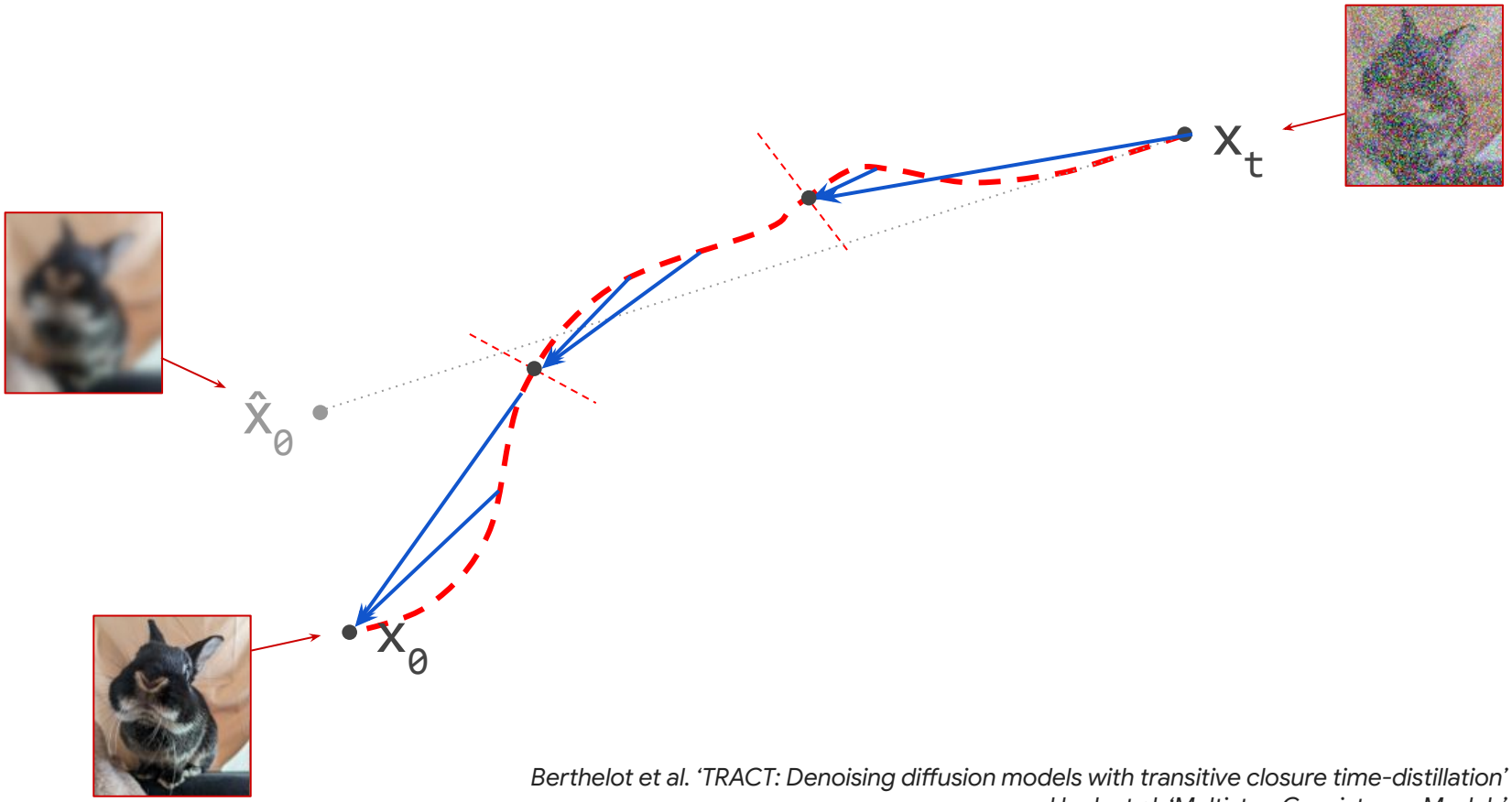




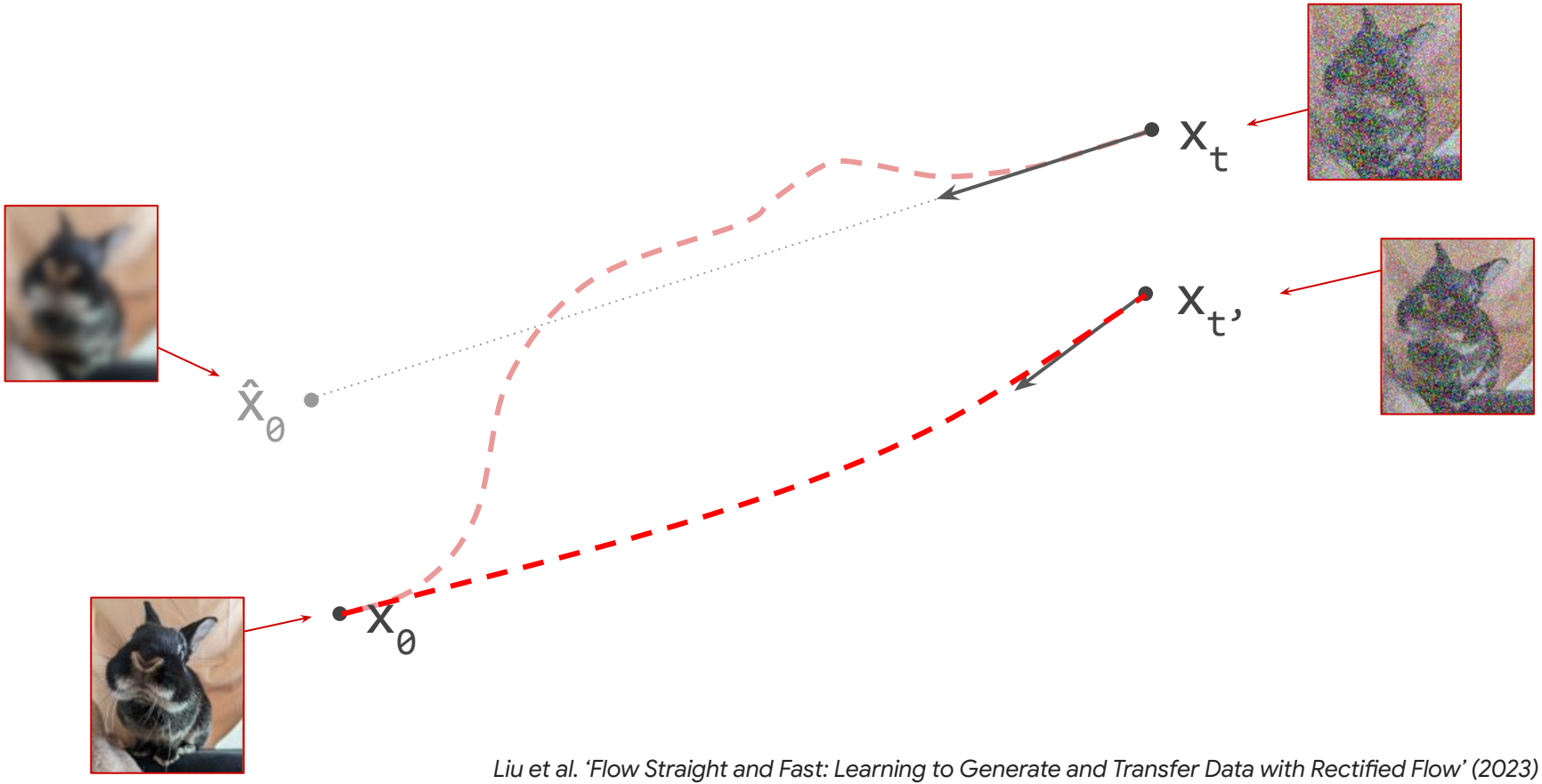
x_{t-1}
ε
x_t
$\hat{x}_0$
$\hat{x}_0$
x_0



Song et al. 'Consistency Models' (2023)



Berthelot et al. 'TRACT: Denoising diffusion models with transitive closure time-distillation' (2023)
Heek et al. 'Multistep Consistency Models' (2024)

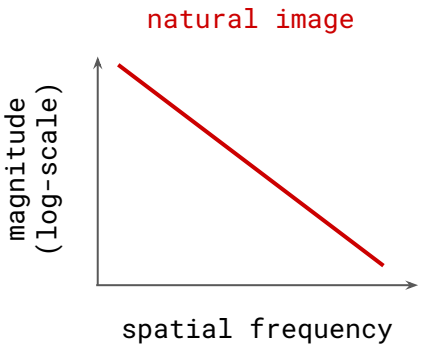


Liu et al. 'Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow' (2023)

Google DeepMind

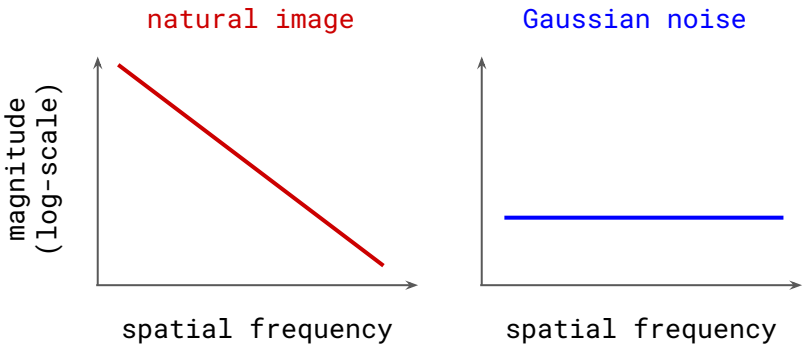
VII. Spectral analysis

Why do diffusion models work so well for images?



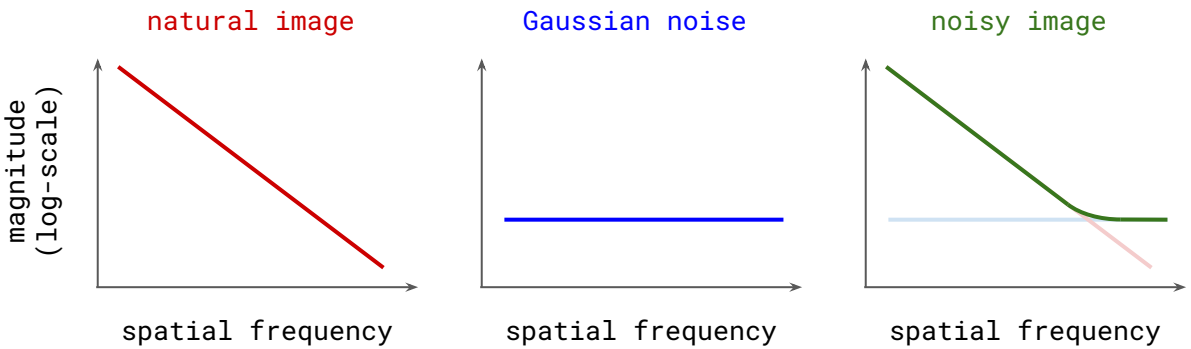
https://sander.ai/2023/07/20/perspectives.html

Why do diffusion models work so well for images?



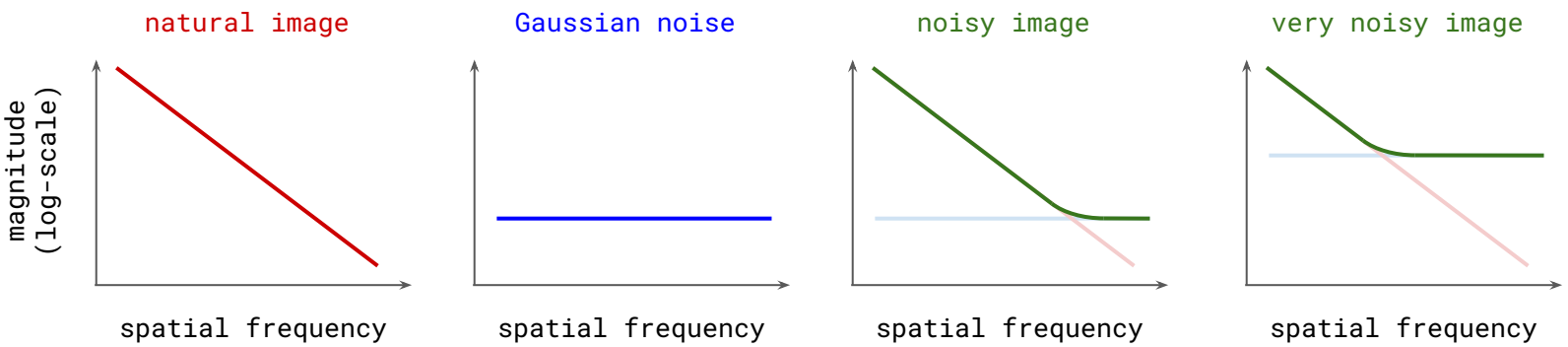
https://sander.ai/2023/07/20/perspectives.html

Why do diffusion models work so well for images?



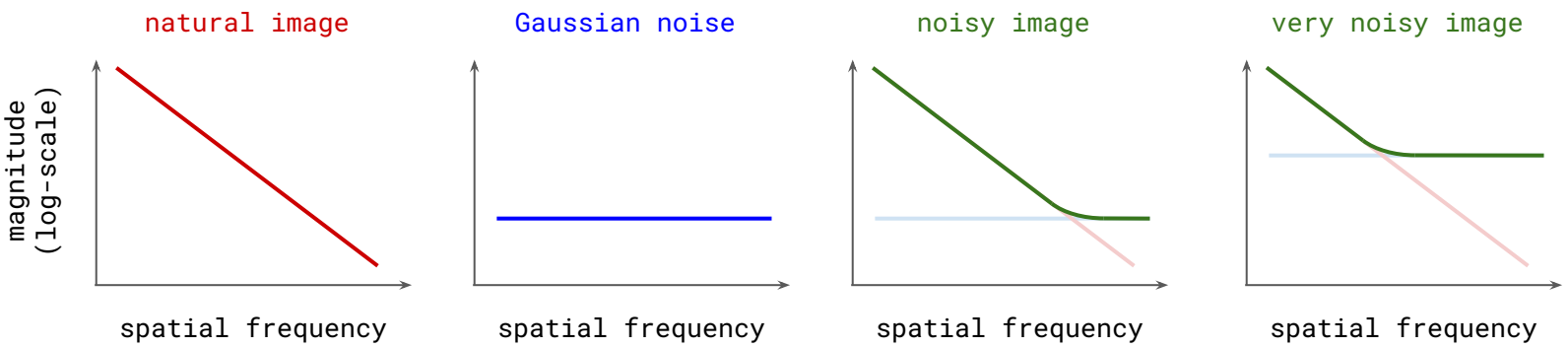
https://sander.ai/2023/07/20/perspectives.html

Why do diffusion models work so well for images?



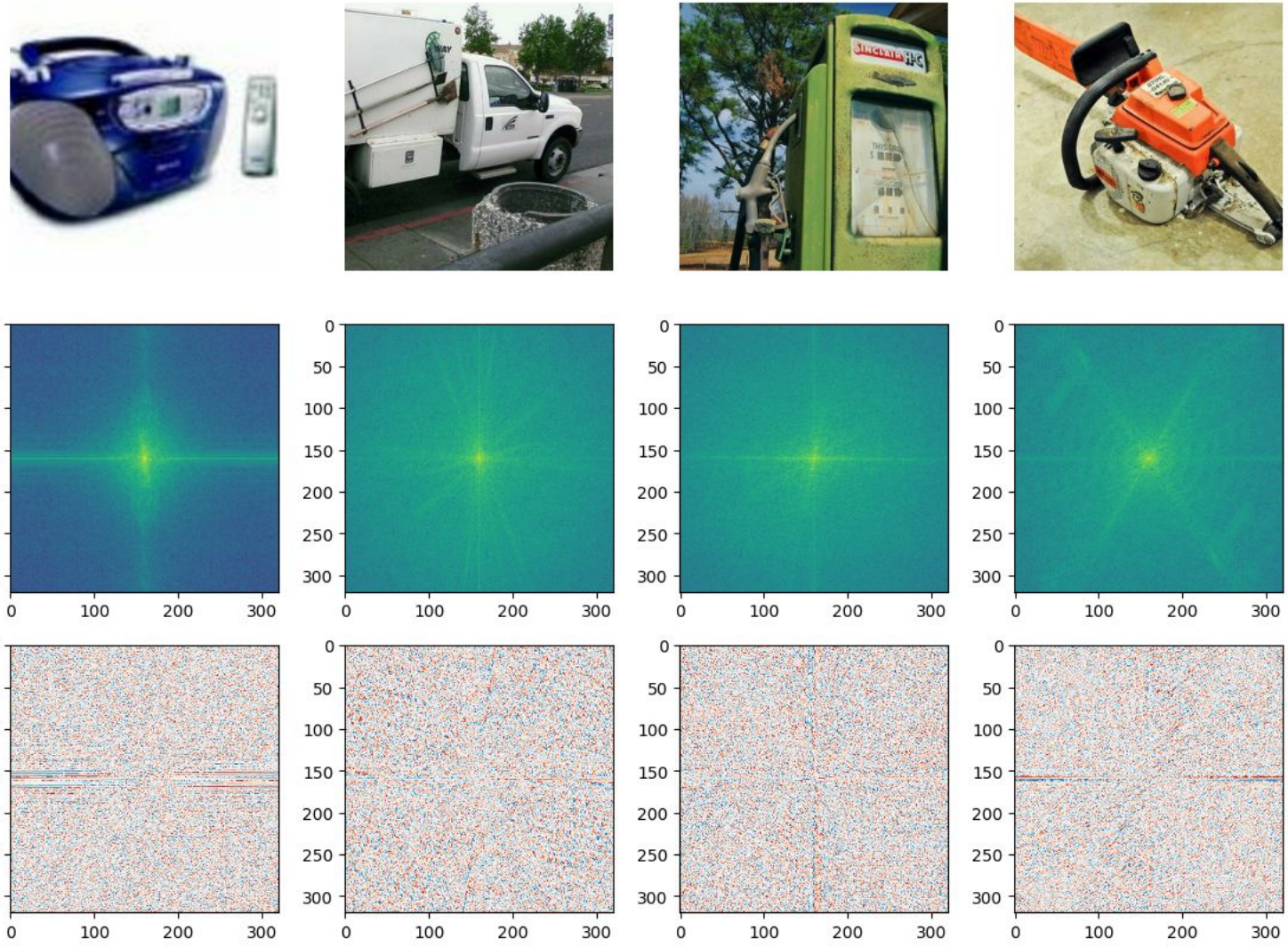
https://sander.ai/2023/07/20/perspectives.html

Why do diffusion models work so well for images?

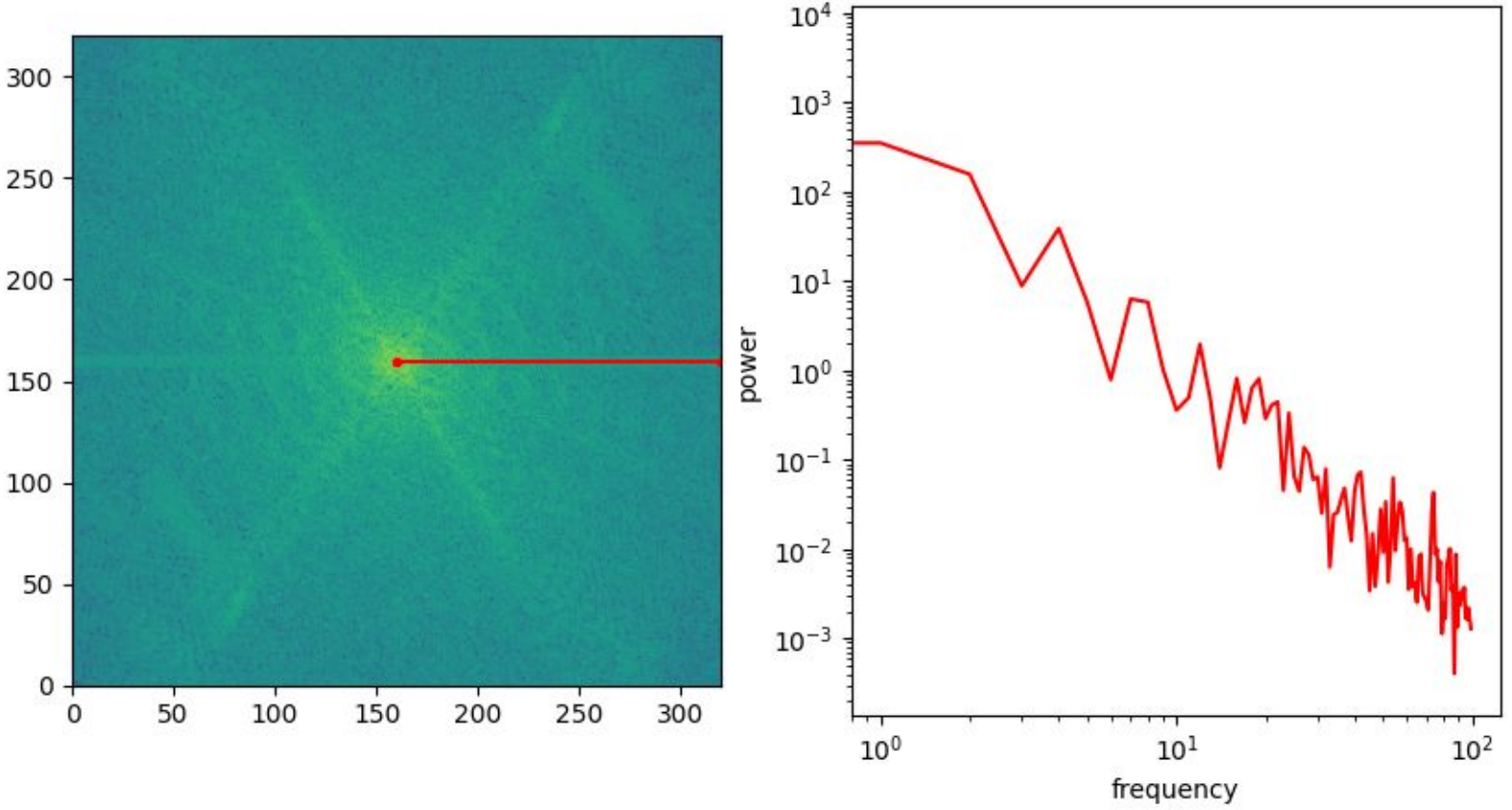


Diffusion is (approximate) **spectral autoregression** 🤯

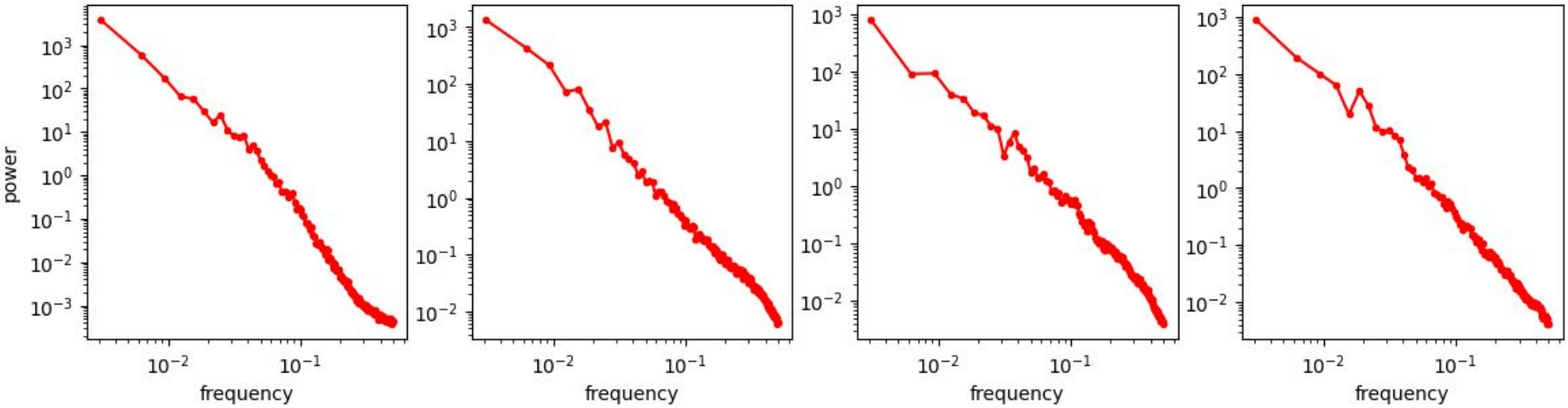
https://sander.ai/2023/07/20/perspectives.html


0
50
100
150
200
250
300
0
100
200
300

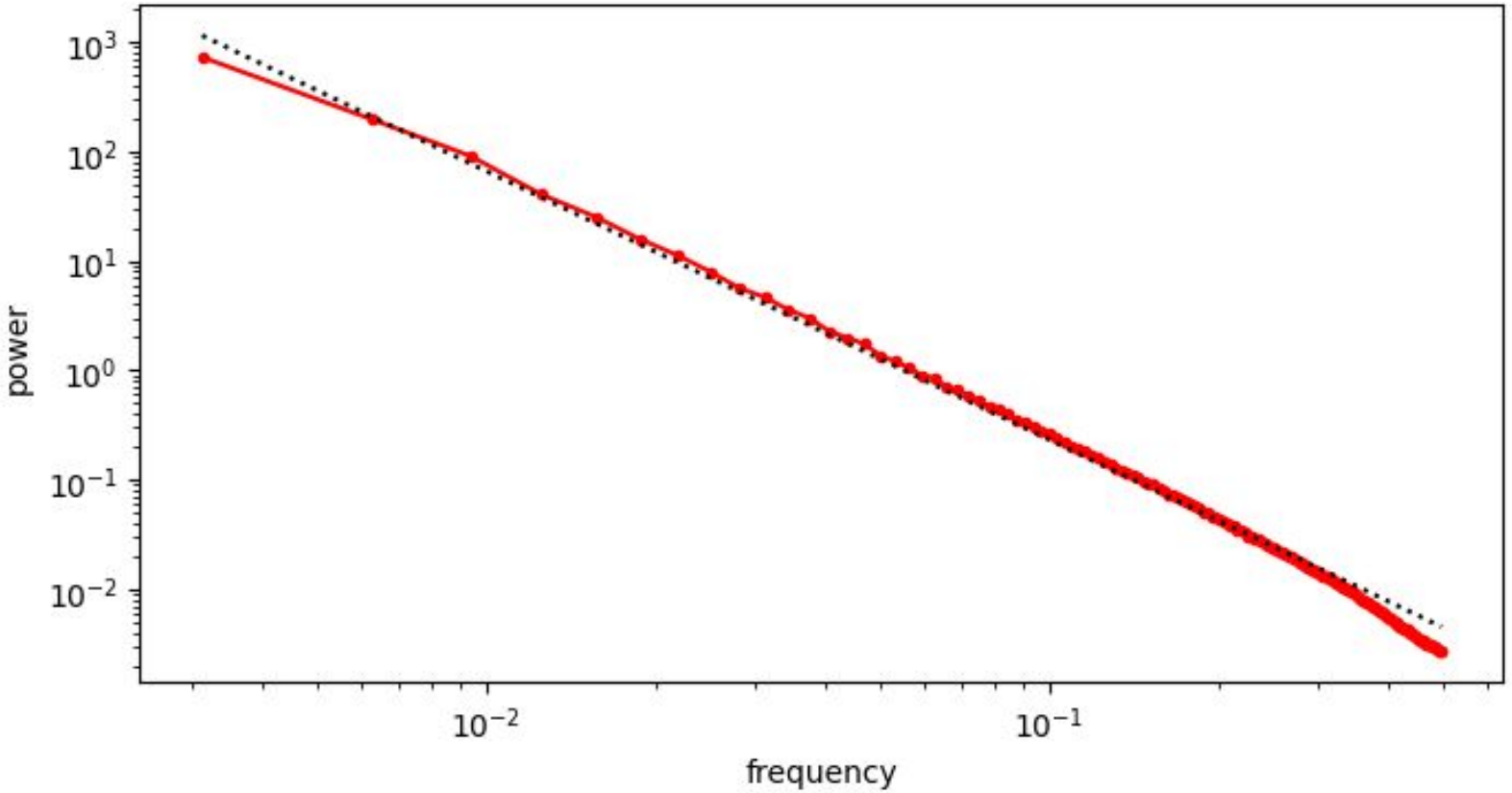
Radially averaged power spectral density (RAPSD)

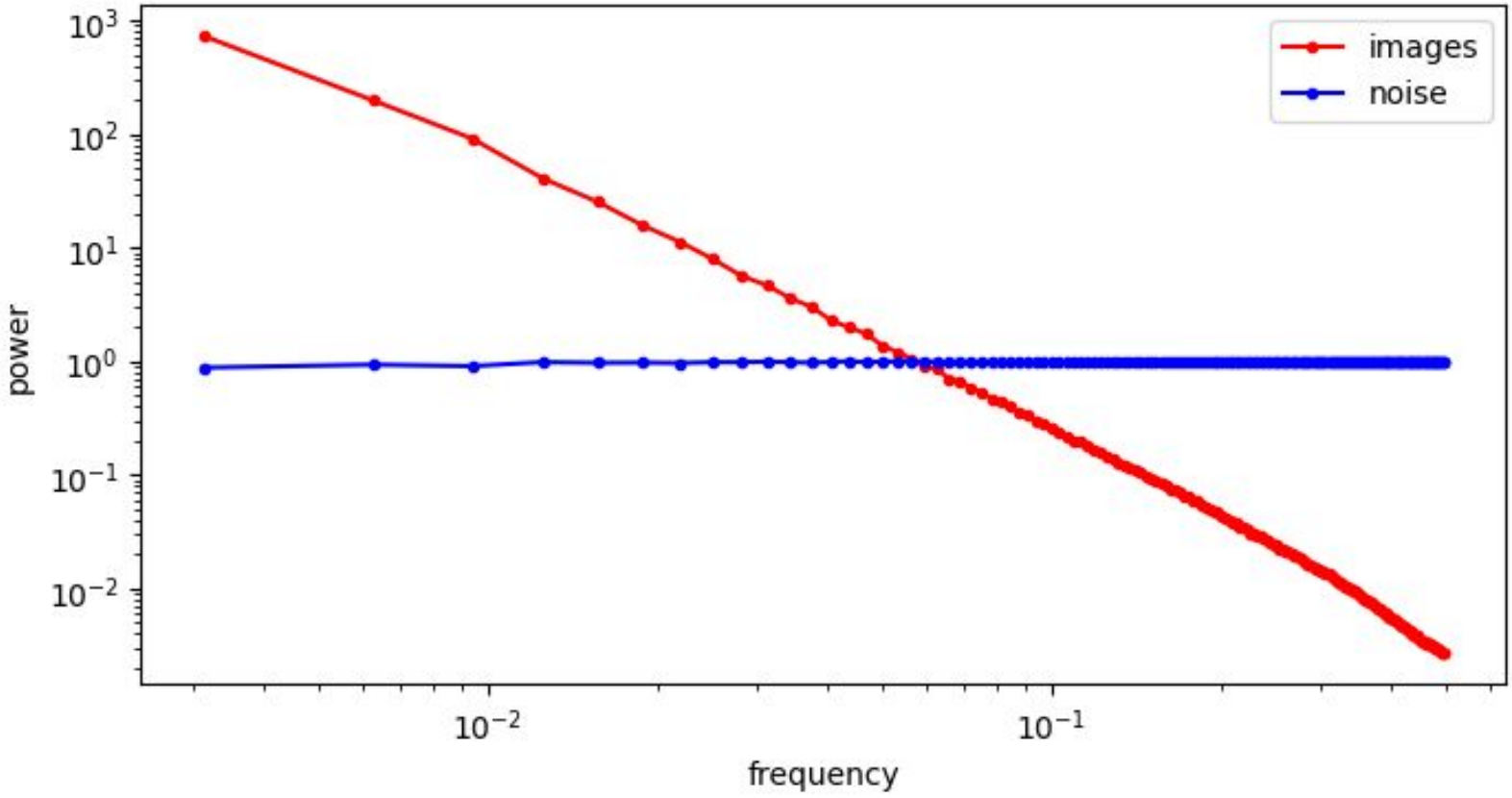


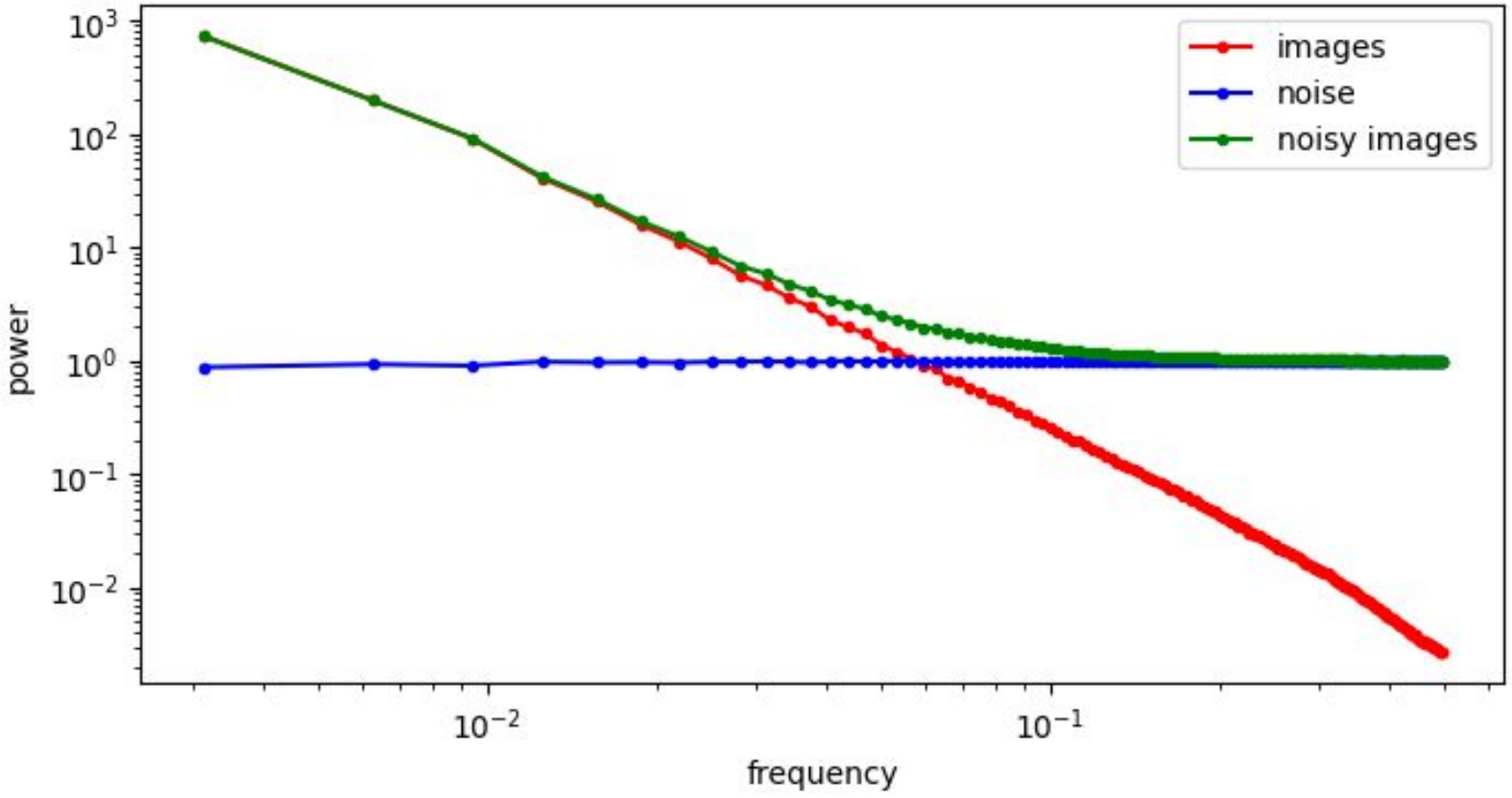
SINCLAIR H-C

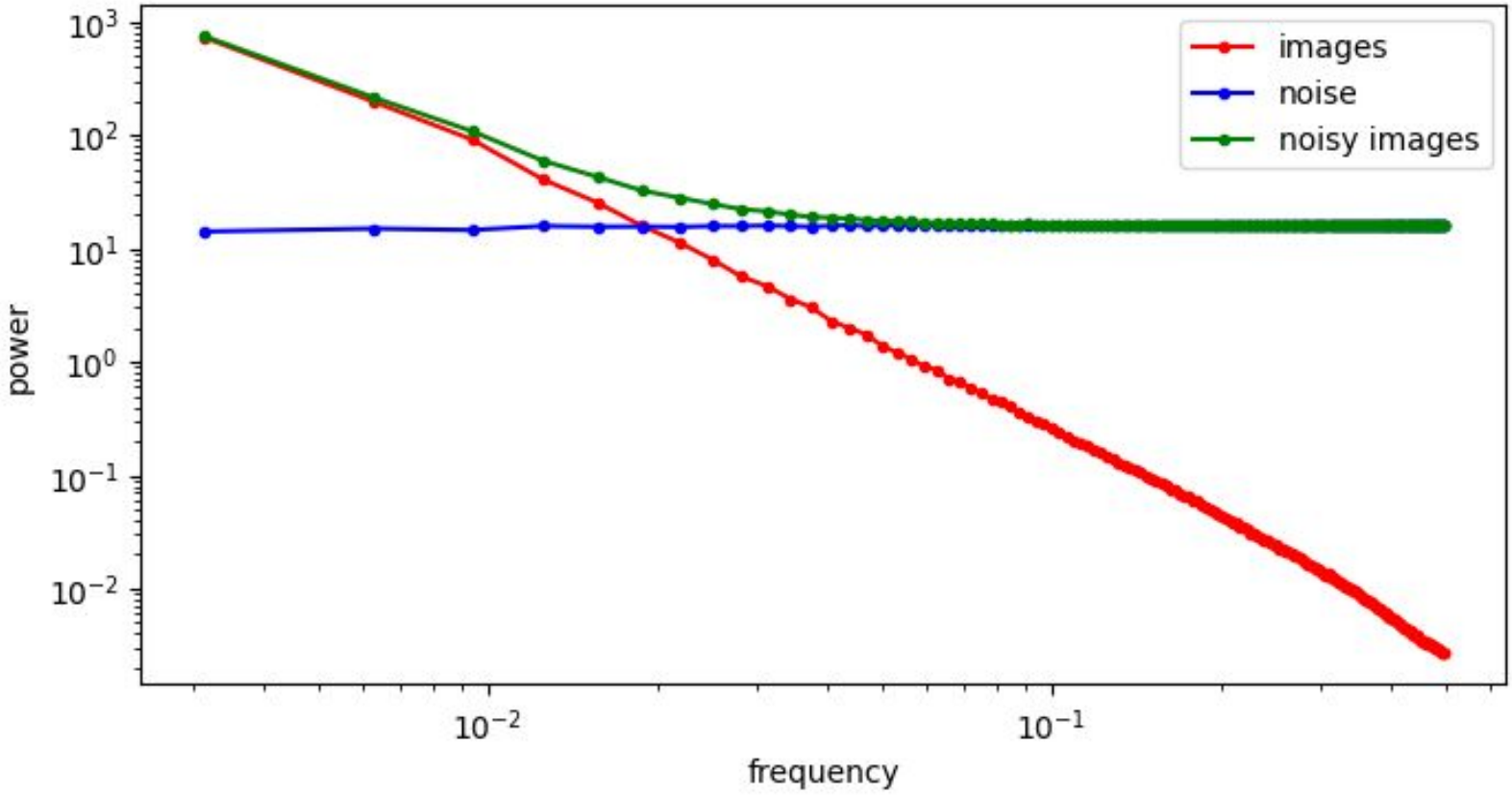

power
frequency
frequency
frequency
frequency

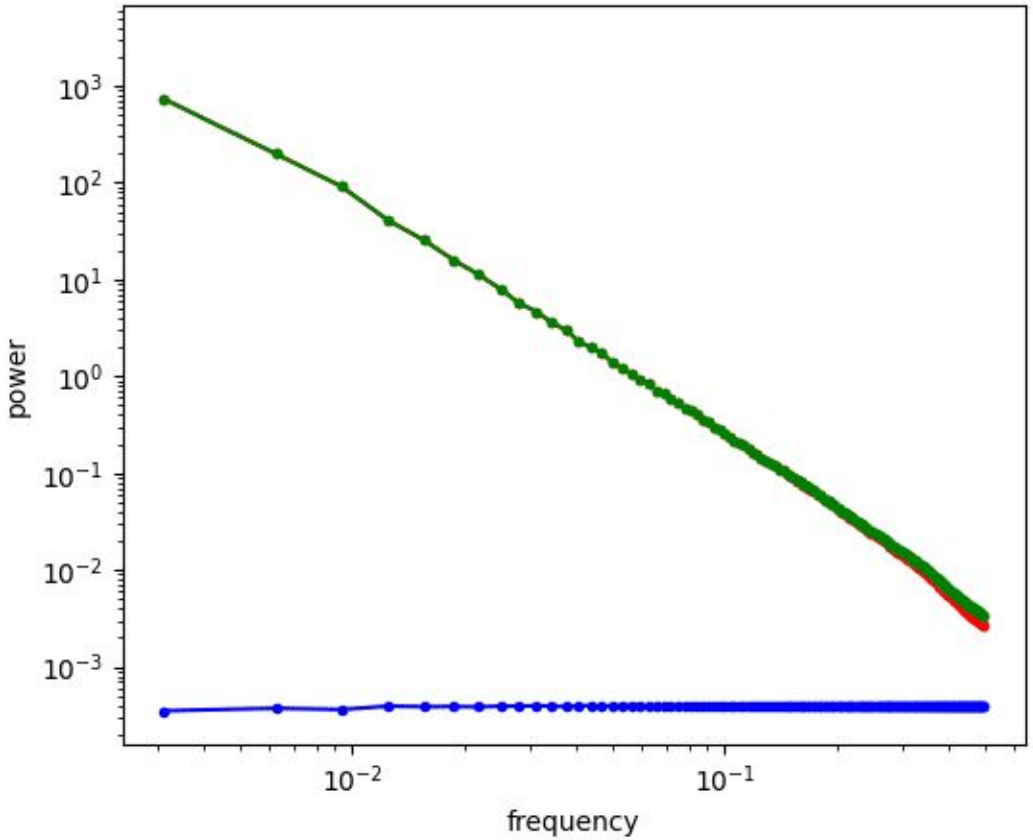
Average RAPSD (log-domain)



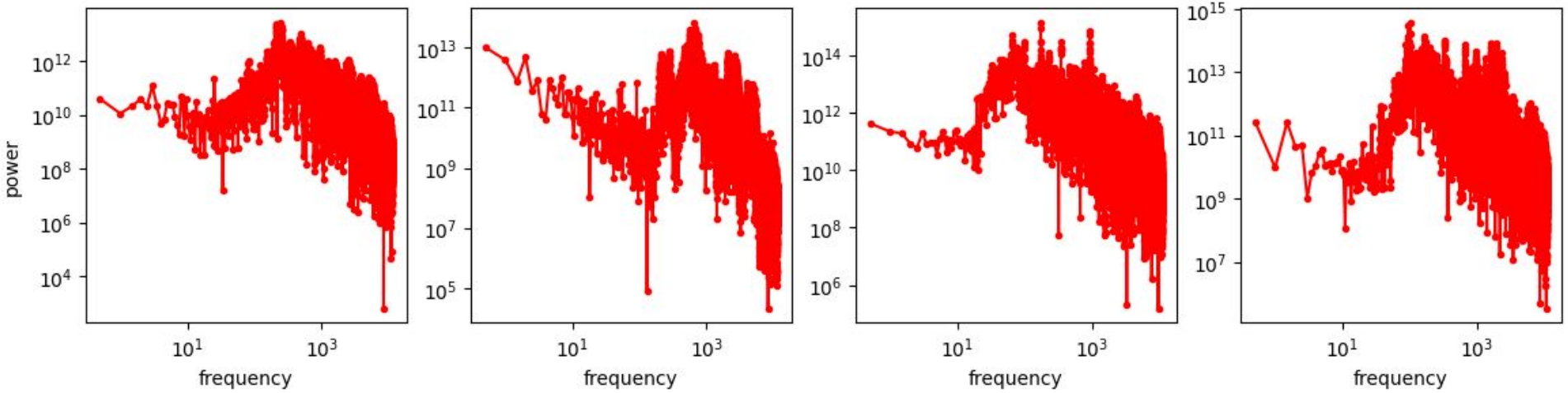

images
noise
power
frequency
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-2}
10^{-1}


images
noise
noisy images
power
frequency
10^3
10^2
10^1
10^0
10^{-1}
10^{-2}
10^{-2}
10^{-1}

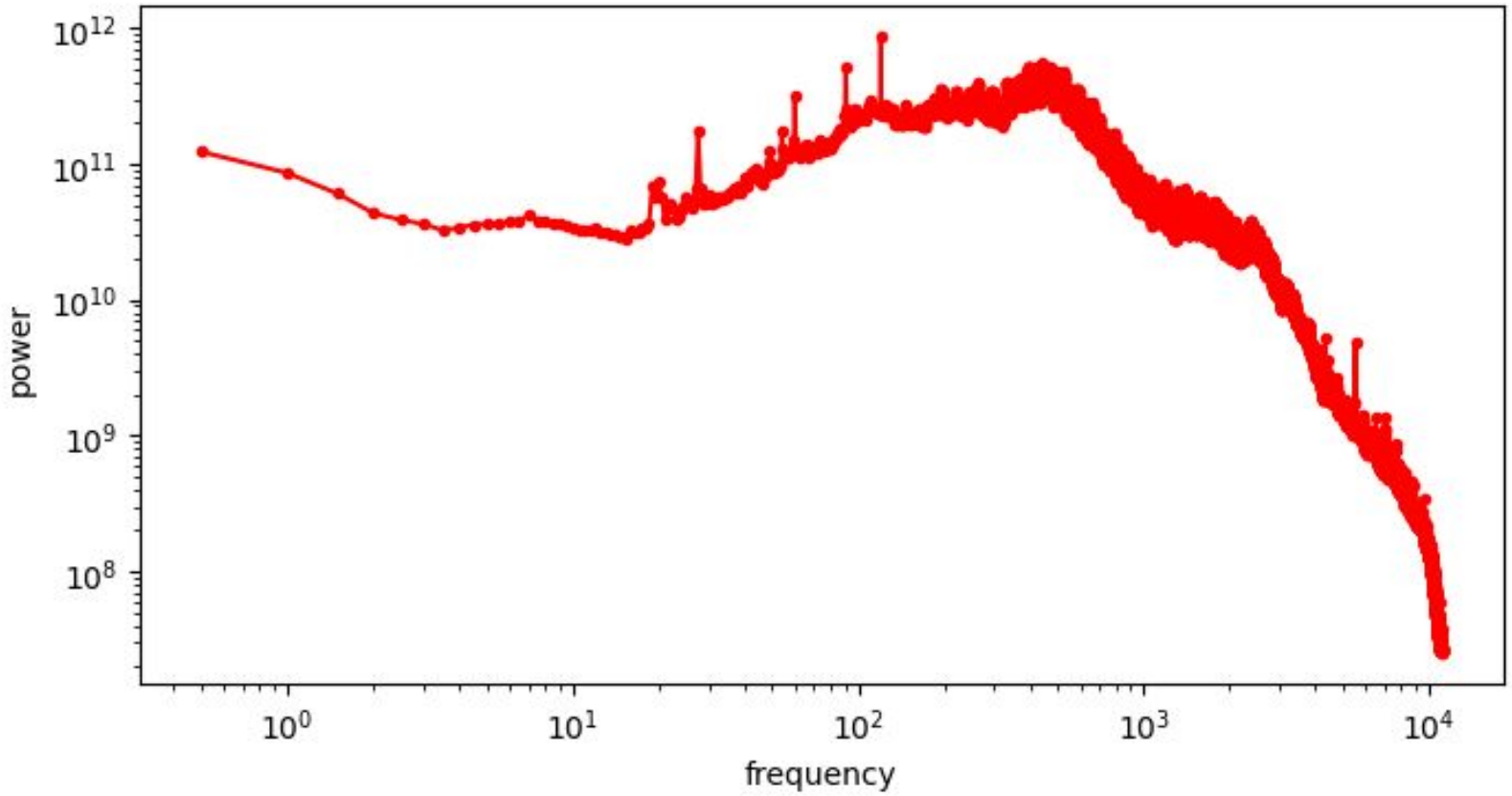

images
noise
noisy images
power
frequency
10^3
10^2
10^1
10^0
10^{-1}
10^{-2}
10^{-2}
10^{-1}

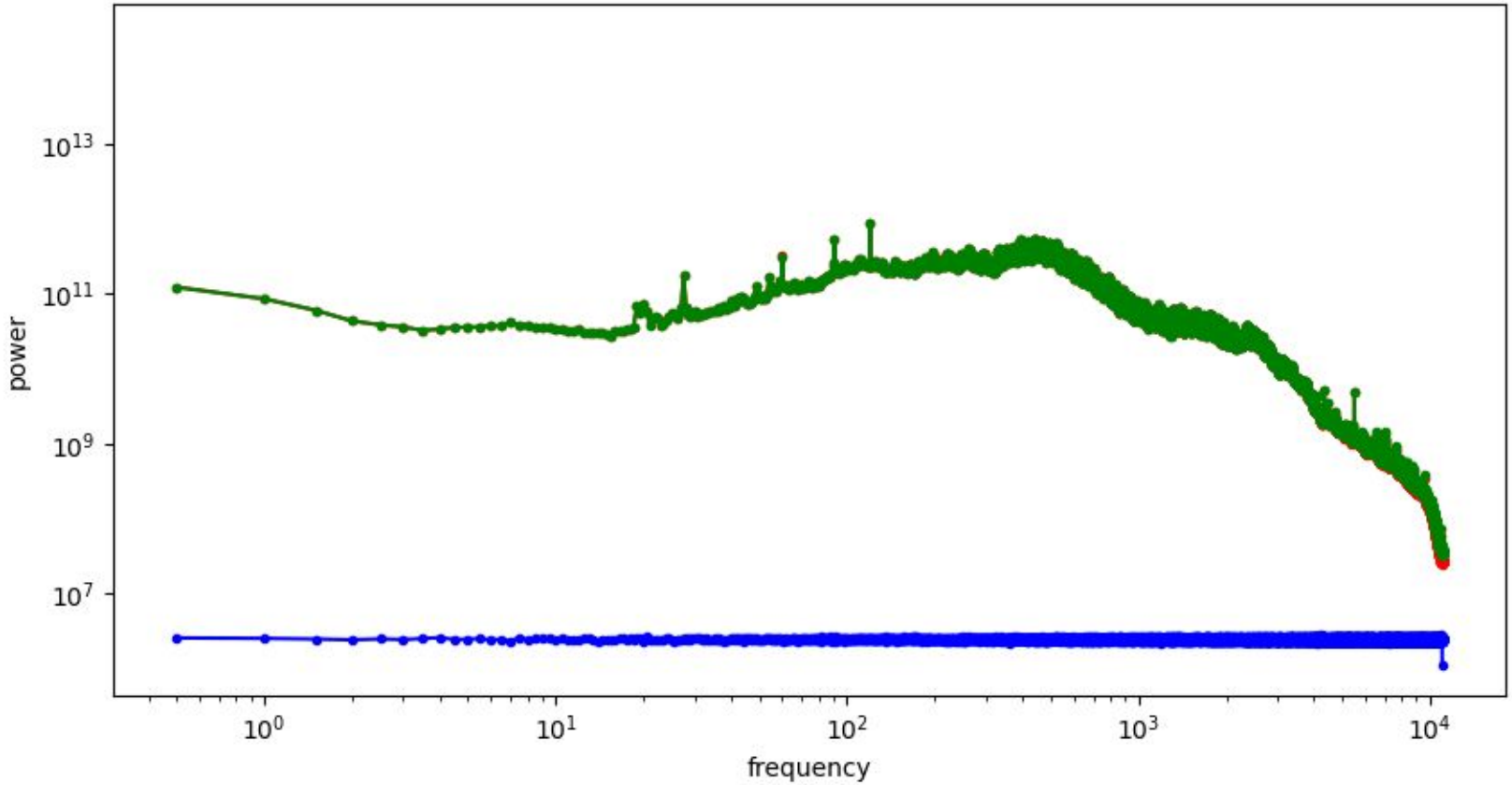

power
frequency
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-2}
10^{-1}

What about sound?

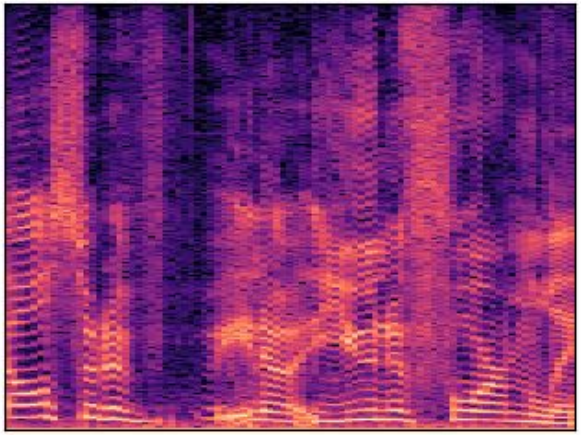
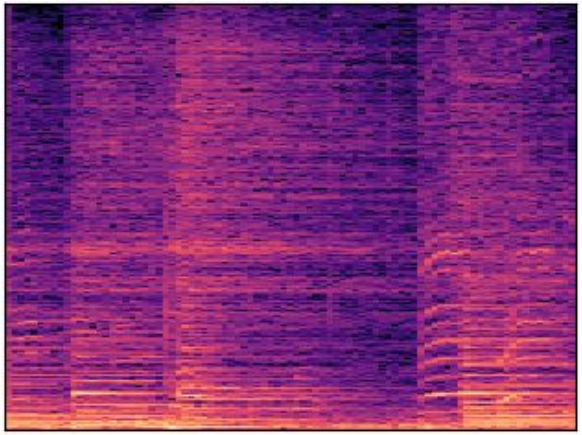
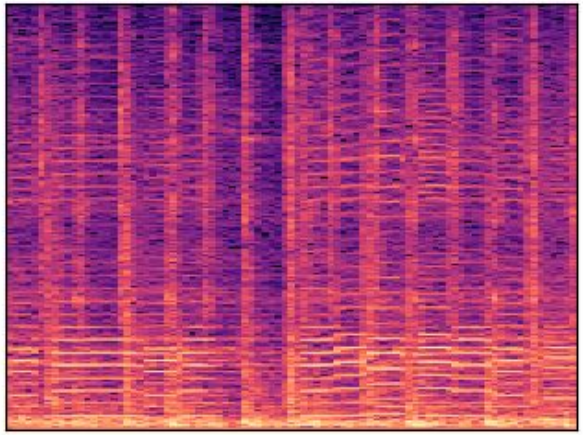


Average audio spectrum (log-domain) is not monotonic

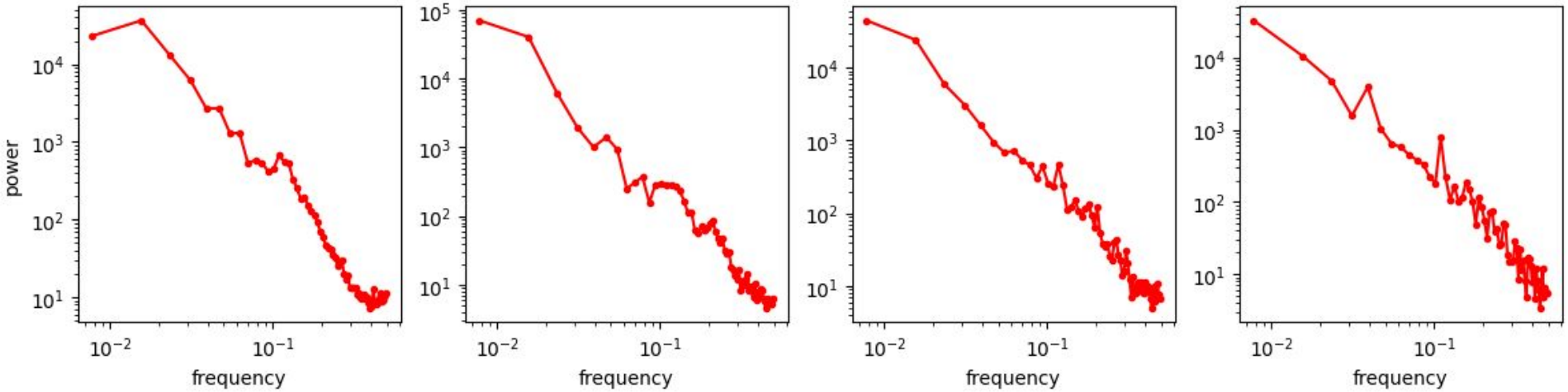



power
10^{13}
10^{11}
10^{9}
10^{7}
10^{0}
10^{1}
10^{2}
10^{3}
10^{4}
frequency

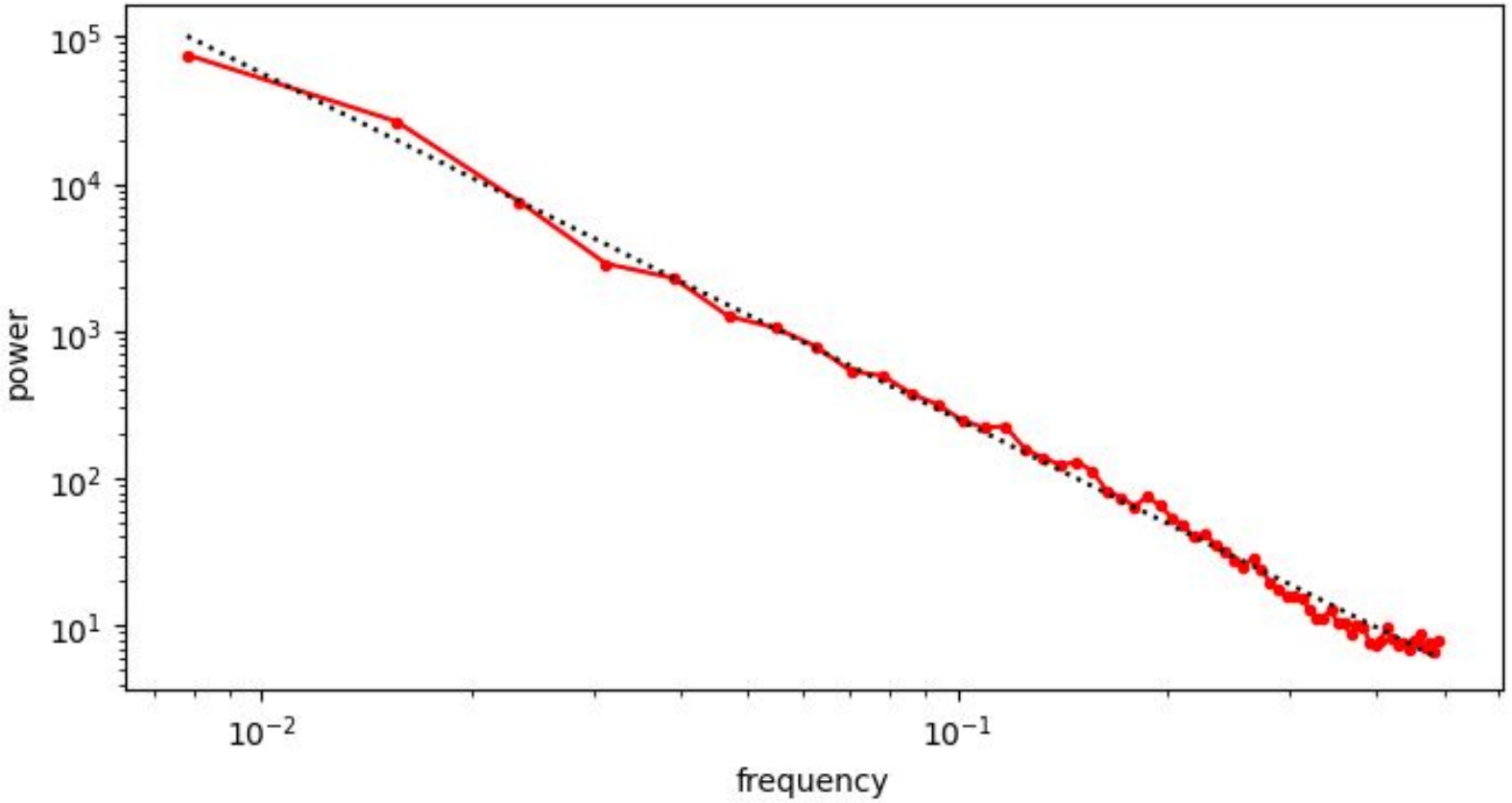
Spectrograms

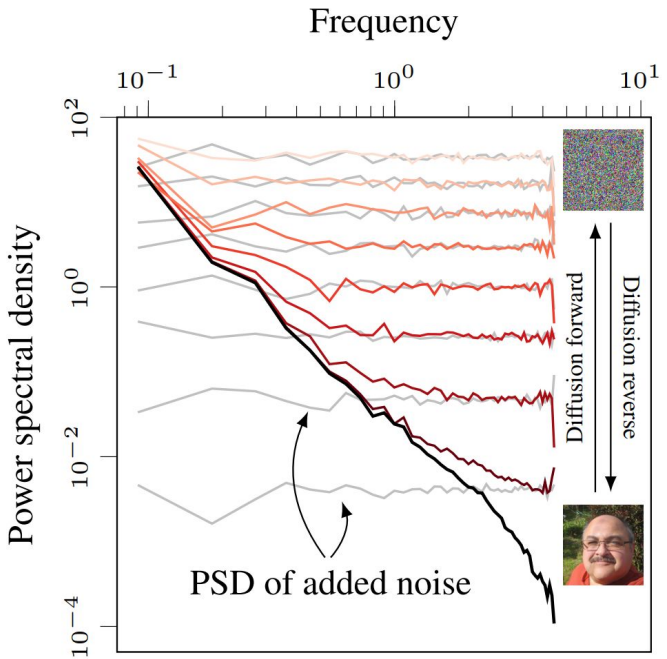
Spectrogram spectra (= cepstra)



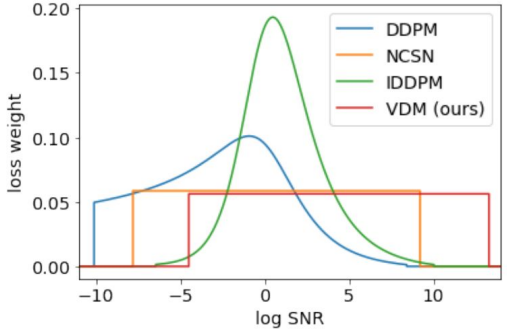
Average cepstrum



Why do diffusion models work so well for images?



- Noise variance ~ spatial frequency ~ feature scale
- Implicit downweighting of high frequencies in the loss
- **Better perceptual loss** than likelihood



Song et al. 'Maximum Likelihood Training of Score-Based Diffusion Models' (2021)
Kingma et al. 'Variational Diffusion Models' (2021)
Rissanen et al. 'Generative Modelling With Inverse Heat Dissipation' (2022)

Diffusion

Spectral autoregression

Google DeepMind

VIII. Other topics

Weighting and spacing different noise levels

- **Weighting** determines relative importance of different noise levels (= spatial frequencies = feature scales) at **training time**

- Choice of weighting depends on:
  - Perceptual relevance
  - Difficulty of the learning task

- Weighting is affected by:
  - Noise schedule $\sigma(t)$
  - Explicit weighting factor
  - Time step distribution
  - Model parameterisation ($\mathbf{x}$ or ε)

https://sander.ai/2024/06/14/noise-schedules.html

Weighting and spacing different noise levels

- **Spacing** determines relative importance of different noise levels (= spatial frequencies = feature scales) at **sampling time**

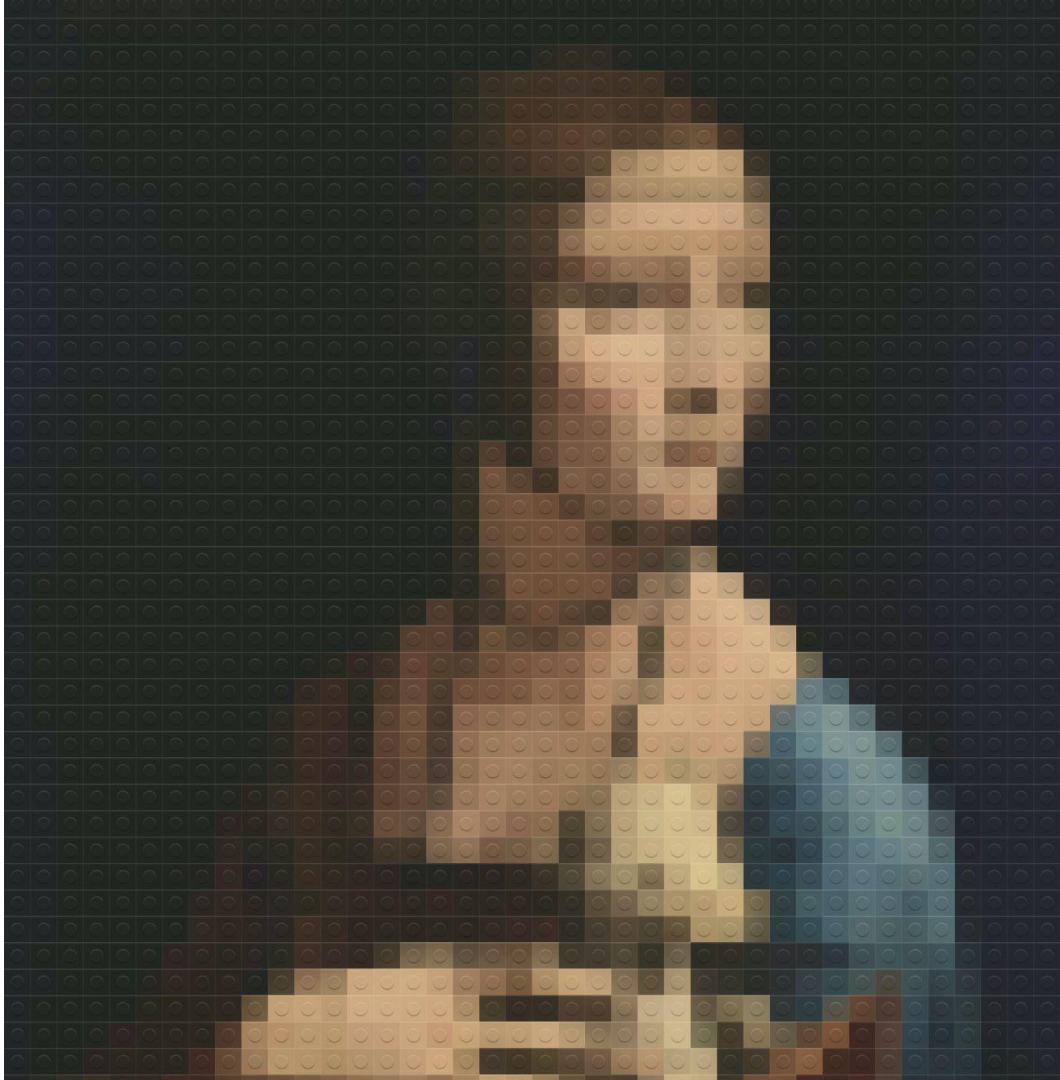
- Choice of spacing depends on:
  - Perceptual relevance
  - Prediction accuracy
  - Potential for error accumulation

- Spacing is affected by:
  - Noise schedule $\sigma(t)$
  - Explicit non-uniform time step spacing

https://sander.ai/2024/06/14/noise-schedules.html

Moving away from likelihood in input space

- Likelihood is a poor perceptual metric
- (Works well for language though!)
- AR models:
  measure likelihood in latent space
- Diffusion models:
  reweight the noise levels and/or
  measure loss in latent space



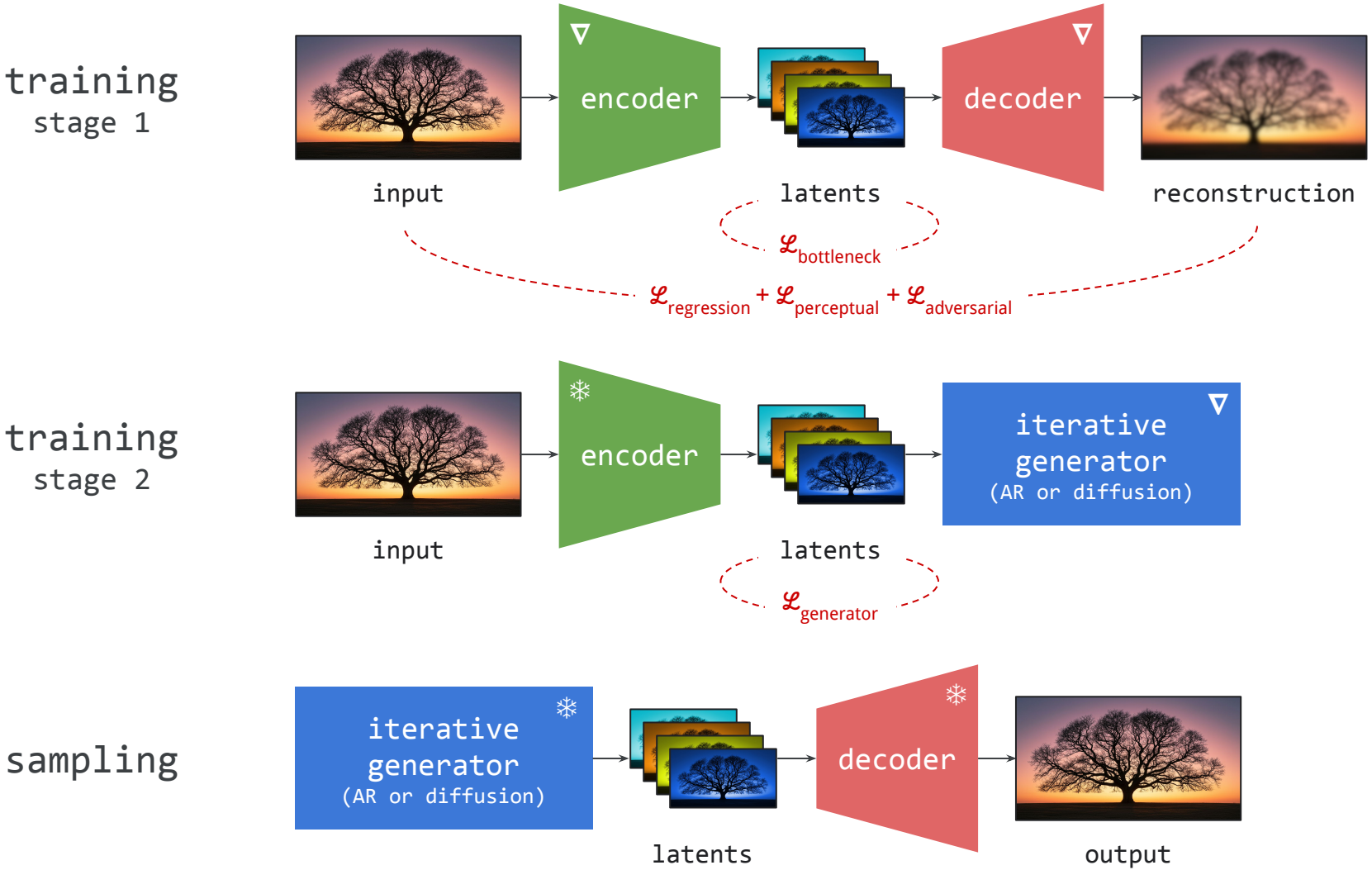
Song et al. 'Maximum Likelihood Training of Score-Based Diffusion Models' (2021)
Kingma et al. 'Variational Diffusion Models' (2021)

Latent diffusion

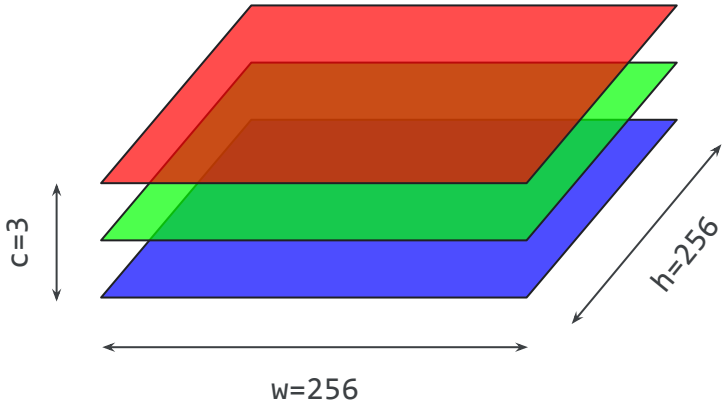
- Visual perception works differently at fine scales and large scales
- Fine-scale perception makes abstraction of **texture**
- It is **not necessary** to model all possible realisations of particular textures

  ⇒ use an **adversarial autoencoder** to learn to “paint with textures”
- Diffusion in latent space is usually a simpler, less memory-hungry task

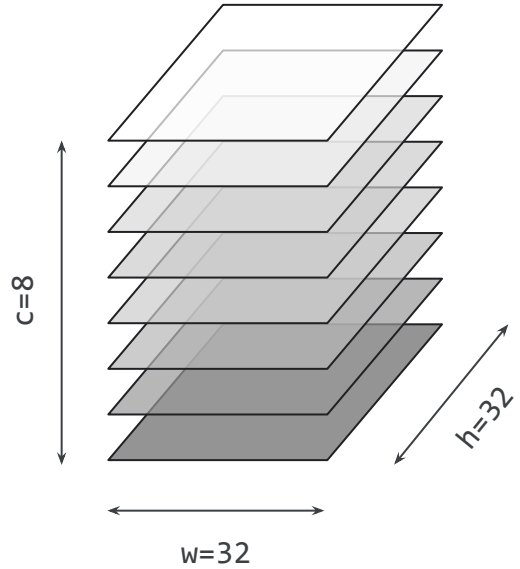
https://sander.ai/2020/09/01/typicality.html
Rombach et al. ‘High-Resolution Image Synthesis with Latent Diffusion Models’ (2021)

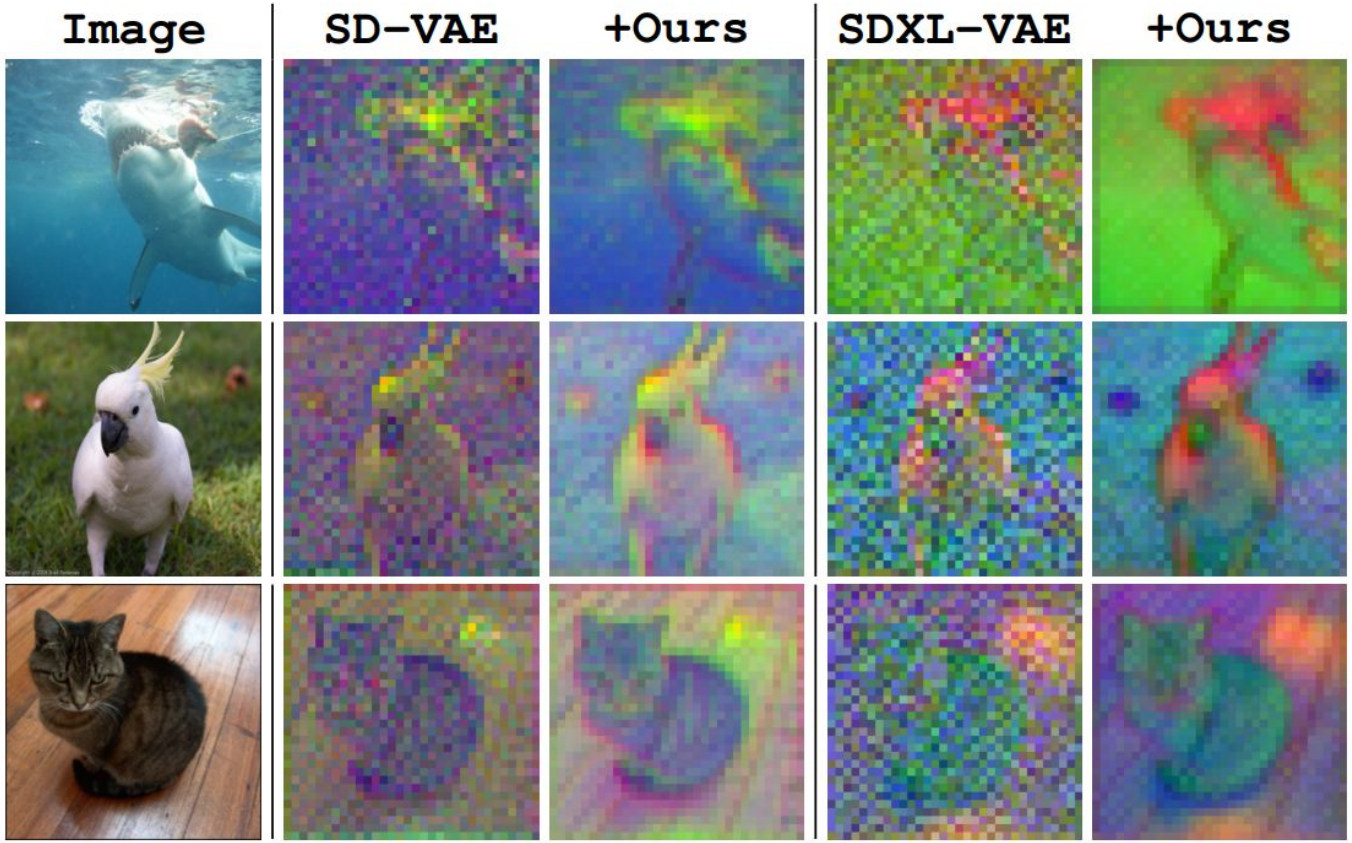

training
stage 1
encoder
decoder
input
latents
reconstruction
$\mathcal{L}_{\text{bottleneck}}$
$\mathcal{L}_{\text{regression}} + \mathcal{L}_{\text{perceptual}} + \mathcal{L}_{\text{adversarial}}$
training
stage 2
encoder
iterative generator
(AR or diffusion)
input
latents
$\mathcal{L}_{\text{generator}}$
sampling
iterative generator
(AR or diffusion)
decoder
latents
output

pixels



latents





'EQ-VAE: Equivariance Regularized Latent Space for Improved Generative Image Modeling', Kouzelis, Kakogeorgiou, Gidaris, Komodakis, arXiv, 2025.

Model architecture

- Predict $\log p(x)$ (scalar)...
  ... or $\nabla \log p(x)$ (vector)?

Salimans & Ho 'Should EBMs model the energy or the score?' (2021)

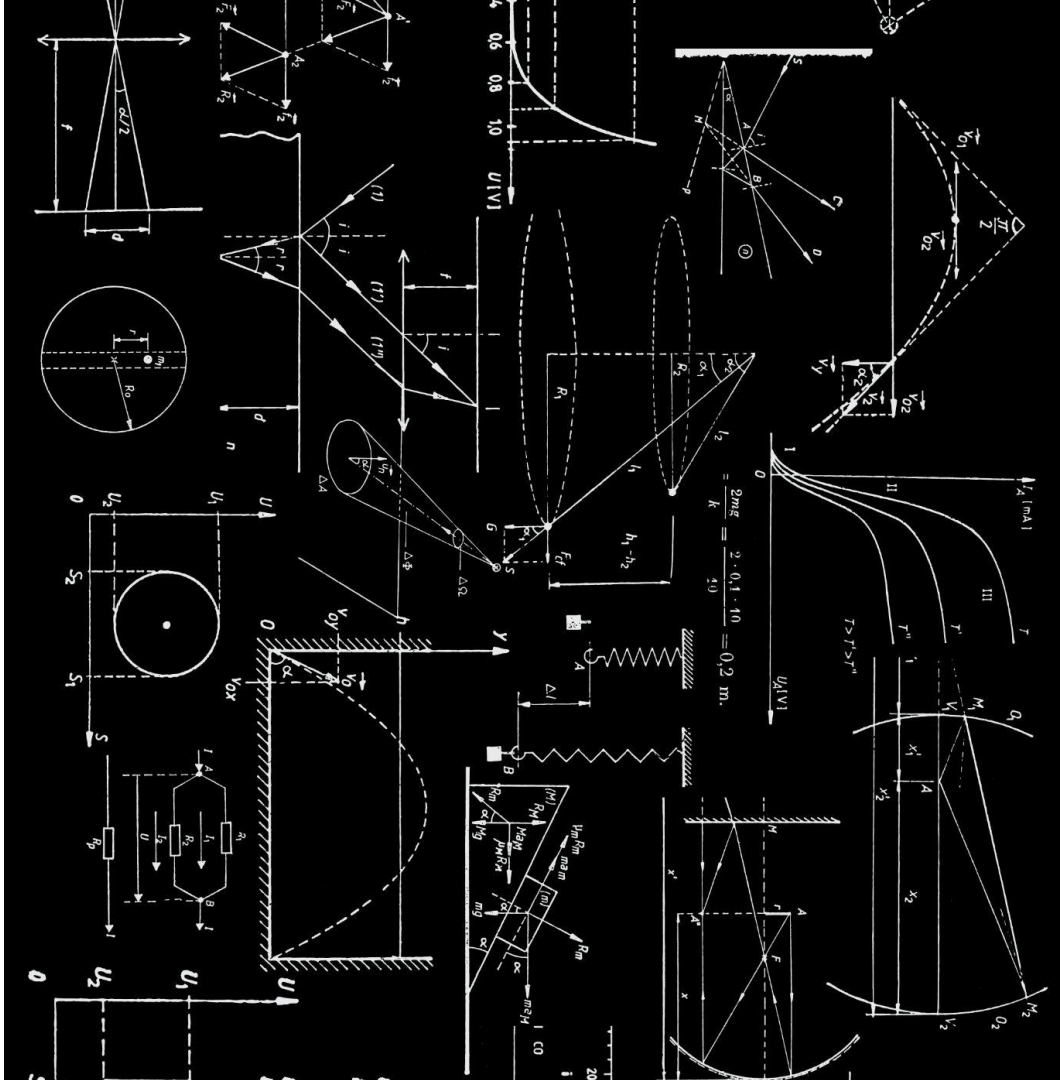
- No architectural restrictions!
  (unlike normalising flows, AR models)

- Historically mostly **U-Net**s

Ronneberger et al. 'U-net: Convolutional networks for biomedical image segmentation' (2015)

- ... but **Transformers** are taking over

Peebles & Xie 'Scalable Diffusion Models with Transformers' (2022)



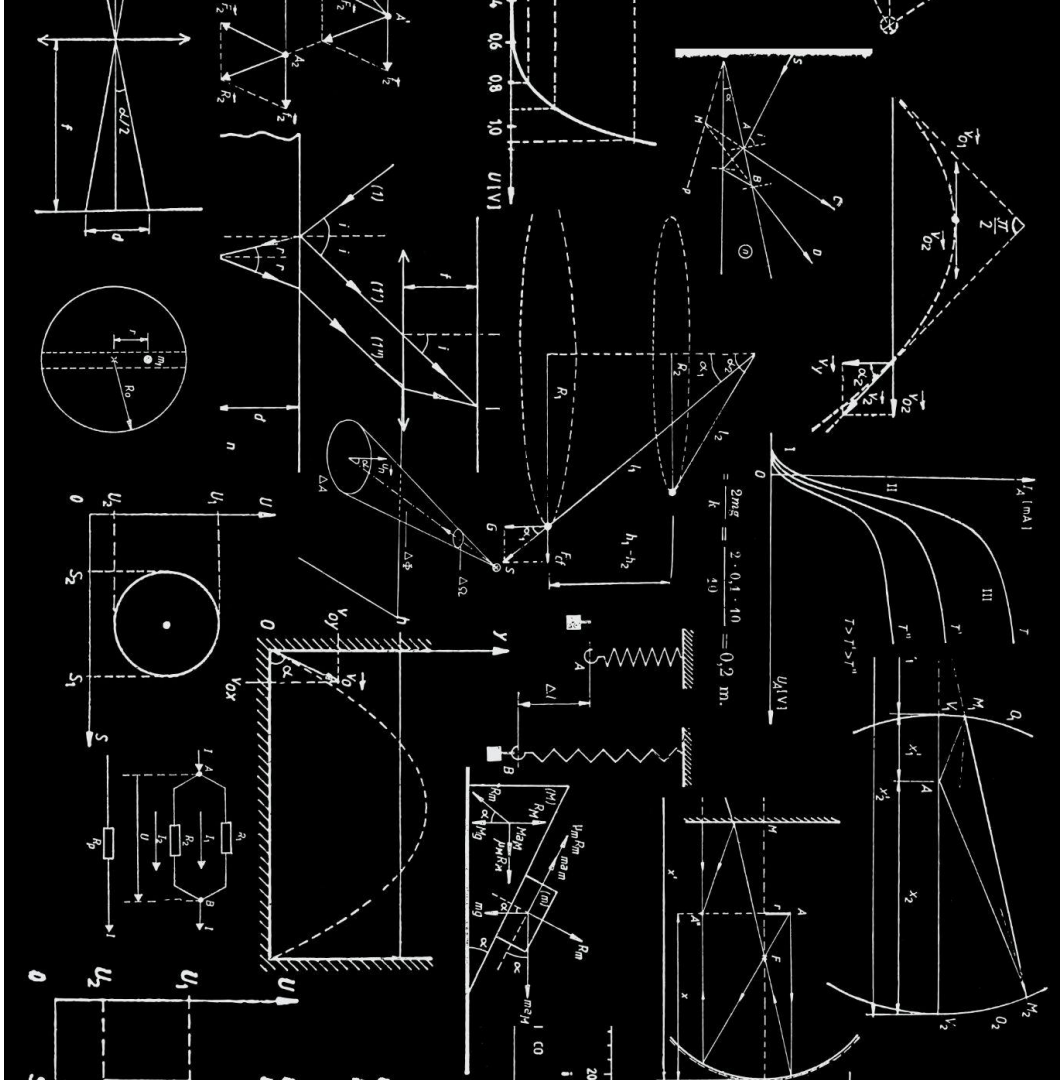
Model architecture

- No architectural restrictions!
  (unlike normalising flows, AR models)
- Historically mostly **U-Net**s

Ronneberger et al. ‘U-net: Convolutional networks for biomedical image segmentation’ (2015)

- ... but **Transformers** are taking over

Peebles & Xie ‘Scalable Diffusion Models with Transformers’ (2022)



Diffusion for discrete data

$\nabla_x \log p(x)$ is not defined when x is discrete!

- Option 1: define a **discrete corruption process**

Hoogeboom et al. 'Argmax Flows and Multinomial Diffusion: Learning Categorical Distributions' (2021)

Hoogeboom et al. 'Autoregressive Diffusion Models' (2021)

Savinov et al. 'Step-unrolled Denoising Autoencoders for Text Generation' (2021)

Chang et al. 'MaskGIT: Masked Generative Image Transformer' (2022)

- Option 2: **embed** x in continuous space

Chen et al. 'Analog Bits: Generating Discrete Data using Diffusion Models with Self-Conditioning' (2022)

Li et al. 'Diffusion-LM Improves Controllable Text Generation' (2022)

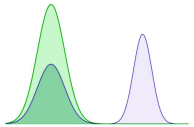
Strudel et al. 'Self-conditioned Embedding Diffusion for Text Generation' (2022)

Dieleman et al. 'Continuous diffusion for categorical data' (2022)

Gao et al. 'Difformer: Empowering Diffusion Model on Embedding Space for Text Generation' (2022)

What about GANs?

- One-shot generation, no iterative refinement
- Possible due to **mode-seeking** nature



Sauer et al. 'StyleGAN-T: Unlocking the Power of GANs for Fast Large-Scale Text-to-Image Synthesis' (2023)

Kang et al. 'Scaling up GANs for Text-to-Image Synthesis' (2023)

- Combining GANs and diffusion?

Jolicoeur-Martineau et al. 'Adversarial score matching and improved sampling for image generation' (2020)

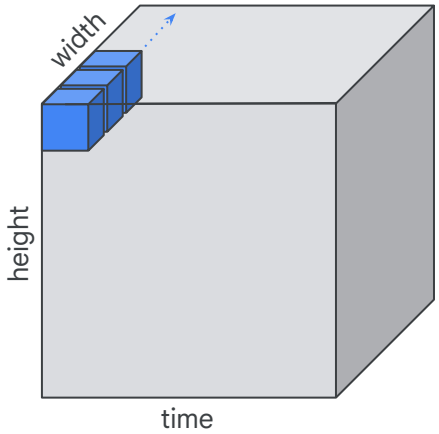
Rombach et al. 'High-Resolution Image Synthesis with Latent Diffusion Models' (2021)

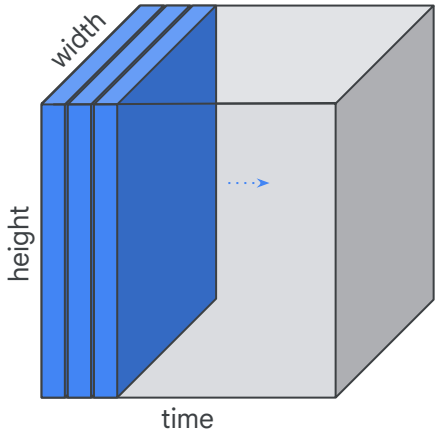
Xiao et al. 'Tackling the Generative Learning Trilemma with Denoising Diffusion GANs' (2022)

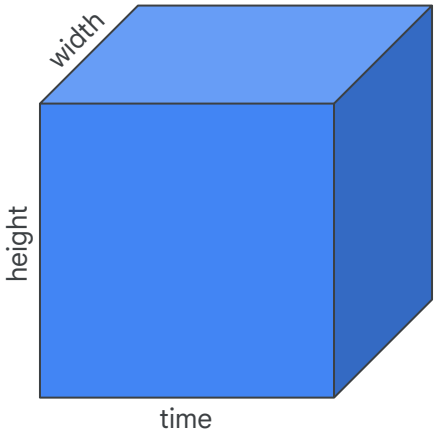
Wang et al. 'Diffusion-GAN: Training GANs with Diffusion' (2022)

Autoregression vs. diffusion for video

Autoregression | Hybrid | Diffusion



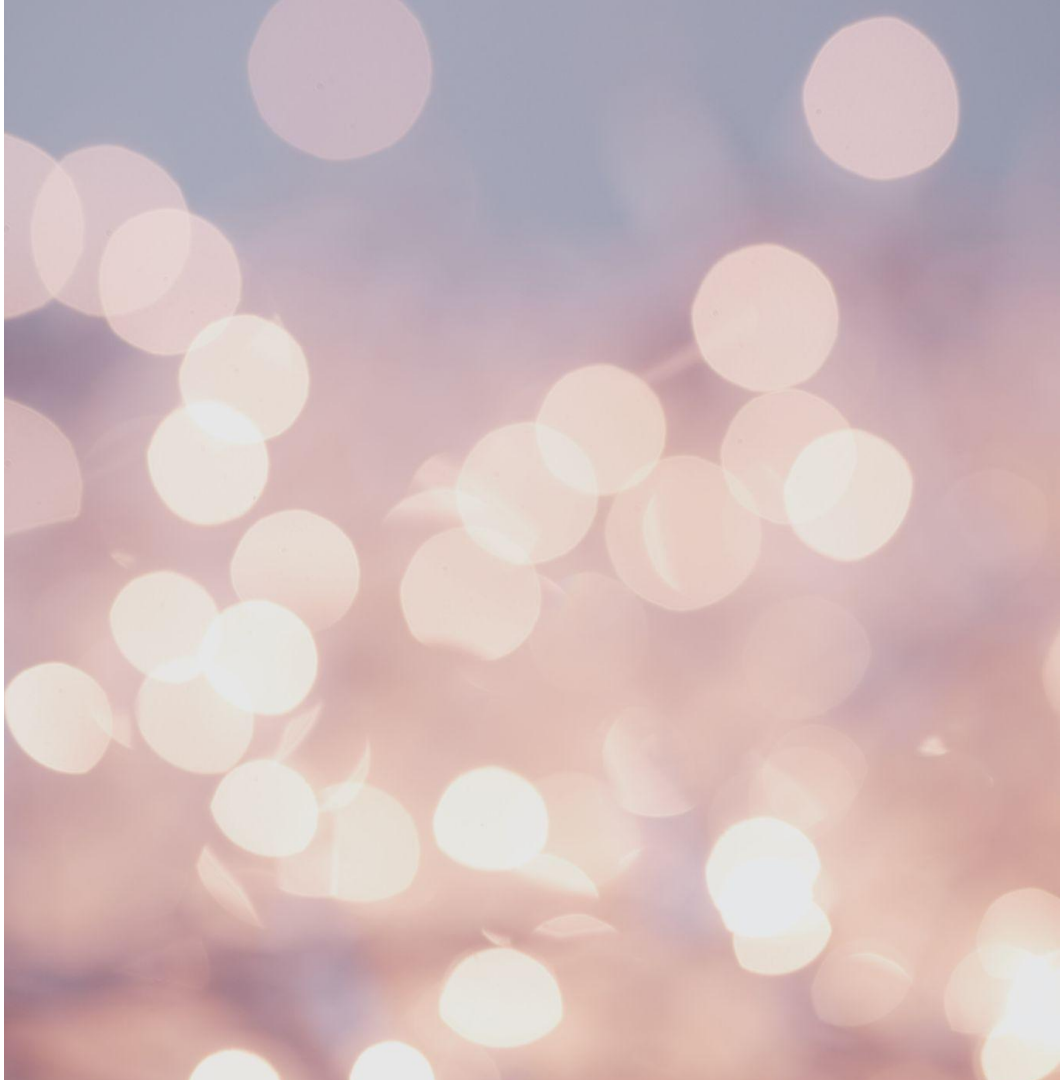




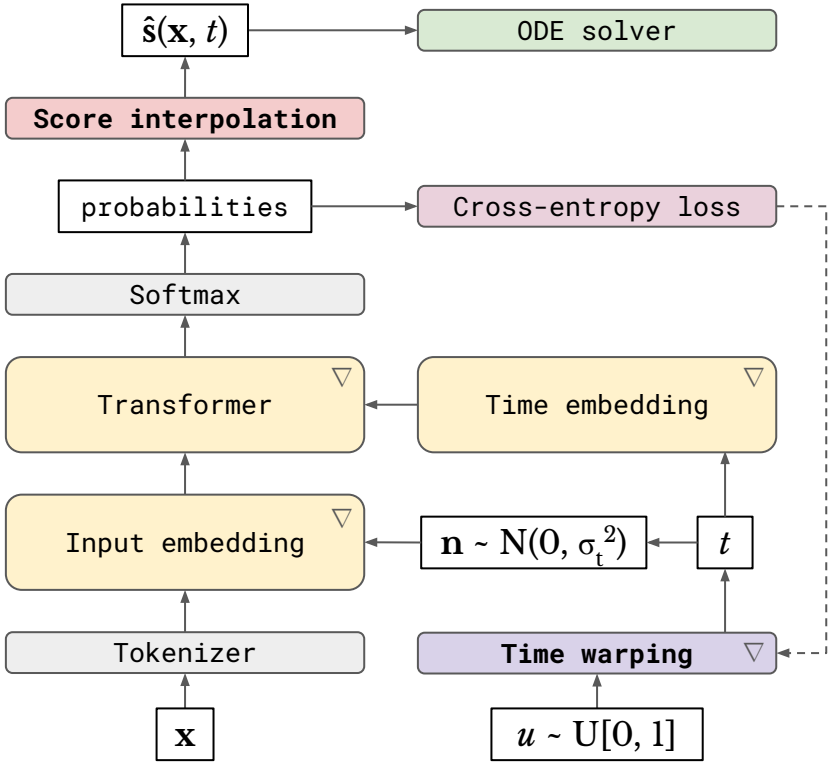
Continuous diffusion for categorical data (CDCD)

- Continuous diffusion in time and input
- Guidance, interpolation, ODE-based sampling, ...
- Make diffusion look familiar to LLM practitioners:
  - Use a **Transformer** (unmasked)
  - **Cross-entropy** loss instead of MSE

Dieleman et al. 'Continuous diffusion for categorical data' (2022)



The CDCD framework

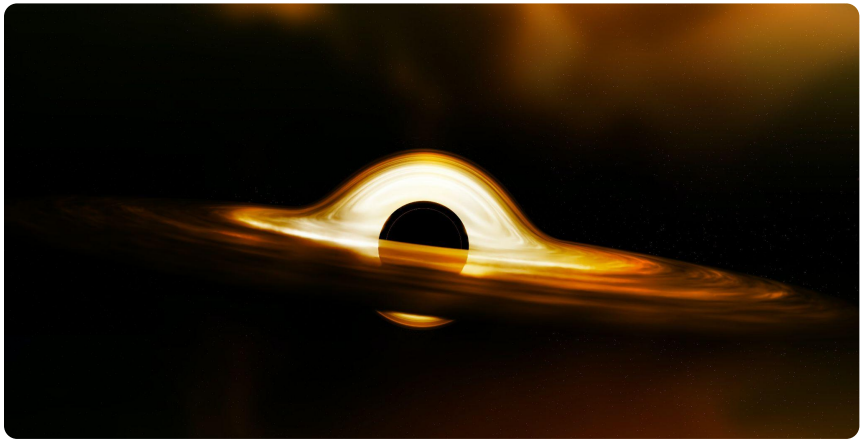


Score interpolation vs. score matching

- Underlying data is discrete
- Finite vocabulary size
- Train a **classifier, interpolate** conditional scores with predictions
- **Stable training** with categorical cross-entropy

$$\hat{\mathbf{s}}_t(\mathbf{x}_t) = \sum_{i=1}^{V} p_t(\mathbf{x}_0 = \mathbf{e}_i|\mathbf{x}_t)\mathbf{s}_t(\mathbf{x}_t|\mathbf{x}_0 = \mathbf{e}_i)$$

Joint learning of denoiser and embeddings



Score matching

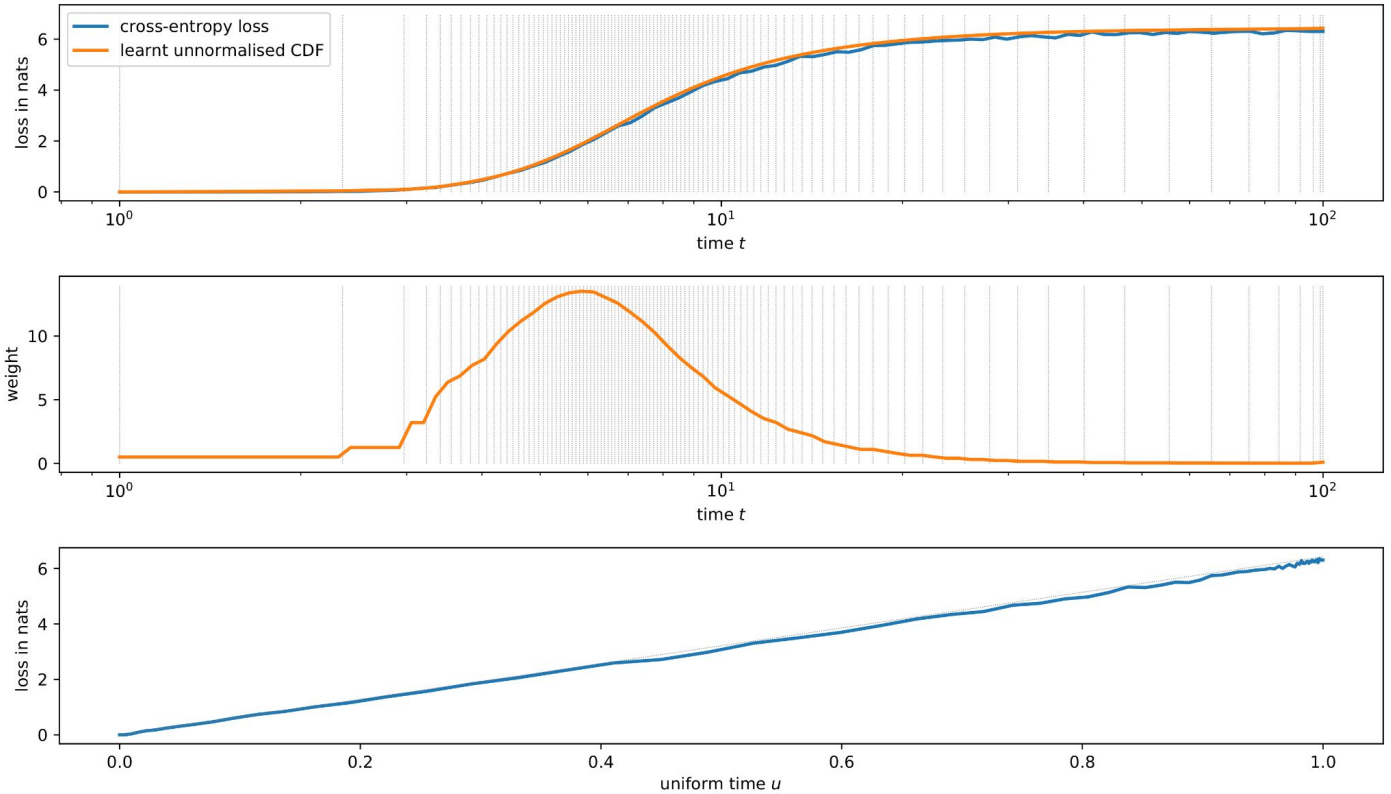
Loss is minimal when all embeddings are identical: they **collapse to a single point**

Score interpolation

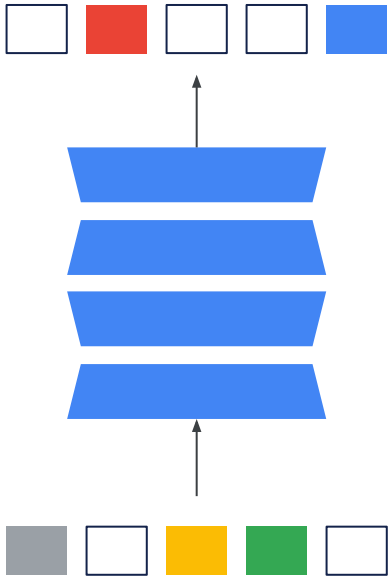
Loss is minimal when all embeddings are maximally distinct: they will **grow uncontrollably**

Easy fix: **normalise them**!

Time warping



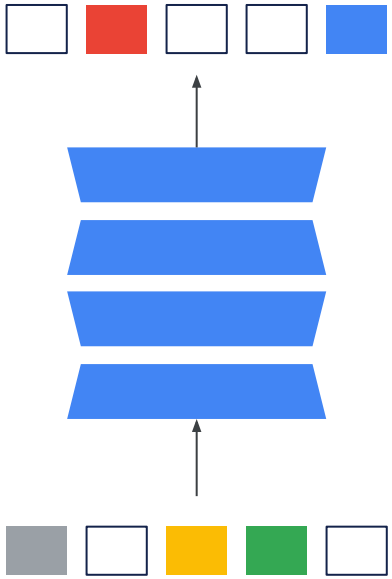
Diffusion language models



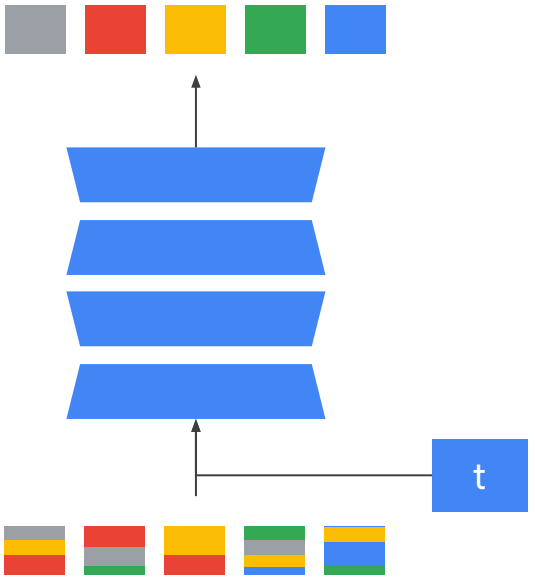
BERT

Devlin et al. 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding' (2018)

Diffusion language models



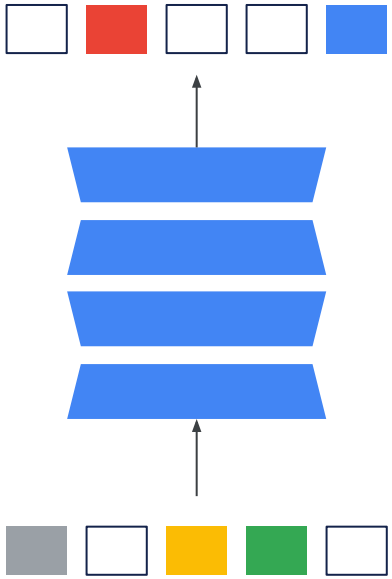
BERT

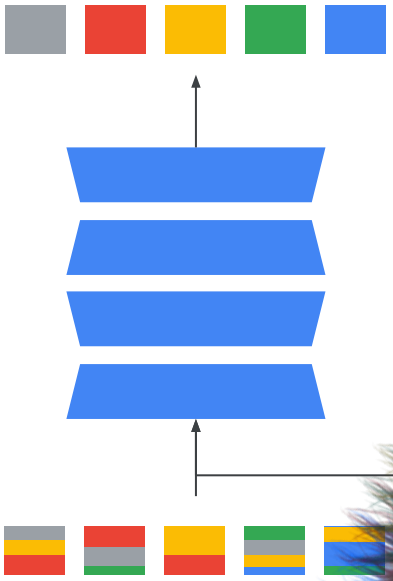


CDCD

Devlin et al. 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding' (2018)

Diffusion language models





BERT

CDCD

Devlin et al. 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding' (2018)

Generative models of text are very versatile: they can be used to interpret texts in various formats including newspapers, magazine articles, bibliographies, scientific journals, etc. Thus, text models (e.g. Matlab, text theory) can extract semantic information from the textual data in a versatile way, so that they can be used to infer the structure and logic of the textual processing processes.

To be or not to be? That is the question. Whether it is a religion, science, art, philosophy, economics, history, sociology, politics or business, it needs to be about the right questions. We say that this question is not the question of absolute answers but also possible answers. is this question the existence of knowledge?

To be or not to be? That is the question. Whether it is the meaning of art, the meaning of wine, or the meaning of life. It is a question that explores the meaning of something beyond, something outside of the 'self or the main category.

To be or not to be? That is the question. Whether it is happiness or depression, compassion or love, acceptance or loneliness, joy or fear of yourself, the answer is the same. The answer is You.

The average air speed velocity of an unladen swallow is about 3.5–4 km per hour (1.75–1.75 mph), and the lightweight prey may accelerate itself up to six times when flying for the full speed. Even though the swallows are relatively small, similar to honey and rice at their average flying speed, this extraordinary propensity allows them to fly very fast in a plane with only 2% of their weight of solid water distributed on them. Chasinocodon can dive at speeds up to 35 km/h (21 mi/h) and can also dive up to 324 meters in length (750–500 feet).

The cat (Felis catus) is a domestic species of small carnivorous mammal. It is the only domesticated species in the family Felidae and is commonly referred to as the domestic cat or house cat to distinguish it from the wild members of the family. Cats are recognized as being the most domesticated species and are thus among the most intelligent, although there are reports that they have super-human intelligence, whereas humans are only known for regard to their years. They are highly social, discriminate in speaking and hearing various forms of human and mental expressions, and have been often referred to as the most fundamental influences on human language.

This easy chicken curry recipe is made with just a handful of ingredients, including a healthy combination of flavored rice that you can easily make at home.

Preheat oven to 350°F (175°C). Line a pan with foil. Combine paprika, garlic, ginger, baking powder and ½ cup (2 tsp) salt. Set aside for 15 minutes.

Prepare rice according to directions. Or, cook rice in boiling water for 1 minute and drain completely. Heat a heavy-bottomed saucepan over high heat. Once the oil has been heated, add the garlic and green onions; stir-fry for 30 seconds until fragrant.

Add the chicken and stir-fry for about 2 minutes. Add ginger, honey, tomatoes, coconut milk and bay leaves; bring to a boil. Reduce heat to low to cover and cook for 10 minutes.

stir-fry in the salt. Add the cooked rice and mix well. Transfer to oven and cook for about 20 minutes or until chicken is well cooked and rice is tender.

A year ago in Paris, we spent three hours at the Eiffel Tower in the morning. We enjoyed the show, visited the tourist sites, went to the ceremony, took photos, and visited the cafe. Wow, **what a great day!**

Flower Shop is pleased to show you our wonderful collection of Gift and Fruit Baskets perfect for sending with New Bedfordshire Flowers. You will be able to find gifts to suit every special occasion such as Valentine's Day, Mother's Day or **Father's day.** We also have a large selection of fresh cakes, gift baskets, balloons and many other gifts.

Diffusion models have quickly become the go-to paradigm for generative modelling of perceptual signals(such as images and sound) through iterative refinement. In this paper, we propose a straightforward computational approach that identifies the signals to model through refinement without relying solely on the extraction and storage of variances of differences. We call our variational diffusion model a multidimensional model rather than an unsupervised model. The formulation is continuous, is able to quantify independent layer changes and is robust across a variety of datasets. The approach is also fully domain-independent and is easily compatible with any other diffusion model on the net. **We demonstrate its efficacy on several language modelling tasks.**

A brief history of depth

before 2006: pre-deep learning era

A brief history of depth

before 2006: pre-deep learning era

2006 - 2011: layer-wise pre-training
⇒ 2-4 layers

A brief history of depth

before 2006: pre-deep learning era

2006 - 2011: layer-wise pre-training
⇒ 2-4 layers

2012 - 2014: ReLUs and dropout
⇒ ~10 layers

A brief history of depth

before 2006: pre-deep learning era

2006 - 2011: layer-wise pre-training
⇒ 2-4 layers

2012 - 2014: ReLUs and dropout
⇒ ~10 layers

2015 - 2019: Residual networks, normalisation
⇒ ~100 layers

A brief history of depth

before 2006: pre-deep learning era

2006 - 2011: layer-wise pre-training
⇒ 2-4 layers

2012 - 2014: ReLUs and dropout
⇒ ~10 layers

2015 - 2019: Residual networks, normalisation
⇒ ~100 layers

2020 - now: Iterative refinement
⇒ **~100k** layers 🫠

Hinton et al. "A Fast Learning Algorithm for Deep Belief Nets" (2006)
Nair & Hinton "Rectified Linear Units Improve Restricted Boltzmann Machines" (2010)
Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting" (2014)
He et al. "Deep Residual Learning for Image Recognition" (2015)
Song & Ermon "Generative Modeling by Estimating Gradients of the Data Distribution" (2019)
Ho et al. "Denoising Diffusion Probabilistic Models" (2020)
...

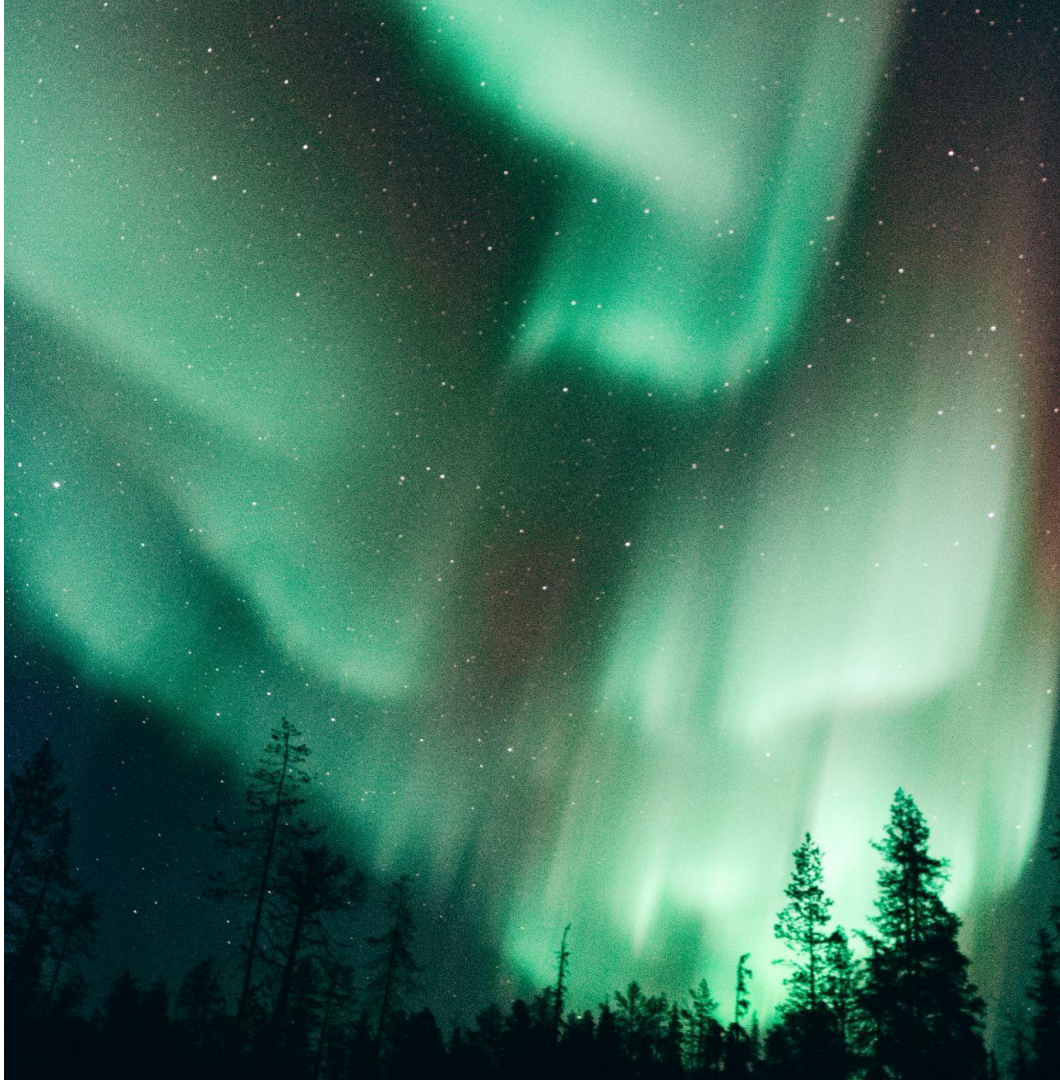
Deep computation graphs without backpropagation

- Iterative refinement: **substantially deeper** deep learning
- Maximal practically achievable depth with backprop is limited:
  - Optimisation difficulties
  - Computational cost
  - Hard to parallelise
- Complex generative tasks require **very deep computation graphs**
- When backprop becomes impractical, iterative refinement is the answer

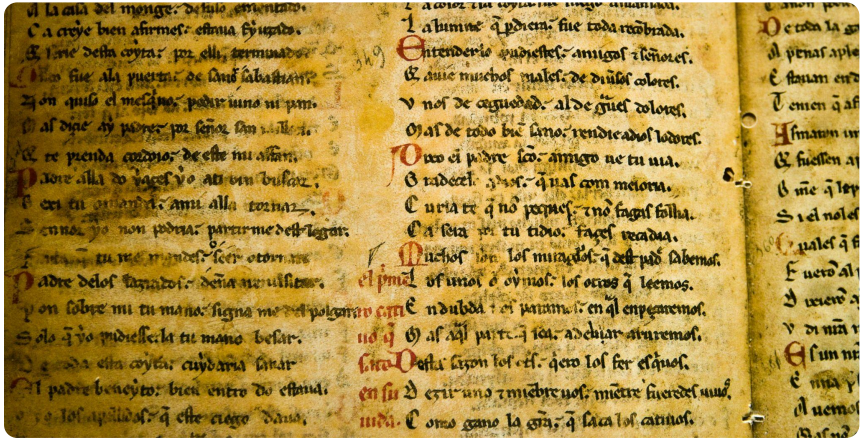
Very deep VAEs

- Modern generative models
  divide and conquer
- With orders of magnitude **more compute**:
  - we don't need to divide
  - we can just conquer
- If we wait ~10 years,
  we could just train VAEs for everything?
- No more iterative refinement!

Child 'Very Deep VAEs Generalize Autoregressive Models and Can Outperform Them on Images' (2020)



Language vs. perceptual signals



Language

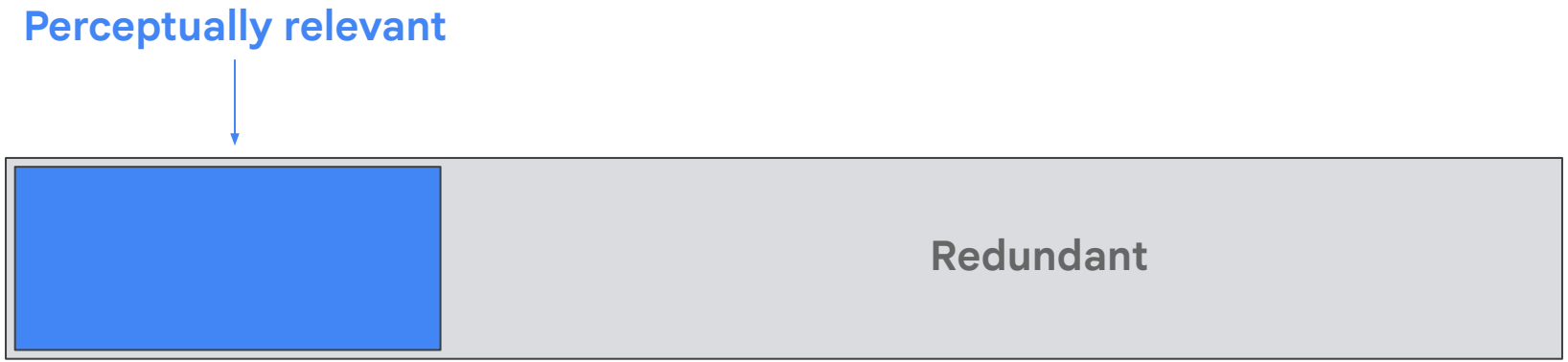
Highly compressed, 1D sequence,
evolved as a means of efficient communication

Perceptual signals

Highly redundant, 1D / 2D / 3D / ...
lots of correlations at various scales

Information in perceptual signals

Information in perceptual signals



Information in perceptual signals

Tokenising perceptual signals

- For **understanding**, we only care about semantically meaningful information
- For **generation**, we care about all perceptually relevant information
- **Adversarial losses** are very common
  - Fast feed-forward decoding
  - Intentional mode-dropping
- Perception works differently at fine and coarse scales!

https://sander.ai/2020/09/01/typicality.html

Three contrasts

Language 🆚 perceptual signals:

- Autoregression 🆚 diffusion
- Discrete 🆚 continuous
- Likelihood 🆚 perceptual loss

Blurred boundaries

- **Autoregressive image models**:
  PixelCNN, VQGAN Transformer, xAI Aurora
- **Diffusion with maximum likelihood**:
  Variational diffusion models, PLAID
- **Continuous diffusion for language**:
  CDCD, SED, PLAID
- **Discrete diffusion for language**:
  MD4, MDLM
- **Autoregression in continuous space**:
  GIVT, JetFormer

References in notes!

Elucidating the Design Space of Diffusion-Based Generative Models

Tero Karras NVIDIA
Miika Aittala NVIDIA
Timo Aila NVIDIA
Samuli Laine NVIDIA

Abstract

We argue that the theory and practice of diffusion-based generative models are currently unnecessarily convoluted and seek to remedy the situation by presenting a design space that clearly separates the concrete design choices. This lets us identify several changes to both the sampling and training processes, as well as preconditioning of the score networks. Together, our improvements yield new state-of-the-art FID of 1.79 for CIFAR-10 in a class-conditional setting and 1.97 in an unconditional setting, with much faster sampling (35 network evaluations per image) than prior designs. To further demonstrate their modular nature, we show that our design changes dramatically improve both the efficiency and quality obtainable with pre-trained score networks from previous work, including improving the FID of an existing ImageNet-64 model from 2.07 to near-SOTA 1.55.

Understanding Diffusion Objectives as the ELBO with Simple Data Augmentation

Diederik P. Kingma
Google DeepMind
durk@google.com

Ruiqi Gao
Google DeepMind
ruiqig@google.com

Abstract

To achieve the highest perceptual quality, state-of-the-art diffusion models are optimized with objectives that typically look very different from the maximum likelihood and the Evidence Lower Bound (ELBO) objectives. In this work, we reveal that diffusion model objectives are actually closely related to the ELBO.

Specifically, we show that all commonly used diffusion model objectives equate to a weighted integral of ELBOs over different noise levels, where the weighting depends on the specific objective used. Under the condition of monotonic weighting, the connection is even closer: the diffusion objective then equals the ELBO, combined with simple data augmentation, namely Gaussian noise perturbation. We show that this condition holds for a number of state-of-the-art diffusion models.

In experiments, we explore new monotonic weightings and demonstrate their effectiveness, achieving state-of-the-art FID scores on the high-resolution ImageNet benchmark.

Thank you! Grazie!

Blog: https://sander.ai/

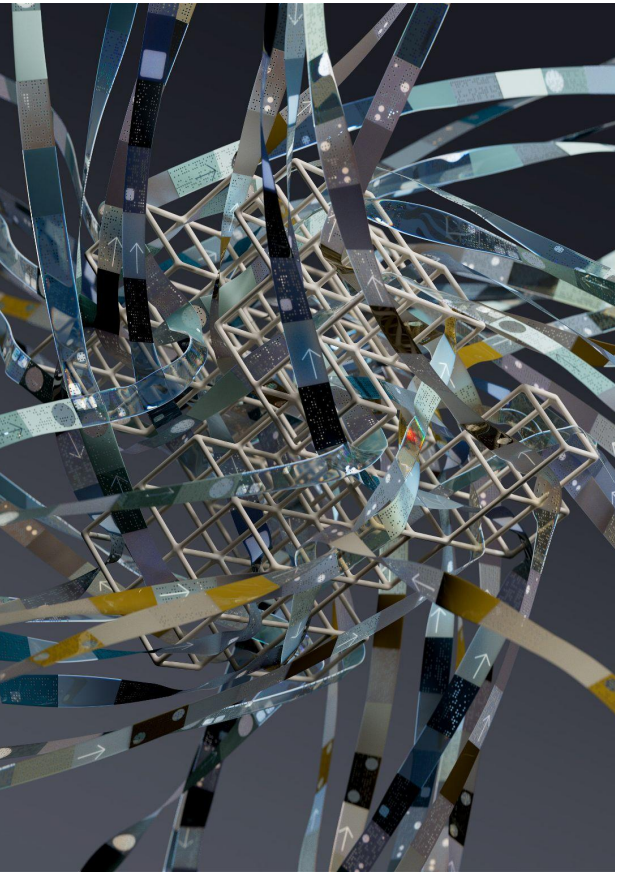
Twitter/X: @sedielem

BlueSky: @sedielem.bsky.social

Email: sanderdieleman@gmail.com

Google DeepMind

Google DeepMind

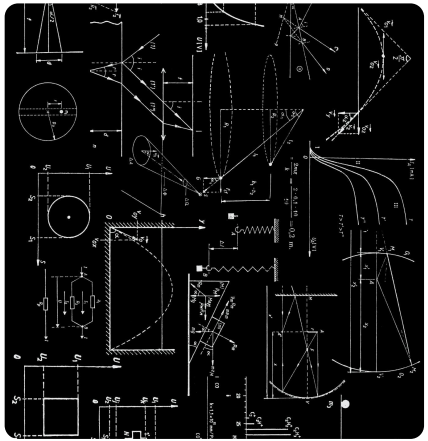


Training neural networks

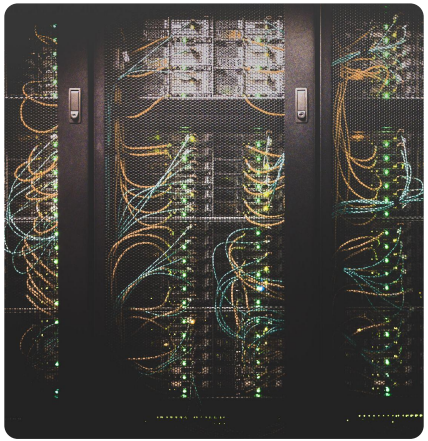
Sander Dieleman
Research Scientist, Google DeepMind

Overview

I. Process



II. Architecture



III. Training

IV. Miscellany

Google DeepMind

I. Process

Goal-driven ML research

- Most ideas **don’t work**
- Don’t get discouraged!
- Try LOTS of ideas
- Don’t get too invested in one specific idea
- Throw things at the wall, see what sticks
- Parallelise exploration (if possible)

Metrics

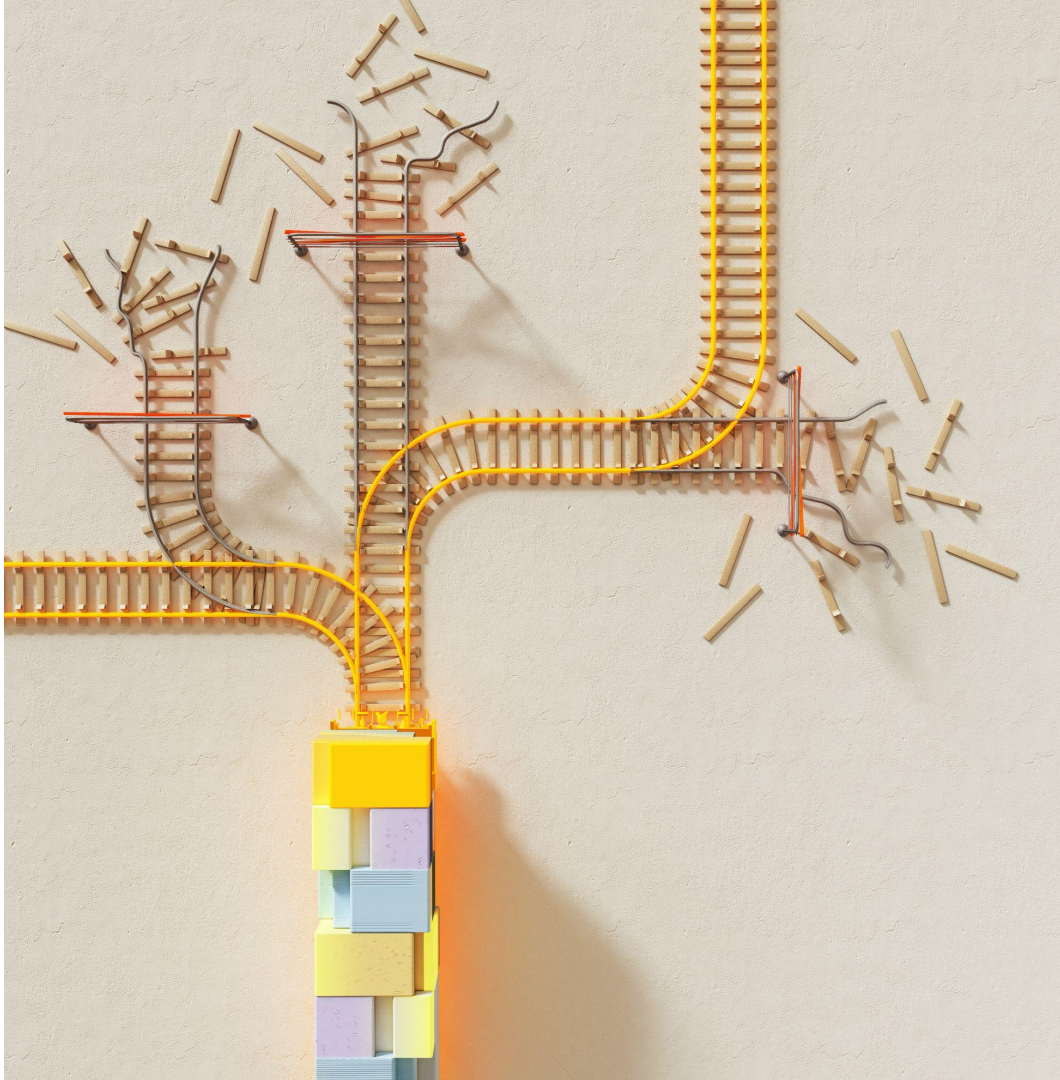
- Decide on a metric first
- Automated if possible
- Human-in-the-loop evaluation makes iteration more difficult
- Even problematic metrics can still be useful (e.g. FID and BLEU persist)
- Metric ≠ loss function

Iteration speed is key

- Optimise for iteration speed, not raw performance
- Resist the urge to scale (at least initially)
- Scaling is easy (up to a point)
- Running more experiments faster is more important
- **Exception**: O(10B) parameters and up

Embrace uncertainty

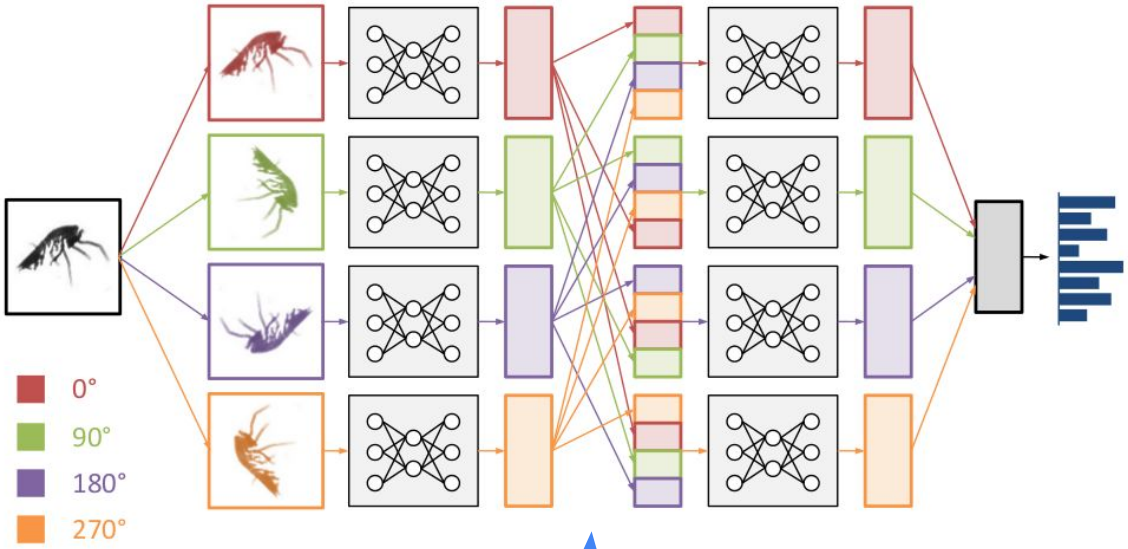
- ML experiments are expensive
- Results are noisy
- Resources are limited
- Repeating experiments many times is inefficient
- Better strategy: **commit & revisit**



Invest in engineering

- Use hardware efficiently
- Use a **profiler**
- Don’t fool yourself: look at performance vs. **wall-clock time**, not # training steps
- 20% faster experiments
  → 25% more experiments!
- Rule of thumb:
  Matrix multiplications are fast
  Everything should be a matmul

Invest in engineering: example



custom CUDA kernel

https://sander.ai/2015/03/17/plankton.html

Invest in engineering: example

```
__global__ void cyclic_roll(float * input, float * output, int batch_size, int num_features) {
    int x = blockIdx.x*blockDim.x + threadIdx.x; // feature dim, fastest varying index!
    int y = blockIdx.y*blockDim.y + threadIdx.y; // batch dim
    int height = 4 * batch_size;
    int width = 4 * num_features;

    if (x < num_features && y < height) {
        for (int i = 0; i < 4; i++) {
            int y_out = (y + batch_size * (4 - i)) % height;
            int x_out = x + num_features * i;
            output[y_out * width + x_out] = input[y * num_features + x];
        }
    }
}
```

https://github.com/benanne/kaggle-ndsb/blob/master/dihedral_ops.py

Two modes of exploration

Optimising an objective

What improves the metric?

Building understanding

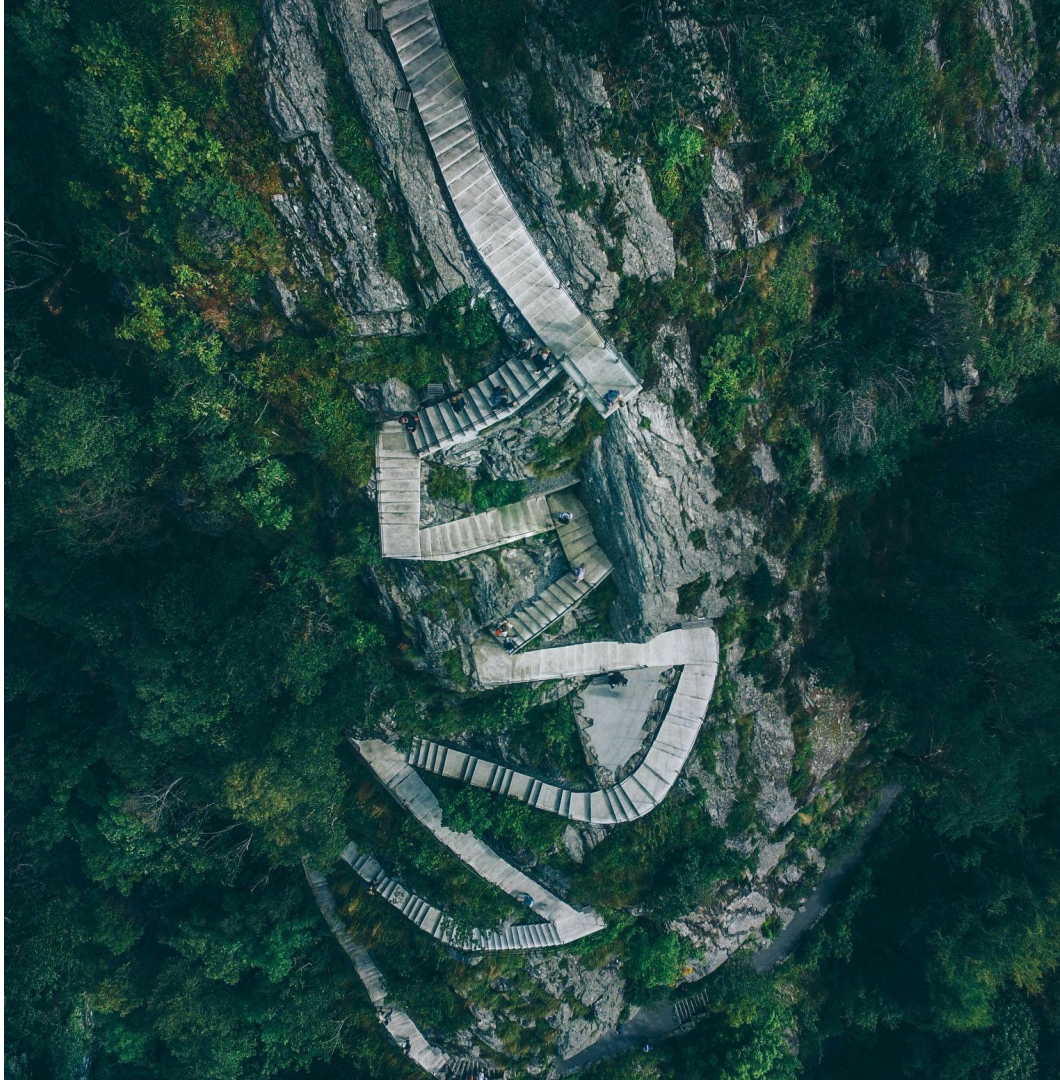
Why does it improve the metric?

How do things interact with each other?

...

Hyperparameter sweeping

- Most parameters should be swept on a logarithmic scale
  `…, 0.1, 0.3, 1.0, 3.0, 10.0, …`
- Grid search is useful to build understanding
- When chasing SoTA, consider random search instead



Random search is underrated

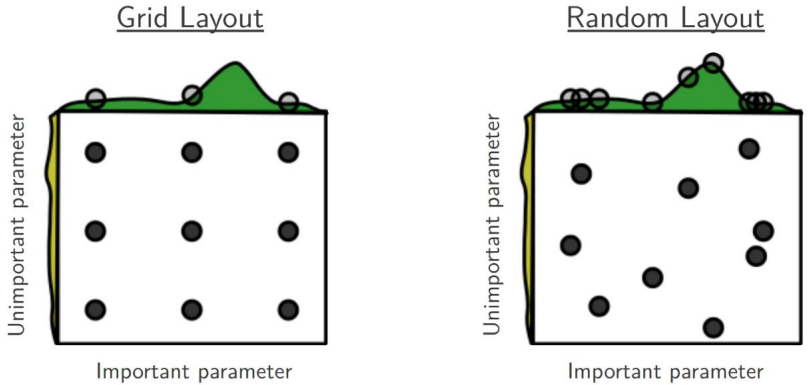
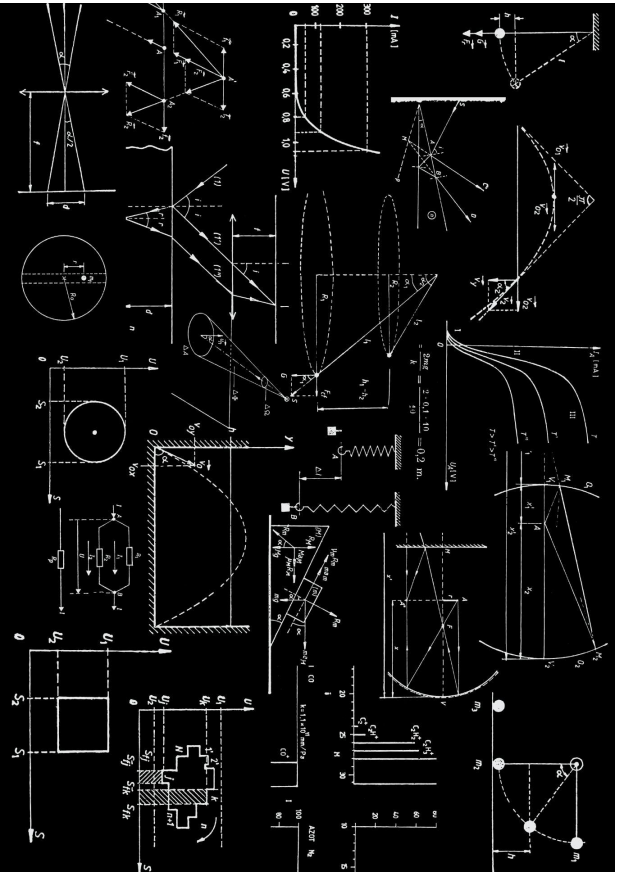


Figure 1: Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g. This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

'Random Search for Hyper-Parameter Optimization', Bergstra & Bengio, JMLR 2012

Google DeepMind



II. Architecture

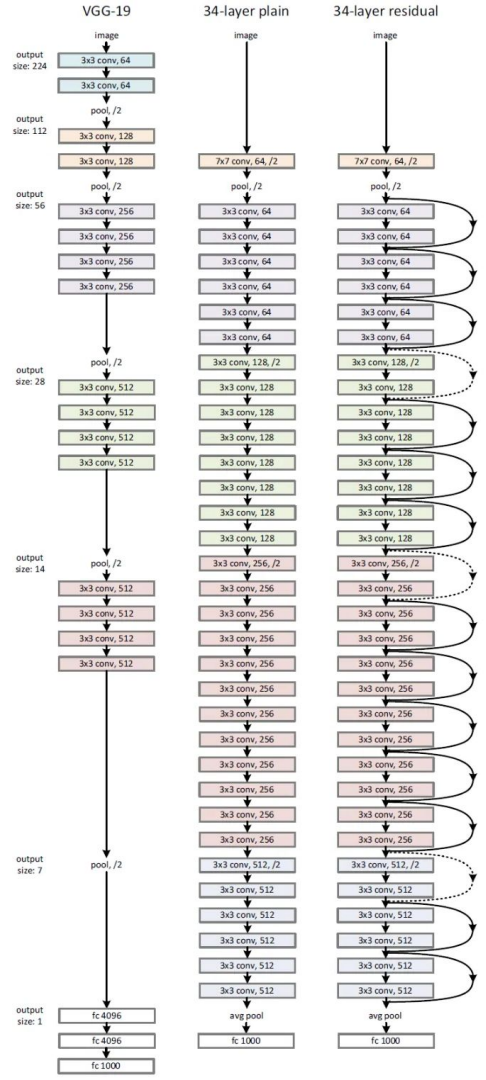
We'll skip the basics

- Linear layers
- Nonlinearities (ReLU, GeLU etc.)
- Convolutions and pooling
- Recurrence
- Attention
- Initialisation
- ...

Residual connections

- The simplest / most effective trick in neural network design
  (maybe tied with dropout?)
- Ensures good gradient flow
  Essential for depth beyond ~10 layers
- Modern version: **divide by √2**
- … or **initialise residual to 0**

'Deep Residual Learning for Image Recognition',
He, Zhang, Ren, Sun, CVPR 2016



Normalisation layers

$$\mathbf{y} = a \cdot \frac{\mathbf{x} - \mu}{\sigma} + b$$

- **μ, σ**: estimated from **x**
  - over **batch**: `BatchNorm`
  - over **channels**: `LayerNorm, GroupNorm`
  - over **space/time**: `InstanceNorm`
  - no μ: `RMSNorm`
  - ...
- **a, b**: learnable parameters ('FiLM')

'Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift', Ioffe, Szegedy, ICML 2015
'Layer Normalization', Ba, Kiros, Hinton, arXiv, 2016
'Group Normalization', Wu, He, ECCV 2018
'Instance Normalization: The Missing Ingredient for Fast Stylization', Ulyanov, Vedaldi, Lempitsky, arXiv, 2016
'Root Mean Square Layer Normalization', Zhang, Sennrich, NeurIPS 2019
'FiLM: Visual Reasoning with a General Conditioning Layer', Perez, Strub, de Vries, Dumoulin, Courville, AAAI 2018

Normalisation layers

- Normalisation can go **before** or **after** linear operations
- Motivation: keep the residual path fully linear to maximise gradient flow

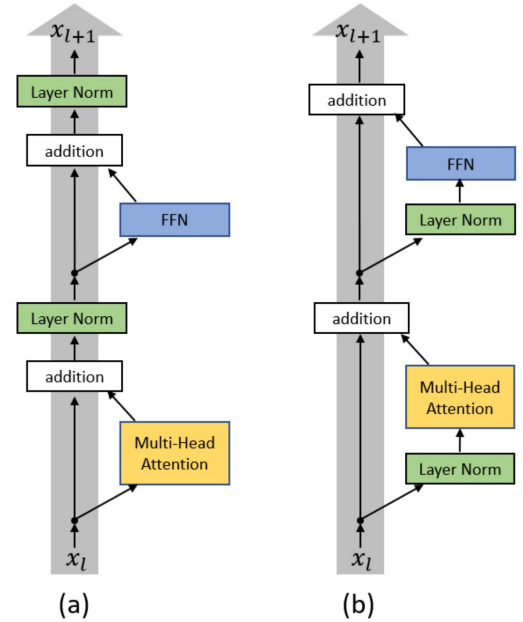


Figure 1. (a) Post-LN Transformer layer; (b) Pre-LN Transformer layer.

'Transformers without Tears: Improving the Normalization of Self-Attention', Nguyen, Salazar, IWSLT 2019
'On Layer Normalization in the Transformer Architecture', Xiong, Yang, He, Zheng, Zheng, Xing, Zhang, Lan, Wang, Liu, ICML 2020

BatchNorm has fallen out of favour

- **Intra-batch dependency**: μ, σ depend on other examples in the batch
- Acts as a **regulariser**, but strength is dependent on batch size
- **Stateful**: need μ, σ at test time
- **Bug magnet!** 🧲

Sander Dieleman @sedielem · Jan 25

Batch normalisation appears to be falling out of favour (probably for the best IMO, so many bugs end up being batchnorm bugs😬).

One area where it persists is GAN discriminators (e.g. in StyleGAN-T and VQGAN). Are there any other settings where batchnorm is still hard to avoid?

19 22 239 175.9K

Tim Zaman @tim_zaman

Top hours wasted in my life
1) gaming
2) tv
3) batchnorm bugs

Probably closely followed by gpu driver updates

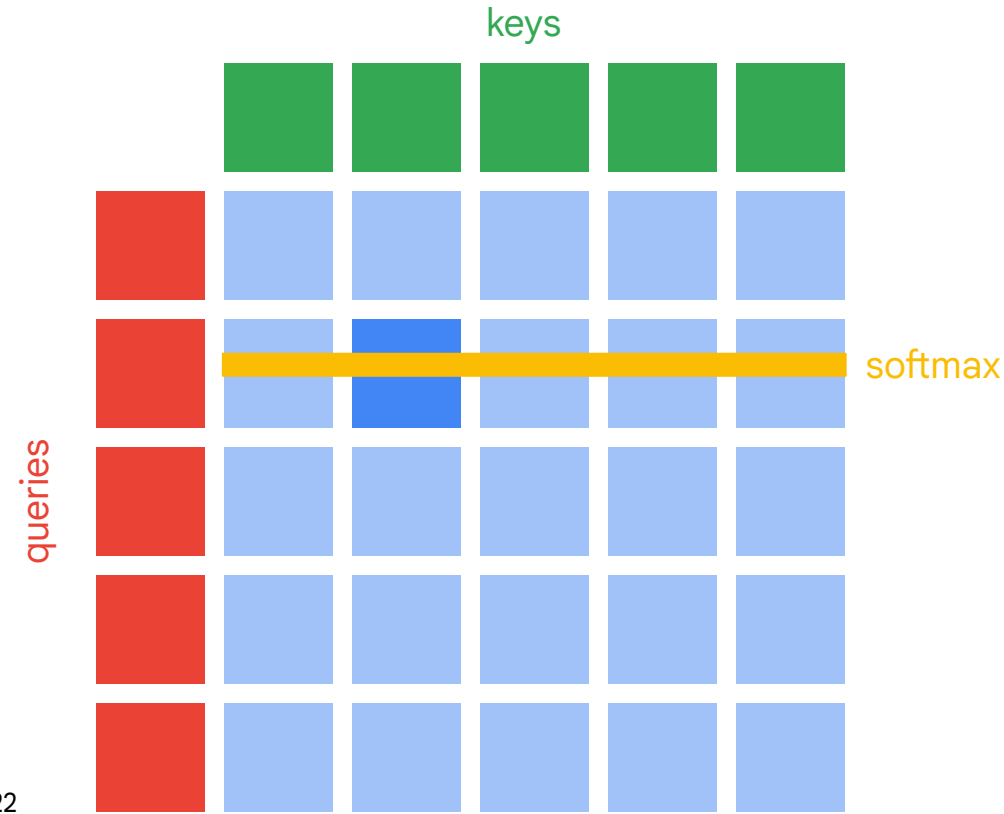
5:33 PM · Jan 25, 2023 · **92.3K** Views

13 Reposts **344** Likes **9** Bookmarks

https://twitter.com/tim_zaman/status/1618300981527334912

FlashAttention

- **Key idea:**
  never compute the full QK^T matrix
- Compute it one **tile** at a time
- Compute softmax normalisation **incrementally**
- Recompute tile in backward pass (**rematerialisation**)
- Speed-ups due to:
  kernel fusion, memory savings



`FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness', Dao, Fu, Ermon, Rudra, Re, NeurIPS 2022

QK normalisation

- In large models, O(10B), logits grow to extreme values
- Causes training instability
- **Simple fix:** LayerNorm on queries and keys

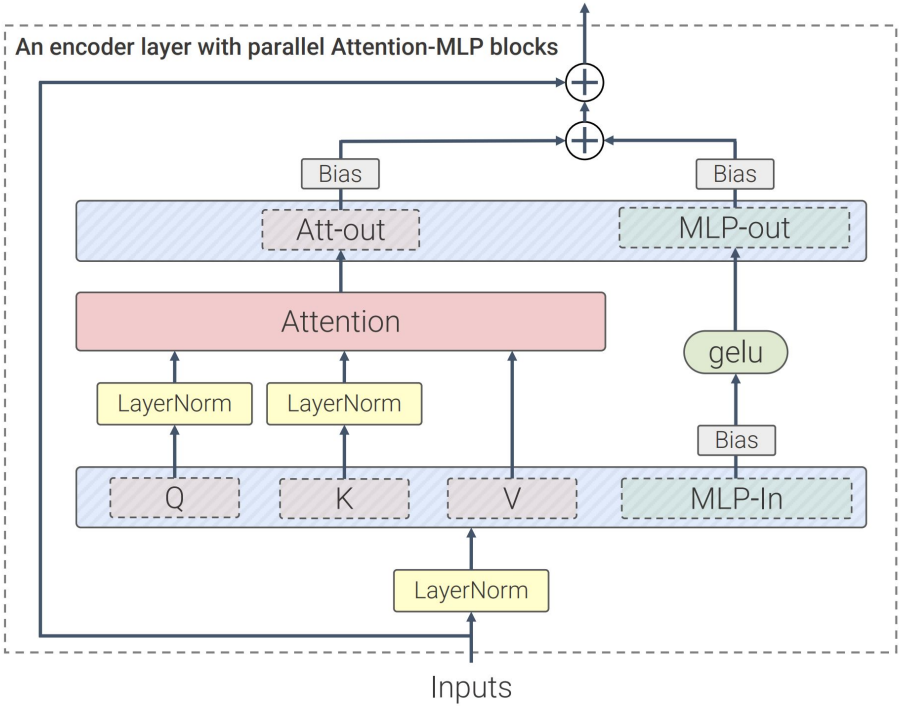


Figure 2: Parallel ViT-22B layer with QK normalization.

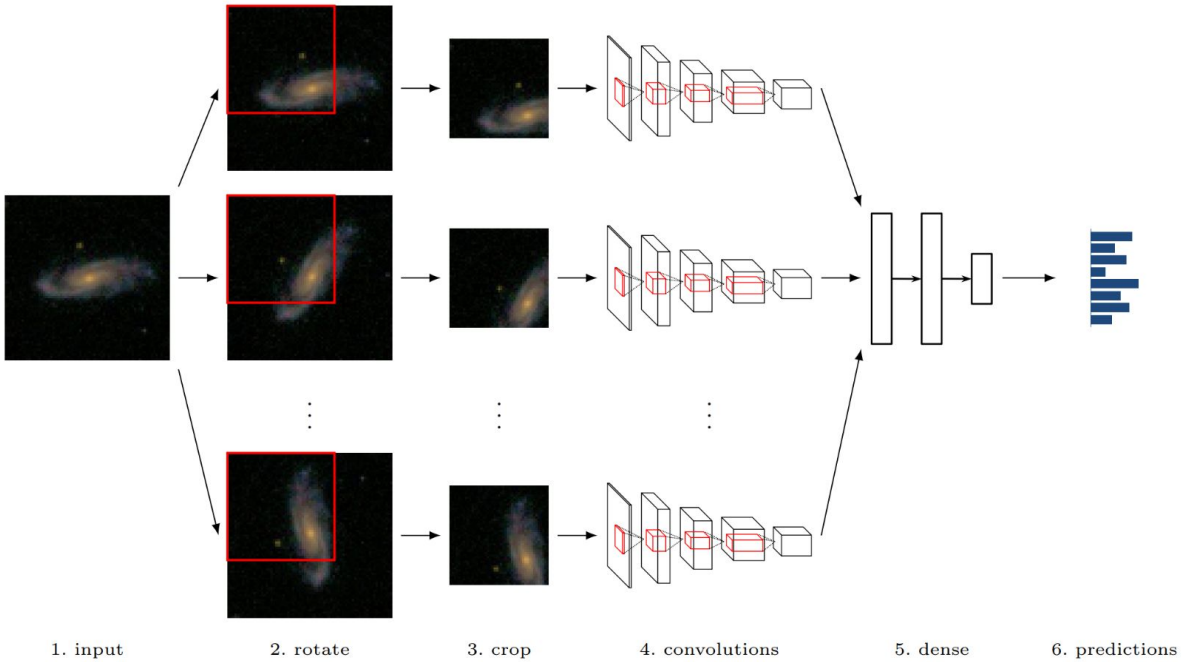
'Scaling Vision Transformers to 22 Billion Parameters', Dehghani et al., ICML 2023

Topology and symmetry

- Models for structured data: sequence, image (2D), video (3D), graph, ...
- Group convolutions, graph networks, **equivariance** and **invariance**
- Transformers have **no topological inductive bias**
- Positional embeddings

'Exploiting Cyclic Symmetry in Convolutional Neural Networks', Dieleman, De Fauw, Kavukcuoglu, ICML 2016
'Group Equivariant Convolutional Networks', Cohen, Welling, ICML 2016

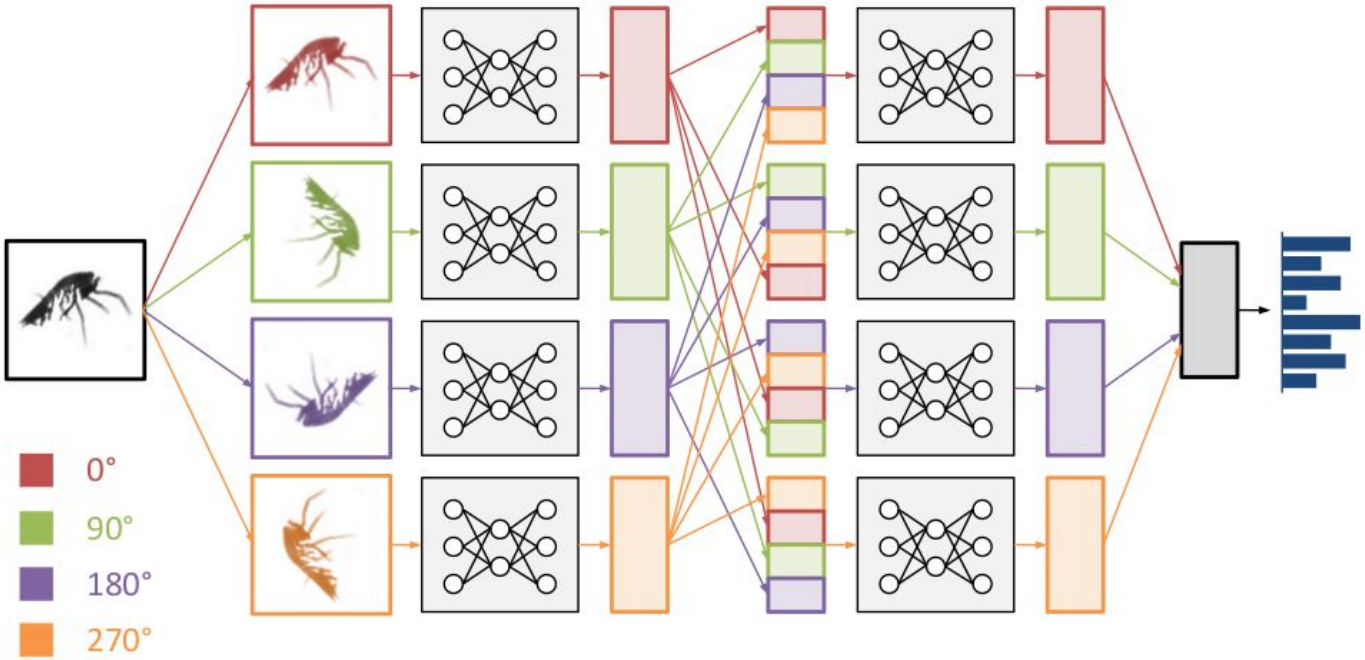
Example: galaxy morphology prediction



'Rotation-invariant convolutional neural networks for galaxy morphology prediction', Dieleman, Willett, Dambre, MNRAS 2015

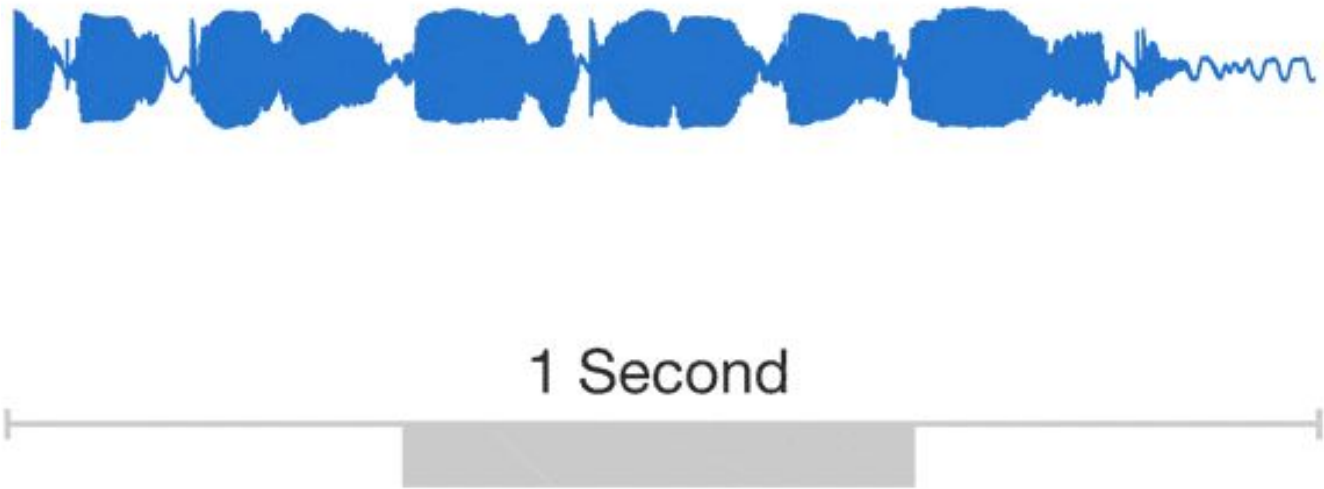
`https://sander.ai/2014/04/05/galaxy-zoo.html`

Example: plankton identification



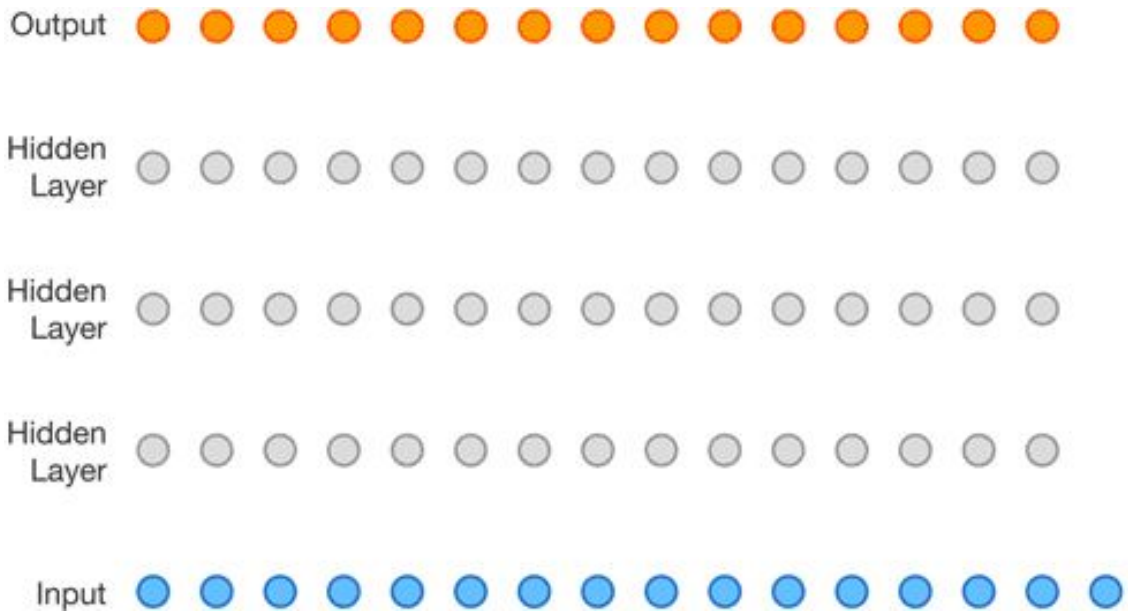
https://sander.ai/2015/03/17/plankton.html

Example: WaveNet



'WaveNet: a generative model for raw audio',
van den Oord, Dieleman, Zen, Simonyan, Vinyals, Graves, Kalchbrenner, Senior, Kavukcuoglu, arXiv 2016

Example: WaveNet

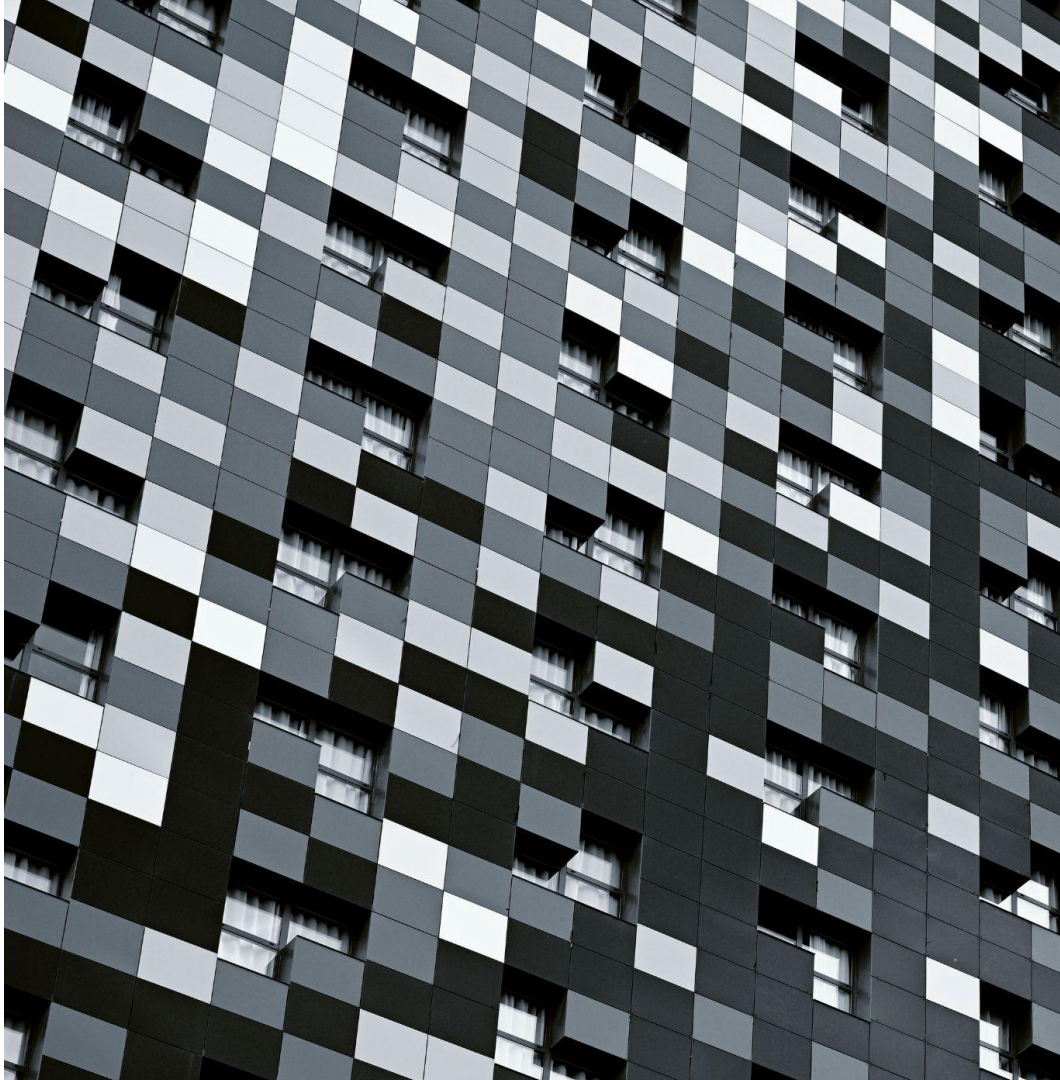


'WaveNet: a generative model for raw audio',
van den Oord, Dieleman, Zen, Simonyan, Vinyals, Graves, Kalchbrenner, Senior, Kavukcuoglu, arXiv 2016

Information bottlenecks

- Gumbel-softmax / Concrete distribution
- VQ (as in VQ-VAE)
- KL penalty (as in VAE)
- ...

'The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables', Maddison, Mnih, Teh, ICLR 2017
'Categorical Reparameterization with Gumbel-Softmax', Jang, Gu, Poole, ICLR 2017
'Neural Discrete Representation Learning', van den Oord, Vinyals, Kavukcuoglu, NeurIPS 2017
'Taming VAEs', Rezende & Viola, arXiv 2018



Vector quantisation

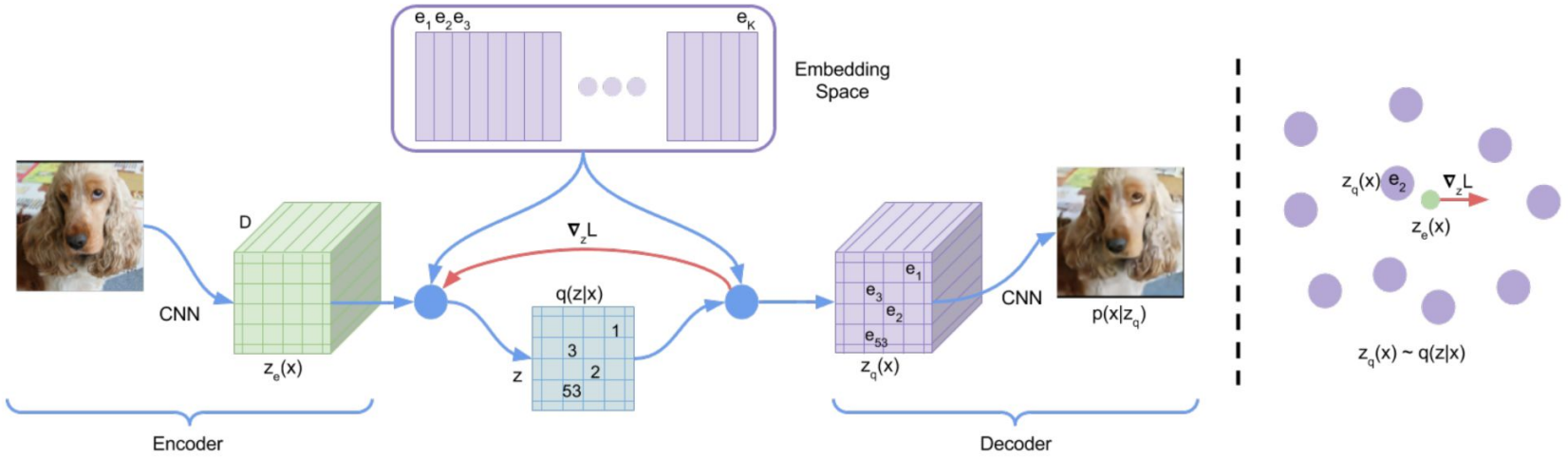
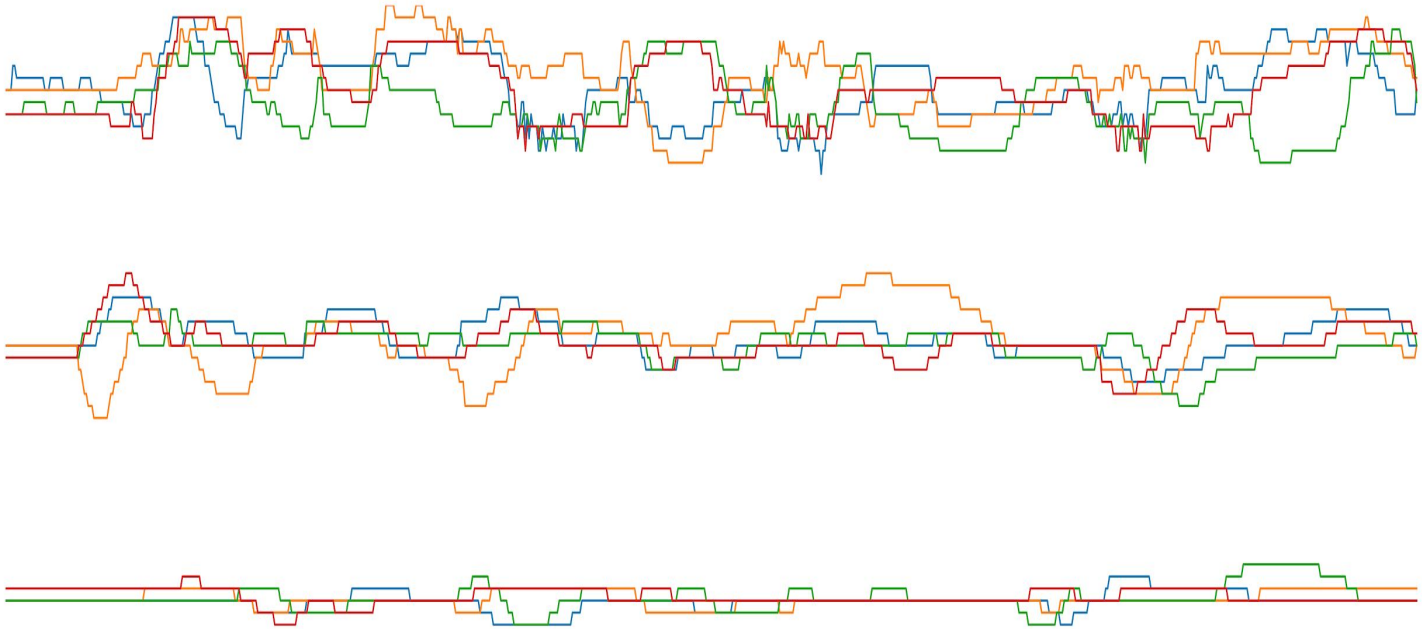


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2. The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

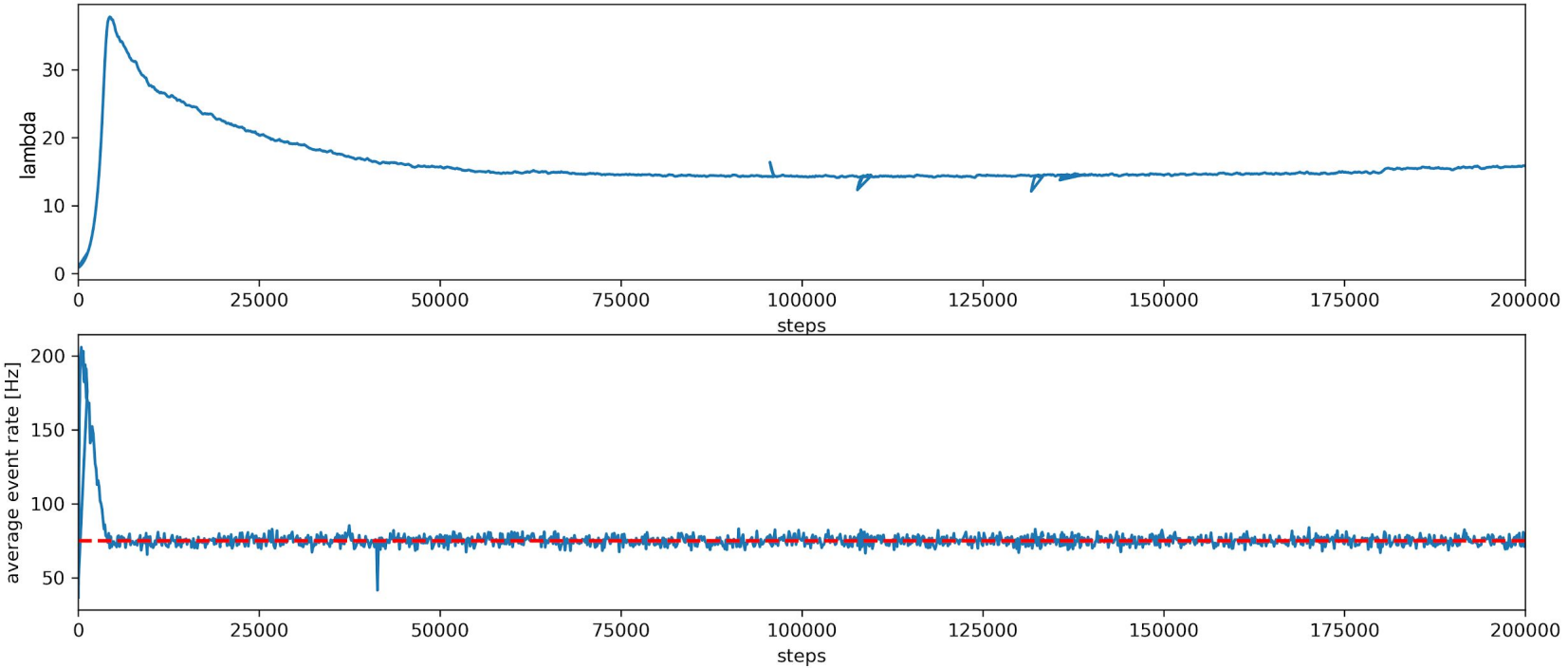
'Neural Discrete Representation Learning', van den Oord, Vinyals, Kavukcuoglu, NeurIPS 2017

Example: variable-rate representation learning



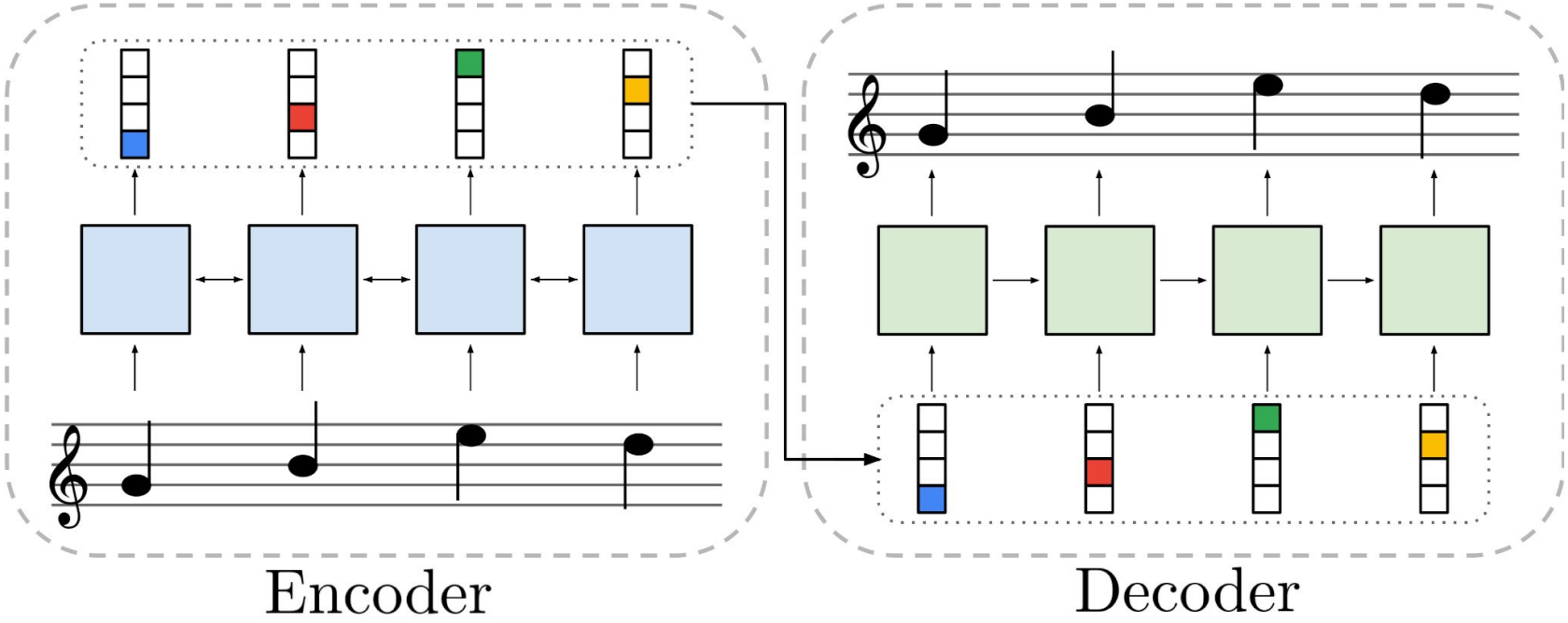
'Variable-rate Discrete Representation Learning', Dieleman, Nash, Engel, Simonyan, arXiv 2021

Example: variable-rate representation learning



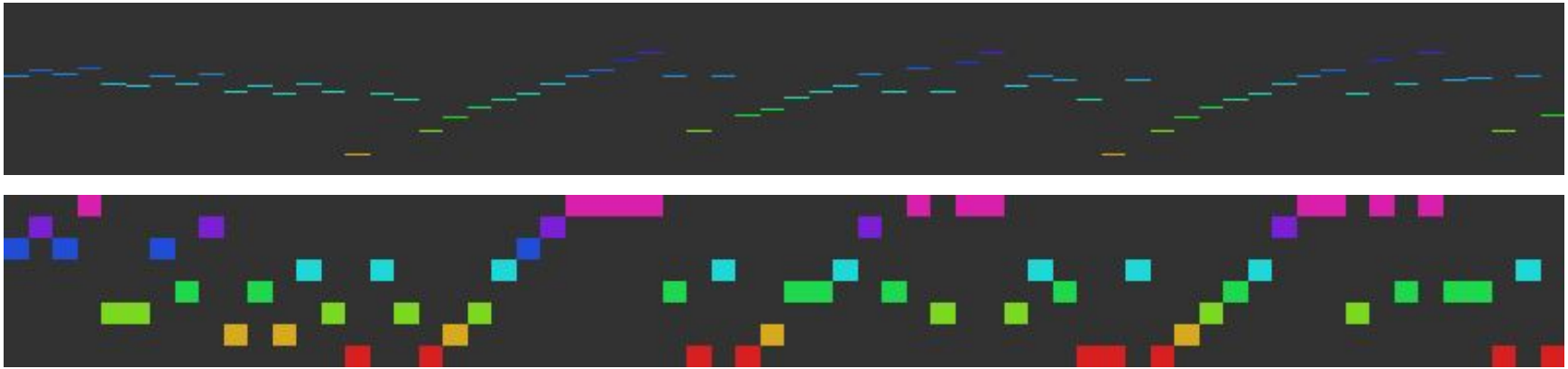
'Variable-rate Discrete Representation Learning', Dieleman, Nash, Engel, Simonyan, arXiv 2021

Example: Piano Genie



'Piano Genie', Donahue, Simon, Dieleman, IUI 2019

Example: Piano Genie



$$L = L_{\text{recons}} + L_{\text{margin}} + L_{\text{contour}}$$

$$L_{\text{recons}} = -\Sigma \log P_{\text{dec}}(\boldsymbol{x}|\text{enc}(\boldsymbol{x}))$$

$$L_{\text{margin}} = \Sigma \max(|\text{enc}_s(\boldsymbol{x})| - 1, 0)^2$$

$$L_{\text{contour}} = \Sigma \max(1 - \Delta\boldsymbol{x}\Delta\text{enc}_s(\boldsymbol{x}), 0)^2$$

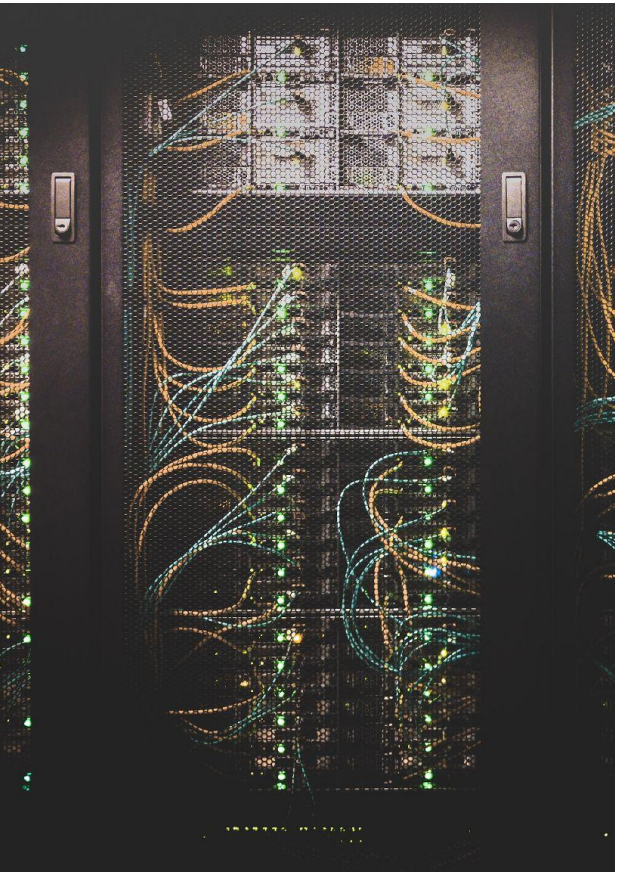
'Piano Genie', Donahue, Simon, Dieleman, IUI 2019

YAMAHA

Iterative refinement

- Generative modelling requires **deep computation graphs**
- **Divide and conquer:** split modelling task into steps
- Train very deep computation graphs without backprop all the way through
- **Autoregression** and **diffusion**

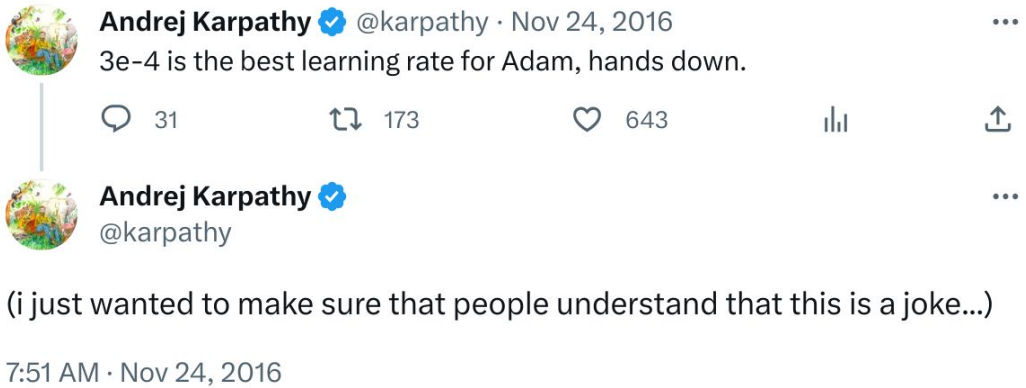
Google DeepMind



III. Training

Optimisers

- Unless you have a good reason not to, **start with Adam**
- At scale, tuning β_2 is important (Also for GANs)
- Treat ε as a hyperparameter as well
- Exotic optimisers are often application-specific (no free lunch), **benefits often disappear at scale**

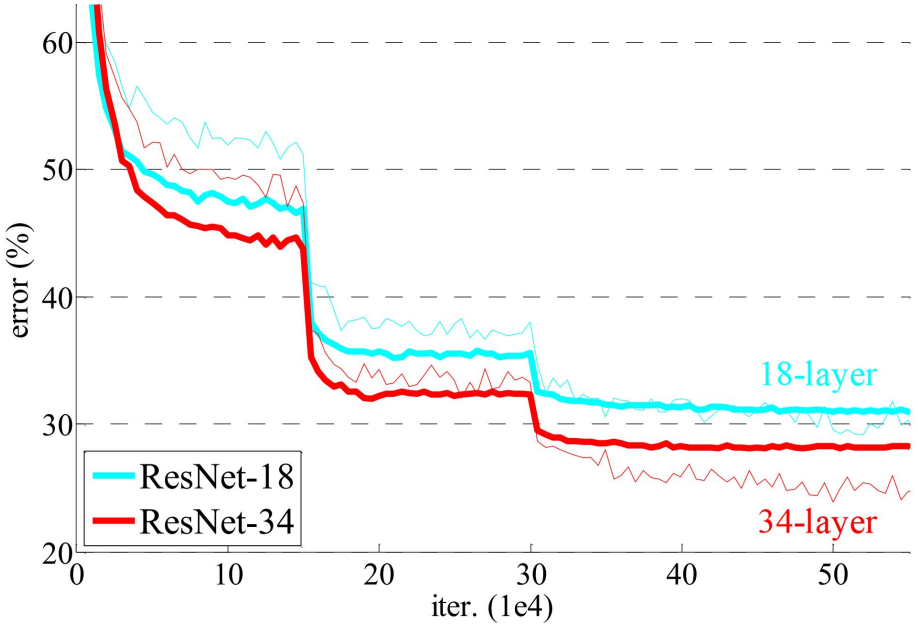


'Adam: A method for stochastic optimization', Kingma & Ba, ICLR 2015

https://twitter.com/karpathy/status/801694597009113088

Schedules

- Popular learning rate schedules:
  - discrete drops
  - cosine
  - warmup (for Transformers)
- Dropping the learning rate is **critical for performance**
- `annealing', lowering the temperature
  - Big steps: exploration
  - Small steps: exploitation

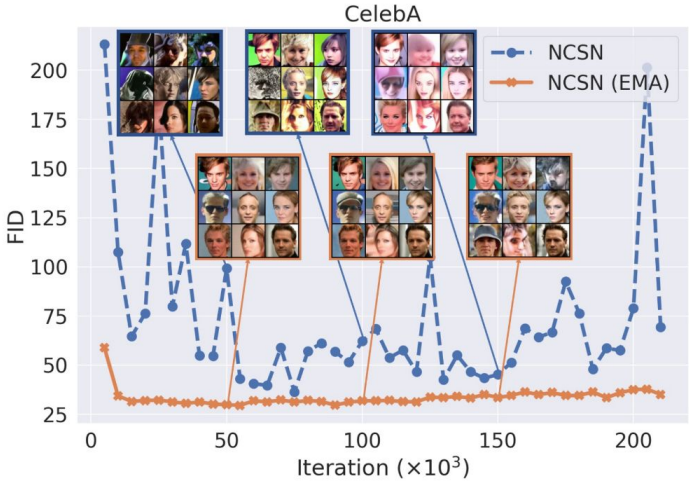


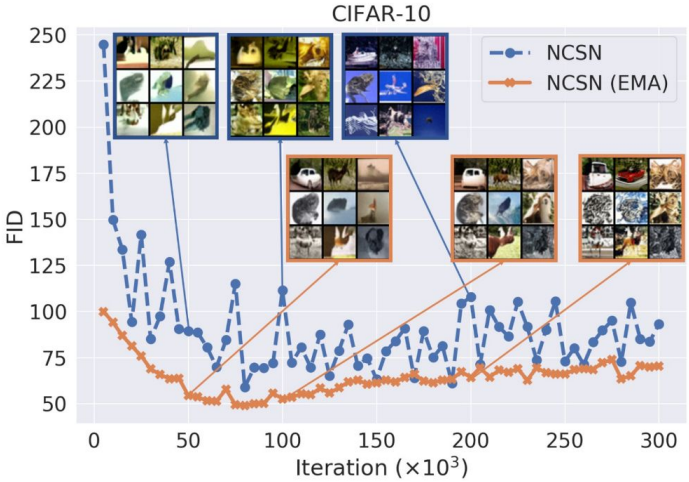
'Deep Residual Learning for Image Recognition', He, Zhang, Ren, Sun, CVPR 2016

Exponential moving average

- Update exponential moving average of weights during training (e.g. decay=0.9999)
- Use **averaged weights** for evaluation
- Significant memory cost
- Very effective, especially for GANs and diffusion models

'Improved Techniques for Training Score-Based Generative Models', Song & Ermon, NeurIPS 2020

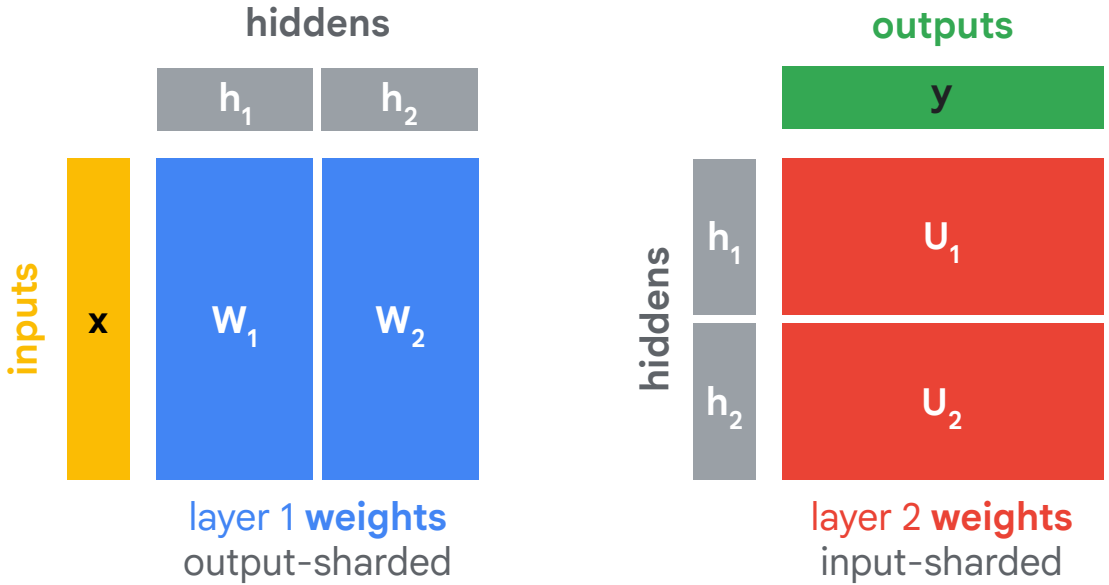




Scaling across devices: parallelism and sharding

- **Data parallelism** (split up the batch) is usually good enough below O(10B)
- Model parallelism is getting easier (e.g. `jax.jit`)
- Rule of thumb: **mimimise communication** across devices
- **Microbatching**: data parallelism in time

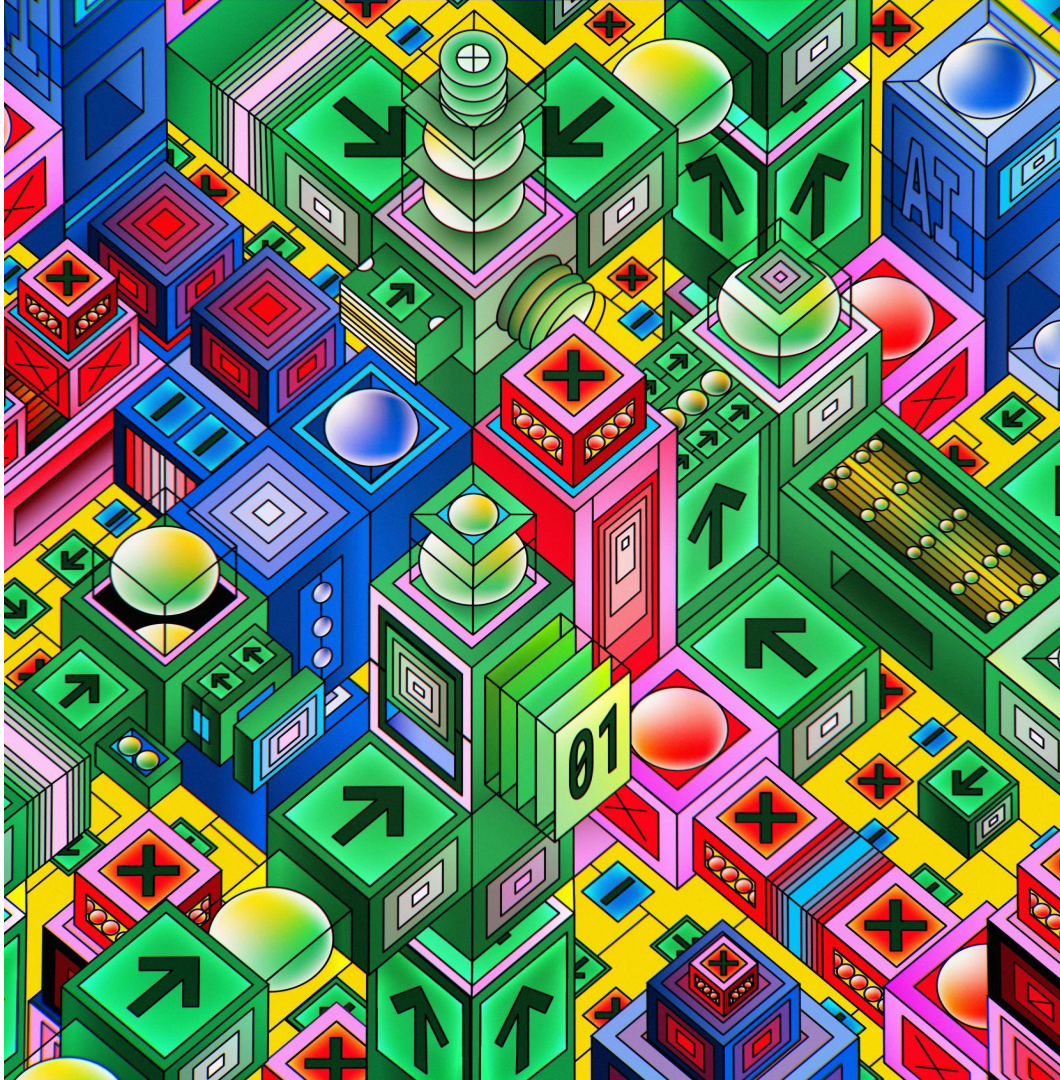
Example: Megatron sharding of an MLP



'Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism',
Shoeybi, Patwary, Puri, LeGresley, Casper, Catanzaro, arXiv 2019

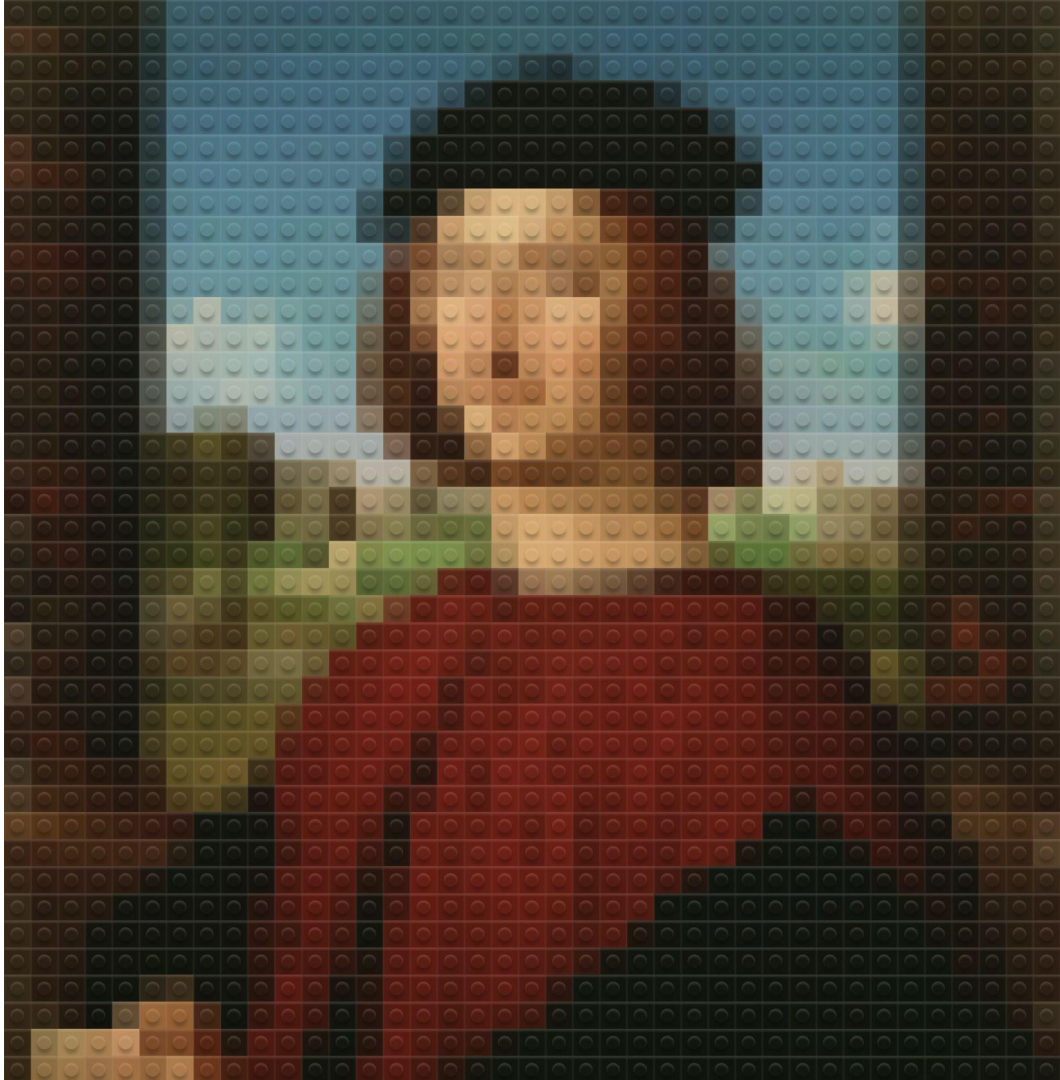
Rematerialisation

- on GPUs, **memory is slower than compute**
- Sometimes, **recomputation** is faster than fetching from memory
- Even if not, it **saves memory**
- Happens automatically in
- Fine-grained control: `jax.remat`



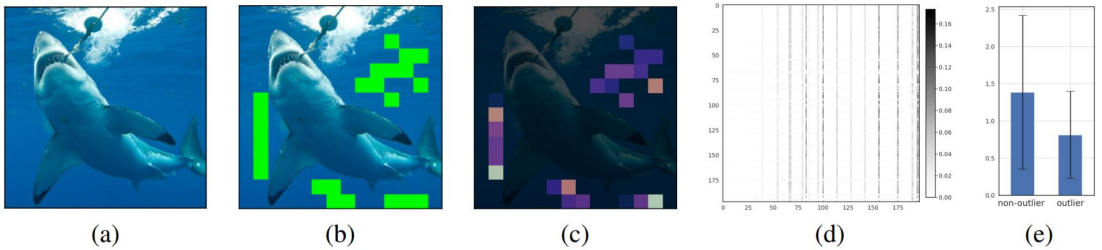
Low-precision arithmetic

- Training in **half precision** (`float16, bfloat16`) saves memory
- Avoid instability:
  - rescaling
  - stochastic rounding
- Quantisation
  - currently mostly done in post-processing ('PTQ')
  - weights, activations, or both
  - main challenge: **outliers**
  - quantised training is on the rise (`int8, int4`)



Example: quantizable Transformers

- Attention produces activation outliers in Transformers
- Softmax weights cannot all be zero, so optimisation finds a **workaround**
- Fix: clip softmax outputs, gating, or just **add a zero logit**



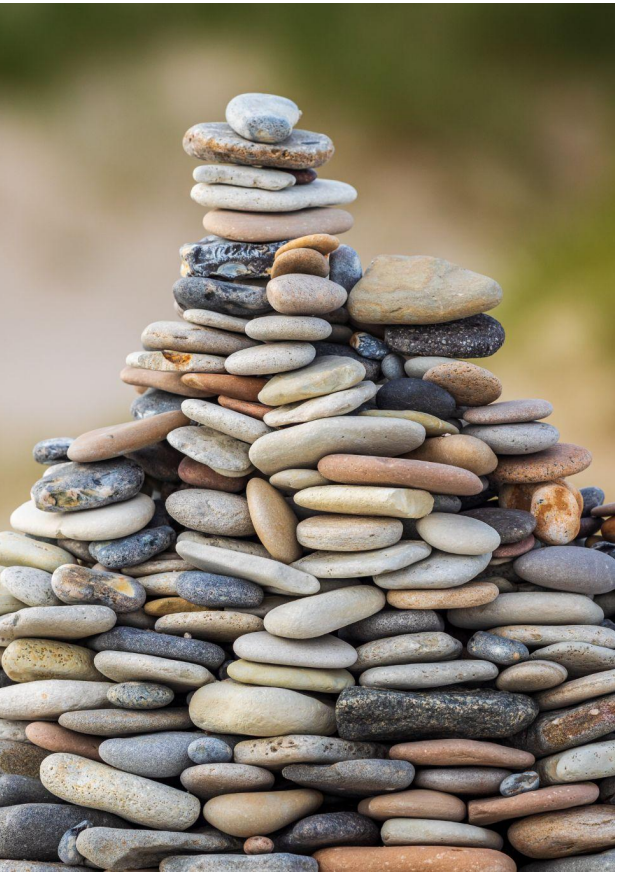
(a) (b) (c) (d) (e)

Figure 3: A summary of our outlier analysis for ViT demonstrated on a random image from ImageNet validation set. (a) An input image. (b) Outliers in the output of layer #11. (c) Cumulative attention weight spent on every patch (matrix of attention probabilities summed over rows) in the attention head #1, layer #12. (d) A corresponding matrix of attention probabilities. (e) An average magnitude of values for outlier and non-outlier patches.

'Quantizable Transformers: Removing Outliers by Helping Attention Heads Do Nothing', Bondarenko, Nagel, Blankevoort, arXiv 2023

`https://www.evanmiller.org/attention-is-off-by-one.html`

Google DeepMind



IV. Miscellany

Scaling laws

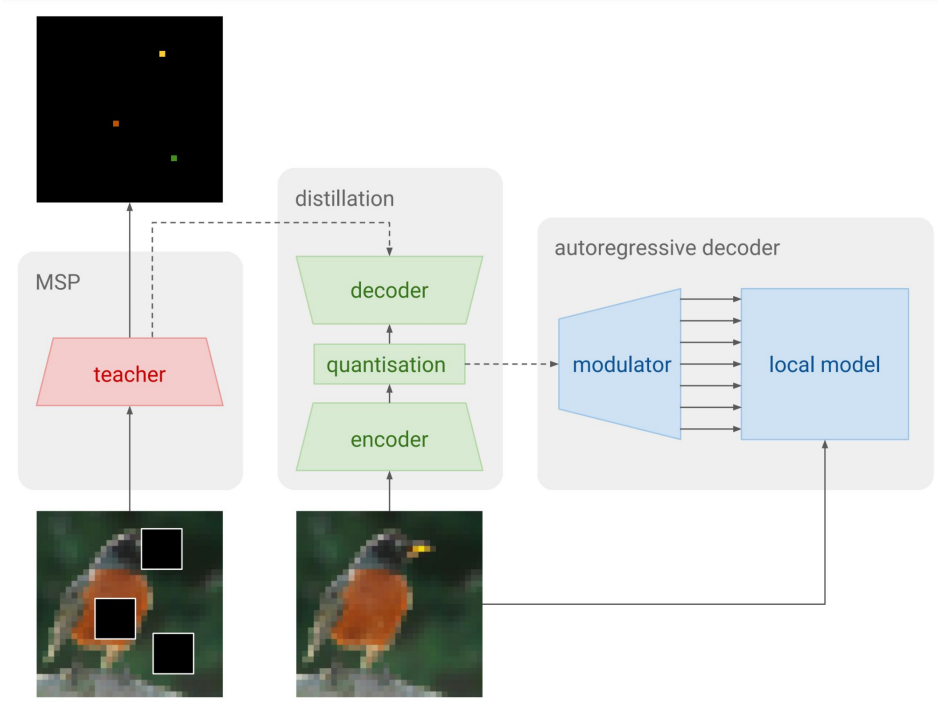
- Big models are too expensive to iterate on
- **Scaling laws** try to predict what will happen based on small-scale experiments
- Training behaviour can change quite dramatically with every 10x size increase

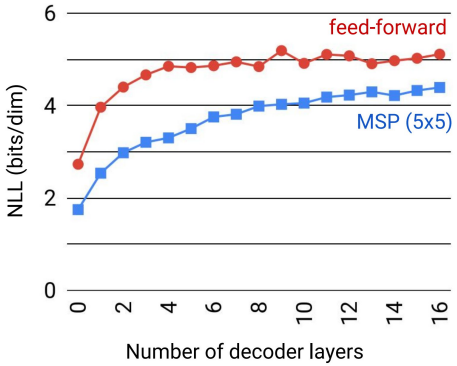
Distillation

- **Reduce model size**:
  distill larger model into smaller model
- **Born-again neural networks**:
  distill model into the same size model
- **Change architectures**:
  e.g. distill Transformer into RNN

'Distilling the Knowledge in a Neural Network', Hinton, Vinyals, Dean, NIPS 2014 Deep Learning workshop
'Born Again Neural Networks', Furlanello, Lipton, Tschannen, Itti, Anandkumar, ICML 2018

Example: masked self-prediction





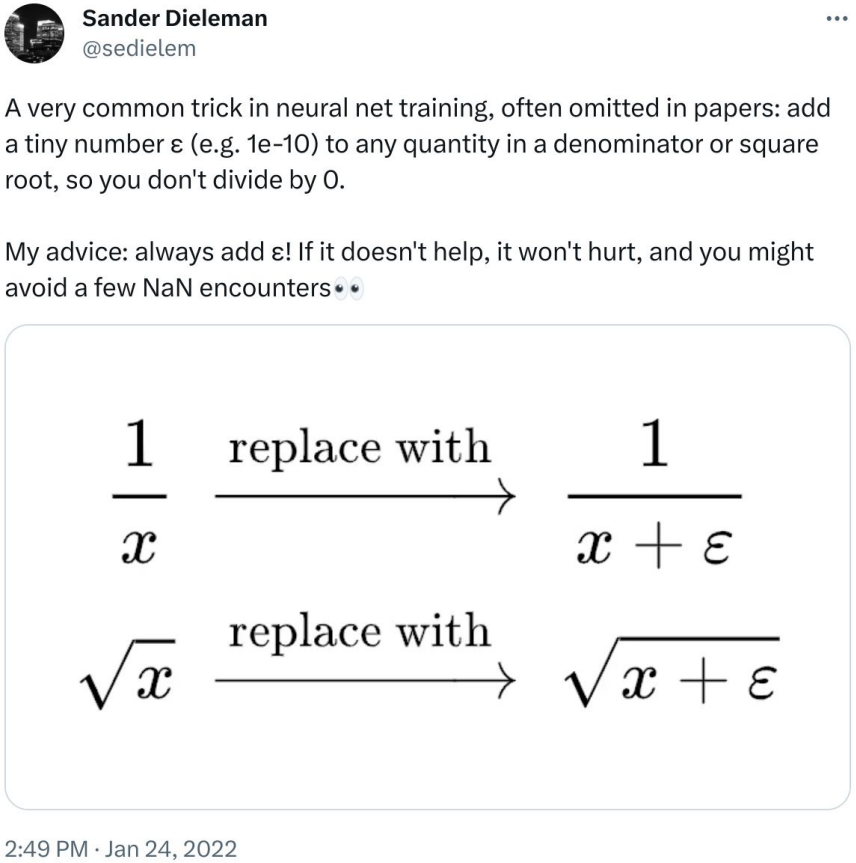
'Hierarchical Autoregressive Image Models with Auxiliary Decoders',
De Fauw, Dieleman, Simonyan, arXiv 2019

The luxury of autodiff

- **Automatic differentiation** is very powerful
- Understanding what goes on under the hood is worthwhile:
  - estimate computational cost
  - debugging
- Watch out with **gradients of gradients**
  Neural networks are **mostly linear**
  → second order gradients are **sparse**

Sprinkling epsilons

- Add ε in any **denominator** to avoid division by zero
- Add ε under any **square root** to avoid division by zero **in the gradient**
- Sometimes this becomes a hyperparameter, e.g. in Adam



https://x.com/sedielem/status/1485625814511198210

Neural nets have low Lipschitz constants

- **Lipschitz constant** of **f(x)**:
  if **x** changes by **Δ**, **f(x)** changes by at most **C·Δ**
- If **C** is small, **f(x)** can't be highly nonlinear
- Workaround: use positional encodings / random Fourier features
- ...or use activations functions like **sin(x)** (as in SIREN)
- Enforcing Lipschitzness: **spectral normalisation**

'Spectral Normalization for Generative Adversarial Networks',
Miyato, Kataoka, Koyama, Yoshida, ICLR 2018
'Implicit Neural Representations with Periodic Activation Functions',
Sitzmann, Martel, Bergman, Lindell, Wetzstein, NeurIPS 2020

Thank you! Grazie!

Blog: https://sander.ai/

Twitter/X: @sedielem

BlueSky: @sedielem.bsky.social

Email: sanderdieleman@gmail.com

Google DeepMind